



Fayoum University



كلية الحاسبات والذكاء الاصطناعي
Faculty of Computers and Artificial Intelligence
كلية معتمدة



ColorMate

Team Members

- Germeen George Halim
- Beshoy Gamal Wahba
- Youssef Sameh Awad
- Mariam Naeem Saeed
- Mina Magdy Neseem
- Youssef Nasan Sedhom

Supervised By

Dr. Mary Monir

- Eng. Khaled Ahmed
- Eng. Hossam Ahmed

2026

Acknowledgment

We would like to express our sincere gratitude to our supervisor, **Dr. Mary Monir**, for her continuous guidance, encouragement, and valuable feedback throughout every stage of this project. We are also deeply thankful to **Eng. Khaled Ahmed** and **Eng. Hossam Ahmed** for their technical insight and constructive advice during the development and implementation phases.

We would like to thank the **Faculty of Computers and Artificial Intelligence at Fayoum University** for providing the academic environment, resources, and support that made this work possible.

Finally, we extend our heartfelt appreciation to our families and friends for their patience, encouragement, and unwavering support throughout this journey.

Abstract

Color vision deficiency (CVD) affects an estimated 300 million people worldwide, creating daily challenges such as mismatched clothing, misread signals, and difficulty interpreting visual information. Existing assistive applications are typically narrow in scope, lacking the integrated depth needed to cover diagnosis, learning, and real-world assistance in a single platform. **ColorMate** addresses this gap through a smart mobile application that combines computer vision, deep learning, and AI-driven personalization into one cohesive assistive tool.

At the core of **ColorMate** are three specialized AI modules. The first is a Fruit Freshness Classification Model built on MobileNetV3Large, trained using a two-phase transfer learning strategy on a dataset of 28,645 images. The model first trains a custom classification head while the base is frozen, then fine-tunes all 187 layers end-to-end, achieving a final **test accuracy of 95.12%**, a **precision of 0.9523**, **recall of 0.9512**, **F1-score of 0.9511**, and an **AUC of 0.9781**. The second AI module is the **Perceptual Color Harmony System (PCHS)**, a context-aware scoring framework that evaluates how well outfit colors work together and generates recommendations for improvement. The third is an **Object Detection** Model based on **YOLOv8**, which detects and localizes objects and clothing items in real time before their colors are analyzed. Together, these models allow the system to assess produce quality, evaluate fashion color compatibility, and support object-level color awareness.

Beyond the AI core, the application provides an **Ishihara-based color blindness test** to identify a user's type and severity of **CVD**, an image color correction feature to enhance color distinction, and an AI-powered chatbot for guidance and education. The system is built on a three-tier architecture using **Flutter** for the mobile frontend, **ASP.NET Core Web API** for the backend, and **SQL Server** for data storage. **ColorMate** demonstrates that a deeply integrated, AI-driven approach can meaningfully improve daily independence for individuals with color vision deficiencies, with future work planned around multilingual support, real-time video detection, and augmented reality integration.

Table of Content

List of Figures	V
List of Tables	VII
Chapter 1: Introduction	1
1.1 Overview	2
1.2 Problem Definition	3
1.3 Proposed Solution.....	4
1.4 System Objectives	4
1.5 Project Scope	5
1.6 Project Limitations	5
1.7 Document Organization.....	5
Chapter 2: Project planning.....	8
2.1 Introduction	9
2.2 Project Objectives and Scope	10
2.3 Stakeholder Identification	10
2.4 Work Breakdown Structure	11
2.5 Project Schedule	11
2.6 Resource Allocation	12
2.7 Risk Management	12
2.8 Milestones and Deliverables.....	12
2.9 Monitoring and Evaluation.....	18
2.10 Conclusion	15
Chapter 3: Related work	19
3.1 Overview Of Existing Platforms	20
3.2 ColorMate	25
3.3 Why Our App Is Better Than Existing Systems.....	26
3.4 Datasets Used.....	27
3.5 Related Papers	31
3.6 Used Technologies	37
3.7 Conclusion	40
Chapter 4: system analysis	41
4.1 System Requirements	42

4.2 Use Case Analysis	46
4.3 Use Case Scenarios.....	47
4.4 Sequence Diagram	57
4.5 Use Case Diagram	62
4.6 Activity Diagram	63
4.7 Class Diagram.....	68
4.8 Entity Relationship Diagram	69
4.9 Context Diagram.....	70
4.10 Conclusion	71
 Chapter 5: System Architecture	72
5.1 Introduction.....	73
5.2 Logical Architecture (Three-Tier Architecture).....	73
5.3 AI & Computer Vision Architecture	73
5.4 Chatbot Architecture.....	74
5.5 Security Architecture	74
5.6 Deployment Architecture	74
5.7 System Architecture Overview.....	75
5.8 System Architecture Diagram	75
5.9 API Communication Architecture	76
5.10 Summary	77
 Chapter 6: System Implementation.....	78
6.1 Overview.....	79
6.2 System Architecture	79
6.3 Technologies Used.....	80
6.4 Backend Implementation Details.....	81
6.5 Database Implementation	81
6.6 AI Modules Implementation.....	81
6.7 Image Processing Modules	116
6.8 Ishihara Test Module	125
6.9 Game Life Cycle.....	128
6.10 Mobile App Implementation	132
6.11 System Modules Description.....	133
6.12 Security Implementation.....	133
6.13 Deployment & Hosting.....	133
6.14 Application Interface	134

6.15 Testing and Debugging.....	153
6.16 Challenges During Implementation.....	153
6.17 Conclusion	154

Chapter 7: Conclusion & Future Work.....155

7.1 Conclusion	156
7.2 Future Work	157

References159

List of figures:

Figure 1.1 Distribution of color blindness types.....	3
Figure 2.1: ColorMate MS Project.....	13
Figure 2.2: ColorMate MS Project Graph	14
Figure 2.3: ColorMate Jira Project	15
Figure 2.4: ColorMate Jira Project Graph	18
Figure 3.1 detecting the color of a specific area.....	21
Figure 3.2 Color simulation as perceived by a person with Deuteranomaly	23
Figure 3.3 Color simulation as perceived by a person with Tritanomaly	23
Figure 3.4 Original image under normal vision	23
Figure 3.5 Sample image from COCO (clock and household objects)	28
Figure 3.6 Sample image from COCO 2017 dataset (kitchen scene).....	28
Figure 3.7: Sample 1 DeepFashion dataset	29
Figure 3.8: Sample 2 DeepFashion dataset	29
Figure 3.9: Sample 3 DeepFashion dataset	30
Figure 3.10: Sample 4 DeepFashion dataset	30
Figure 3.11 Healthy apple sample from Fruit and Vegetable Disease dataset.....	31
Figure 3.12 Rotten apple sample from Fruit and Vegetable Disease dataset.....	31

Figure 4.1 Register Sequence Diagram	57
--	----

Figure 4.2 Login Sequence Diagram	58
Figure 4.3 Take Test Sequence Diagram.....	58
Figure 4.4 Detect Objects and their colors Sequence Diagram	59
Figure 4.5 Edit Profile Sequence Diagram	59
Figure 4.6 Confirm Password Sequence Diagram.....	60
Figure 4.7 Perform Color Correction Sequence Diagram	60
Figure 4.8 Rate Outfit and recommend color improvements Sequence Diagram..	61
Figure 4.9 Ask Chatbot questions Sequence Diagram	61
Figure 4.10 Use Case Diagram.....	62
Figure 4.11 Login and sign-up Activity Diagram	63
Figure 4.12 Take a test Activity Diagram	64
Figure 4.13 Edit Profile Activity Diagram	65
Figure 4.14 Perform color correction Activity Diagram	66
Figure 4.15 Rate Outfit and recommend color improvements Activity Diagram ..	67
Figure 4.16 Class Diagram	68
Figure 4.17 Entity Relationship Diagram.....	69
Figure 4.18 Context Diagram	70

Figure 6.1 ColorMate System Architecture	79
Figure 6.2 Fruit Model System Architecture.....	85
Figure 6.3 Fruit Model Architecture.	86
Figure 6.4 Fruit Model Training Curves.	89
Figure 6.5 Fruit Model Roc Curve.	90
Figure 6.6 Fruit Model Confusion Matrix.....	91
Figure 6.7 End-to-End Workflow of the Color Harmony Analysis System.	93
Figure 6.8 Color Harmony System Architecture.....	94
Figure 6.9 ColorMate Harmony Scoring Framework	97
Figure 6.10 ColorMate Recommendation Generation Pipeline	104
Figure 6.11 Human Perception vs ColorMate Score Alignment.	107
Figure 6.12 Object Detection Model Architecture.....	112
Figure 6.13 Image Simulation Workflow.....	120
Figure 6.14 Image Correction Workflow	123
Figure 6.15 Original Image	124
Figure 6.16 Simulated Image (Protan)	124
Figure 6.17 Corrected Image (Protan).....	124
Figure 6.18 Ishihara Test Diagnostic Workflow.....	127
Figure 6.19 Games Life Cycle.	131

Figure 6.20 Application Intro Screens.	136
Figure 6.21 Login Screen	137
Figure 6.22 Sign-Up Screen	138
Figure 6.23 OTP Verify Screen.....	138
Figure 6.24 Forget Password Screen.....	139
Figure 6.25 Home Screen	140
Figure 6.26 Start Test Screen	141
Figure 6.27 Test Screen.....	141
Figure 6.28 Test Result Screen.....	141
Figure 6.29 Upload Image Screen.....	142
Figure 6.30 Object & Color Detection Result Screen	142
Figure 6.31 Upload Outfit Image and The Color Matching Result Screen.....	143
Figure 6.32 Upload Fruit Image Screen	144
Figure 6.33 Fruit Quality Check Screen.....	144
Figure 6.34 Chat bot Chatting Screen	145
Figure 6.35 User Profile Screen	146
Figure 6.36 Edit Profile Screen	146
Figure 6.37 Change Password Screen	146
Figure 6.38 Simulated Image in Protan Screen.....	148
Figure 6.39 Corrected Image from Protan Screen.....	148
Figure 6.40 Select Game Screen.	149
Figure 6.41 Color Collector Game Screens.....	149
Figure 6.42 Memory Match Game Screens.....	150
Figure 6.43 Color the Picture Game Screens.	151
Figure 6.44 Sequence Game Screen.....	152
Figure 6.45 Find the Object Game Screen.	152

List of tables:

Table 2.1 Milestones and Deliverables	12
Table 3.1 Comparison between existing systems and our application.....	26
Table 3.2 Related Work.....	35
Table 4.1 Register use case analysis	48
Table 4.2 Login use case analysis.....	49
Table 4.3 Edit Profile use case analysis.....	50
Table 4.4 Detect Objects and their colors use case analysis.....	51
Table 4.5 Identify color blindness test use case analysis.....	52
Table 4.6 Perform color correction Profile use case analysis.....	53

Table 4.7 Rate Outfit Sequence Diagram use case analysis	54
Table 4.8 Ask Chatbot use case analysis	55
Table 4.9 Logout use case analysis.....	56
Table 5.1 Key API endpoints	76
Table 6.1 ColorMate Technologies Used.....	80
Table 6.2 Fruit and Vegetable Dataset Classes	83
Table 6.3 Fruit and Vegetable Dataset Spilt.....	83
Table 6.4 Training Hyperparameters and Configuration	87
Table 6.5 Fruit Model Results Table.....	88
Table 6.6 Our Fruit Model and Paper Model Comparison Table.	92
Table 6.7 Human Perception vs ColorMate Score Alignment.....	108
Table 6.8 Quantitative Harmony Score Comparison	109
Table 6.9 Cross-Platform LLM Benchmarking Results.....	111
Table 6.10 Architectural Enhancements and Design Rationales.....	112

— **Introduction** —

Chapter One

1.1 Overview

ColorMate is a smart mobile application designed to assist individuals with color vision deficiencies (CVD) by providing tools that make it easier to interact with colors in daily life. The application combines computer vision and artificial intelligence to offer practical solutions such as color vision testing, object & color detection, image adjustment for enhanced color differentiation, fruits quality check, and interactive educational features.

Color vision deficiency (CVD) affects approximately 300 million people worldwide. The distribution of CVD types is as follows: Deutan (green deficiency) is the most common, affecting around 72% of color-blind individuals, followed by Protan (red deficiency) at approximately 28%. Tritan (blue deficiency) is significantly rarer at around 0.6%, and Monochromacy (complete color blindness) is the rarest type, affecting only about 0.1% of those with CVD, occurring in roughly 1 in 33,000 people globally. Among the general population, red-green color blindness affects approximately 8% of males and 0.5% of females.

1.1.1 Types of Color Vision Deficiency

Color vision deficiency is classified into four main types based on which cone photoreceptors in the eye are affected:

- **Protan (Red Deficiency)**
Protan color blindness results from defective L-cones (long-wavelength sensitive cones), which are responsible for detecting red light. It includes two subtypes: Protanomaly (reduced red sensitivity) and Protanopia (complete absence of red cone function). Individuals with this type often perceive red, orange, yellow, and green as similar colors. Red colors may appear darker than usual, purple may appear blue, and pink may appear gray.
- **Deutan (Green Deficiency)**
Deutan color blindness is caused by defective M-cones (medium-wavelength sensitive cones). It is the most common form of color blindness and includes Deuteranomaly and Deuteranopia. Affected individuals often confuse green and yellow, may find blue and purple difficult to distinguish, and may perceive pink as gray or white.
- **Tritan (Blue Deficiency)**
Tritan color blindness stems from defective S-cones (short-wavelength

sensitive cones). Unlike Protan and Deutan types, Tritan deficiency often develops later in life and is rarely inherited. It includes Tritanomaly and Tritanopia. Individuals with this type struggle to differentiate blue and green, and blue colors in general are difficult to perceive.

- **Monochromacy (Complete Color Blindness)**

Monochromacy is an extremely rare condition in which the individual cannot perceive any color at all. Vision appears entirely in shades of gray, ranging from black to white. The main subtypes are Monochromacy and Achromatopsia. This condition affects approximately 1 in 33,000 people.

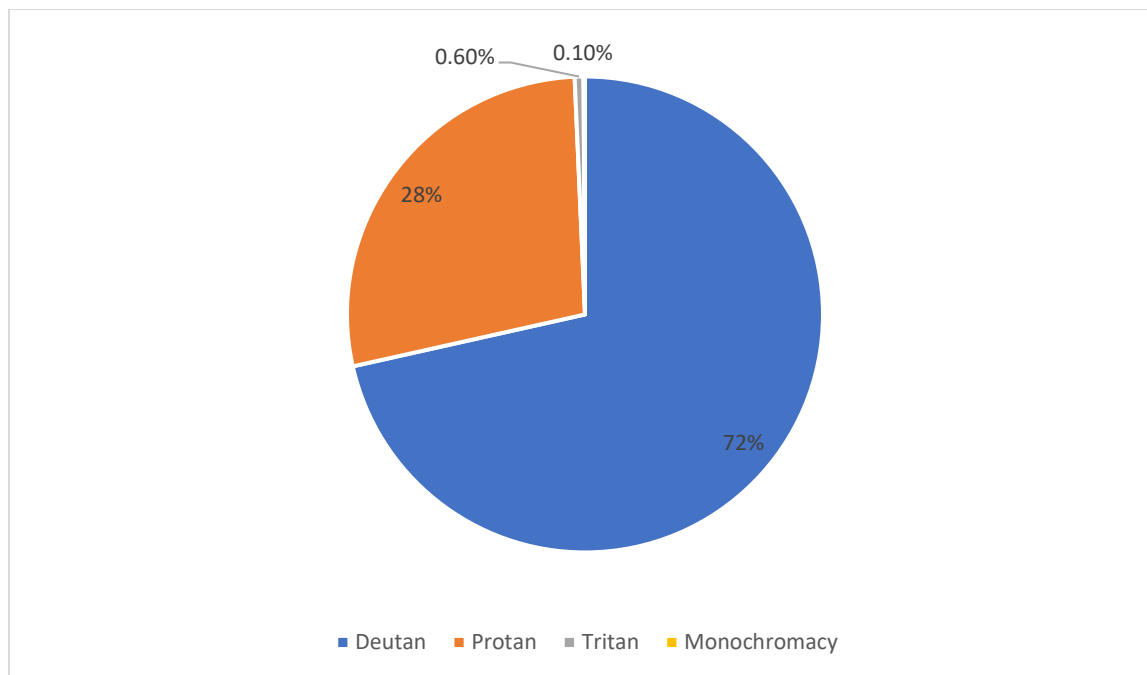


Figure 1.1 Distribution of color blindness types

Reference : [\[1\]](#)

1.2 Problem Definition

People with color blindness experience difficulty distinguishing certain colors or shades, leading to daily challenges such as mismatched clothing, misinterpreting traffic signals, or reading charts and graphs. While there are existing applications, most of them lack precision and do not provide a comprehensive experience that covers diagnosis, learning, and real-world assistance.

Hence, the main problem is the absence of an integrated solution that helps

users both identify their color vision condition and adapt effectively to their surroundings.

1.3 Proposed Solution

This project proposes a smart system named ColorMate, which integrates artificial intelligence and image processing to provide a set of connected features, including:

- Performing a test to identify the type of color vision deficiency.
- Object & color detection using the mobile camera.
- Adjusting images to enhance color distinction without altering the user's natural color perception.
- A clothing assistant that helps users match colors effectively.
- An educational and interactive section for raising awareness about color vision.
- A smart chatbot to support and guide users through the app's features.

1.4 System Objectives

1. Identify the type and severity of color vision deficiency for each user.
2. Provide an accurate color recognition tool.
3. Enhance users' ability to distinguish colors through visual image adjustments.
4. Raise awareness about color blindness through educational and interactive content.
5. Enable users to scan fruits and assess their freshness and quality based on visual and color analysis.
6. Design a simple and user-friendly interface suitable for all age groups.
7. Integrate AI-based personalization to improve user interaction.

1.5 Project Scope

1.5.1 In-Scope

- Testing color vision condition.
- Object & color detection feature.
- Image adjustment to improve color distinction.
- AI-based smart chatbot assistant.
- Educational and awareness sections.
- Scanning fruits to evaluate freshness.
- Responsive and accessible UI design.

1.5.2 Out-of-Scope

- Enabling users to see colors as normally sighted people do (since the app cannot change actual visual perception).
- Integration with medical or wearable devices.
- Official medical diagnosis or treatment — the app only provides visual assistance.

1.6 Project Limitations

- Accuracy depends on camera quality and lighting conditions.
- The application is not a medical diagnostic tool.
- Limited to mobile platforms in its initial version.
- Requires an internet connection for some AI-based features.

1.7 Document Organization

Chapter 2: Project Planning:

This chapter will include project plan, the feasibility study, risk identification and team structure.

Chapter 3: Literature Survey:

This chapter includes more information about used fields and technologies in the implementation, and Competitors who have ideas that are similar to our idea and their advantages & disadvantages.

Chapter 4: System Analysis:

This chapter discusses function requirements such as use case diagram, Sequence diagrams, System sequence and discuss non-function requirement such as Supplementary specification requirements.

Chapter 5: System Architecture:

This chapter presents the overall structure of the ColorMate system. It explains the architectural design, including the three-tier architecture (presentation layer, application layer, and data layer), and describes how different components such as the mobile application, backend services, database, and AI modules interact with each other.

Chapter 6: Implementation:

This chapter describes the technologies and tools used to develop the system, including Flutter, ASP.NET Core, and machine learning models. It also explains the implementation of key features such as color detection,

object recognition, and chatbot functionality. In addition, it covers the testing methods used to ensure system reliability and performance.

Chapter 7: Conclusion and Future Work:

This chapter summarizes the project outcomes and highlights the main achievements of the system. It also discusses the limitations and challenges faced during development and provides suggestions for future improvements and enhancements.

Chapter two

2.1 Introduction:

ColorMate is a smart mobile application designed to assist individuals with color vision deficiencies (color blindness) by integrating digital color vision testing, AI-supported image analysis, and color correction tools. The app allows users to detect their type of color blindness through structured test (Ishihara test). It also enables users to upload images for automatic object and color detection, helping them better understand color composition. Additionally, ColorMate offers a clothing color-matching assistant, a chatbot for guidance, a fruit scanning feature that evaluates fruit freshness, and educational resources about color vision.

2.2 Project Objectives and Scope:

Objective:

The primary goal of ColorMate is to develop an intelligent mobile application that assists users in detecting, understanding, and adapting to color vision deficiencies through logical test evaluation, AI-based image color detection, and educational tools.

Scope:

- Development of a mobile application for detecting and classifying color blindness using **logical-based tests** (Ishihara test).
- Integration of **AI-based image object and color detection**.
- Implementation of **image color correction filters** to enhance visibility for color-blind users.
- Inclusion of an **Outfit Color Matching Assistant** to suggest suitable clothing color combinations.
- Implementation of a **fruit scanning** feature that analyzes fruit color and visual characteristics using AI techniques to assess freshness and quality.
- Addition of an **educational and awareness section** about color blindness types and daily adaptation strategies.
- Development of a **smart chatbot assistant** to explain results and offer personalized guidance.

2.3 Stakeholder Identification:

Primary Stakeholders:

- **End Users:** Individuals with color vision deficiencies who use the app for detection, education, and color guidance.

2.4 Work Breakdown Structure (WBS):

ColorMate is structured into the following key phases:

2.3.1 Planning Phase (02-Sep-2025 – 1-Oct-2025) [1 Month]

- Define goals, features, and project deliverables.
- Conduct research on color blindness types, testing methods, and AI image detection.
- Prepare project timeline and documentation plan.

2.3.2 Analysis Phase (2-Oct-2025 – 26-Oct-2025) [1 Month]

- Collect functional and non-functional requirements.
- Data Collection and Cleaning.
- Design the logic for Ishihara test result calculation.

2.3.3 Design Phase (10-Oct-2025 – 11-Nov-2025) [1 Month]

- Develop system architecture and database structure.
- Design UI/UX layouts for testing, color detection, and learning modules.
- Functional & Behavioral Design (Use-Case, Activity, Sequence Diagrams).

2.3.4 Development Phase (12-Nov-2025 – 14-Apr-2026) [5 Months]

- AI Development (Object & Color Detection and Clothing Color Matching).
- Backend Development (Database, APIs).
- Mobile Application Development.

2.3.5 Testing Phase (15-Apr-2026 – 8-May-2026) [3 Weeks]

- Conduct unit, integration, and usability testing.
- Validate test result accuracy and color detection reliability.

2.3.6 Deployment Phase (9-May-2026 – 15-May-2026) [1 Week]

- Application Deployment and Final Review.

2.5 Project Schedule:

- **Start Date:** 02-Sep-2025
- **End Date:** 15-May-2026
- **Total Duration:** 9 Months
- **Timeline:** Matches the phases outlined in the Work Breakdown Structure.

2.6 Resource Allocation:

Human Resources:

- Flutter Developers – Frontend design and navigation.
- .NET Developers – Logic and backend management.
- AI Developers – Image detection and color recognition module.
- Project Supervisor – Guidance and progress evaluation.

Technical Resources:

- Cloud Storage and Hosting Services.
- AI Models for Object and Color Detection and Clothing Color Matching.
- API and Database Infrastructure.

2.7 Risk Management:

Potential Risks and Mitigation Strategies:

- **AI Model Accuracy:** Use high-quality datasets and perform regular validation to ensure reliable color and object detection in images.
- **Logic Calculation Errors:** Review and test all color blindness test formulas to confirm accurate result computation.
- **Integration Issues:** Conduct early integration testing between Flutter, .NET, and AI modules to avoid compatibility errors.
- **Time Management:** Follow Agile methodology with weekly sprints and milestone tracking to prevent delays.
- **Data Loss:** Maintain version control using GitHub and regular cloud backups.

2.8 Milestones and Deliverables:

The following table outlines the main project milestones, their corresponding deliverables, and the scheduled completion dates for each phase of the project:

Table 2.1 Milestones and Deliverables

Milestone	Deliverable	Completion Date
Project Planning	Project Plan Document	1-Oct-2025
System Design	Architecture and UI/UX Layout	11-Nov-2025
Backend Development	Functional API & Database	10-Feb-2026
Mobile Development	Beta Application	14-Apr-2026
Testing & Debugging	Fully Tested App	2-May-2026
Final Deployment	Final App Build and Presentation	9-May-2026

2.9 Monitoring and Evaluation:

- Weekly sprints and team updates using Trello or Jira.
- Code and logic validation by supervisors.
- AI module testing with sample images for accuracy.
- User interface testing for accessibility and clarity.
- Continuous quality assurance before deployment.

Task Name	Duration	Start	Finish
Documentation	9 mons	Tue 9/2/25	Sat 5/9/26
Planning	1.1 mons	Tue 9/2/25	Wed 10/1/25
Define Project Scope	1.1 mons	Tue 9/2/25	Wed 10/1/25
Conduct Research on Color Blindness Types and Testing Methods	3 days	Tue 9/2/25	Thu 9/4/25
Define Deliverables and Acceptance Criteria	2 days	Fri 9/5/25	Sun 9/7/25
Define Scope	3 days	Mon 9/8/25	Wed 9/10/25
Project Objectives	10 days	Thu 9/11/25	Wed 9/24/25
Create Project Plan	3 days	Thu 9/25/25	Sat 9/27/25
Divide Project Into Tasks	4 days	Sun 9/28/25	Wed 10/1/25
Planning Completed	0 days	Wed 10/1/25	Wed 10/1/25
Analysis	0.9 mons	Thu 10/2/25	Sun 10/26/25
Collect Functional and non-functional Requirements	1.2 wks	Thu 10/2/25	Thu 10/9/25
Data Collection and Cleaning	2.4 wks	Fri 10/10/25	Sun 10/26/25
Analysis Completed	0 days	Sun 10/26/25	Sun 10/26/25
Design	1.15 mons	Fri 10/10/25	Tue 11/11/25
System Design	1.2 wks	Fri 10/10/25	Fri 10/17/25
ERD Model	2 days	Fri 10/10/25	Sat 10/11/25
Class Diagram	1 day	Sun 10/12/25	Sun 10/12/25
Relational Database Diagram	2 days	Mon 10/13/25	Tue 10/14/25
Database Design	3 days	Wed 10/15/25	Fri 10/17/25
Functional & Behavioral Design	3 days	Sat 10/18/25	Tue 10/21/25
Use-Case Diagram	2 days	Sat 10/18/25	Sun 10/19/25
Context Diagram	1 day	Sat 10/18/25	Sat 10/18/25
Activity Diagram	2 days	Mon 10/20/25	Tue 10/21/25
Sequence Diagram	3 days	Sun 10/19/25	Tue 10/21/25
UI/UX Design	15 days	Wed 10/22/25	Tue 11/11/25
Logo Design	3 days	Wed 10/22/25	Fri 10/24/25
User Interface Design	7 days	Sat 10/25/25	Mon 11/3/25
User Experience Design	6 days	Tue 11/4/25	Tue 11/11/25
Design Completed	0 days	Tue 11/11/25	Tue 11/11/25
Development	5.5 mons	Wed 11/12/25	Tue 4/14/26
AI Development	1.55 mons	Wed 11/12/25	Wed 12/24/25
Implement Object & Color Detection Model	7 days	Wed 11/12/25	Thu 11/20/25
Implement Clothing Color Matching Model	8 days	Fri 11/21/25	Tue 12/2/25
Review Functional Specifications	11 days	Wed 12/3/25	Wed 12/17/25
Review non-functional Specifications	5 days	Thu 12/18/25	Wed 12/24/25
Backend Development	2.65 mons	Wed 11/12/25	Fri 1/23/26
Database Schema Implementation	1.15 mons	Wed 11/12/25	Fri 12/12/25
API Implementation	12 days	Sat 12/13/25	Mon 12/29/25
API Testing & Debugging	7 days	Tue 12/30/25	Wed 1/7/26
Backend Security & Performance Optimization	6 days	Thu 1/8/26	Thu 1/15/26
Backend Deployment	6 days	Fri 1/16/26	Fri 1/23/26
Mobile Development	2.9 mons	Sat 1/24/26	Tue 4/14/26
UI/UX Implementation	13 days	Sat 1/24/26	Tue 2/10/26
State Management Design and Implementation	13 days	Wed 2/11/26	Fri 2/27/26
API Integration	12 days	Sat 2/28/26	Sat 3/14/26
AI Integration	13 days	Sun 3/15/26	Tue 3/31/26
Mobile Deployment	10 days	Wed 4/1/26	Tue 4/14/26
Development Completed	0 days	Tue 4/14/26	Tue 4/14/26
Testing	0.9 mons	Wed 4/15/26	Fri 5/8/26
Validate Test Result Accuracy	6 days	Wed 4/15/26	Wed 4/22/26
Unit Testing	6 days	Thu 4/23/26	Thu 4/30/26
Integration Testing	6 days	Fri 5/1/26	Fri 5/8/26
Testing Completed	0 days	Fri 5/8/26	Fri 5/8/26
Deployment	6 days	Sat 5/9/26	Fri 5/15/26
Publish the Application	6 days	Sat 5/9/26	Fri 5/15/26
Meeting, Every Week	180.38 days	Thu 9/4/25	Thu 5/14/26

Figure 2.1: ColorMate MS Project.

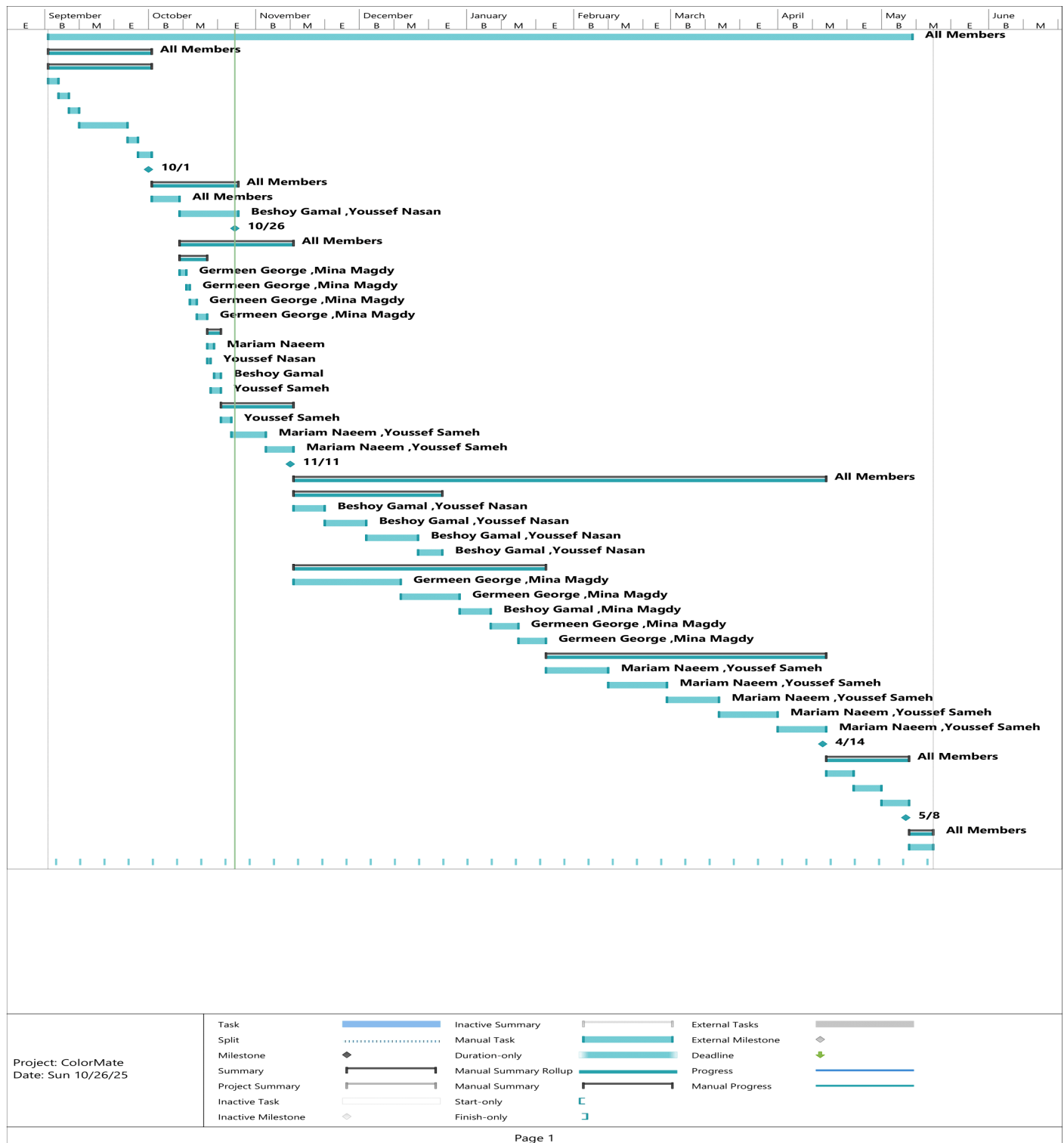


Figure 2.2: ColorMate MS Project Graph.

☐	Type	Key	↑	Summary	Status	Start date	Due date	Team	Assignee
☐	⚡	COL-1		Documentation	IN PROGRESS	<input type="calendar"/> Sep 2, 2025	<input type="calendar"/> May 9, 2026	All Members	
☐	▼ ⚡	COL-2		Planning	DONE	<input type="calendar"/> Sep 2, 2025	<input type="calendar"/> Oct 1, 2025	All Members	
☐	▼ ☒	COL-8		Define Project Scope	DONE	<input type="calendar"/> Sep 2, 2025	<input type="calendar"/> Oct 1, 2025		
☐	🔗	COL-9		Conduct Research on Color Blindness Types and Testin...	DONE	<input type="calendar"/> Sep 2, 2025	<input type="calendar"/> Sep 4, 2025		
☐	🔗	COL-10		Define Deliverables and Acceptance Criteria	DONE	<input type="calendar"/> Sep 5, 2025	<input type="calendar"/> Sep 7, 2025		
☐	🔗	COL-11		Define Scope	DONE	<input type="calendar"/> Sep 8, 2025	<input type="calendar"/> Sep 10, 2025		
☐	🔗	COL-12		Project Objectives	DONE	<input type="calendar"/> Sep 11, 2025	<input type="calendar"/> Sep 24, 2025		
☐	🔗	COL-13		Create Project Plan	DONE	<input type="calendar"/> Sep 25, 2025	<input type="calendar"/> Sep 27, 2025		
☐	🔗	COL-14		Divide Project Into Tasks	DONE	<input type="calendar"/> Sep 28, 2025	<input type="calendar"/> Oct 1, 2025		
☐	▼ ⚡	COL-3		Analysis	DONE	<input type="calendar"/> Oct 2, 2025	<input type="calendar"/> Oct 26, 2025		
☐	☒	COL-15		Collect Functional and non-functional Requirements	DONE	<input type="calendar"/> Oct 2, 2025	<input type="calendar"/> Oct 9, 2025	All Members	
☐	☒	COL-16		Data Collection and Cleaning	DONE	<input type="calendar"/> Oct 10, 2025	<input type="calendar"/> Oct 26, 2025	AI Team	
☐	▼ ⚡	COL-4		Design	IN PROGRESS	<input type="calendar"/> Oct 10, 2025	<input type="calendar"/> Nov 11, 2025		
☐	▼ ☒	COL-17		System Design	DONE	<input type="calendar"/> Oct 10, 2025	<input type="calendar"/> Oct 17, 2025	Backend Team	
☐	🔗	COL-19		ERD Model	DONE	<input type="calendar"/> Oct 10, 2025	<input type="calendar"/> Oct 11, 2025	Backend Team	
☐	🔗	COL-20		Class Diagram	DONE	<input type="calendar"/> Oct 12, 2025	<input type="calendar"/> Oct 12, 2025	Backend Team	
☐	🔗	COL-21		Relational Database Diagram	DONE	<input type="calendar"/> Oct 13, 2025	<input type="calendar"/> Oct 14, 2025	Backend Team	
☐	🔗	COL-22		Database Design	DONE	<input type="calendar"/> Oct 15, 2025	<input type="calendar"/> Oct 17, 2025	Backend Team	
☐	▼ ☒	COL-18		Functional & Behavioral Design	DONE	<input type="calendar"/> Oct 18, 2025	<input type="calendar"/> Oct 21, 2025		
☐	🔗	COL-23		Use-Case Diagram	DONE	<input type="calendar"/> Oct 18, 2025	<input type="calendar"/> Oct 19, 2025		 mariam naeem
☐	🔗	COL-24		Context Diagram	DONE	<input type="calendar"/> Oct 18, 2025	<input type="calendar"/> Oct 18, 2025		 Youssef Nasan
☐	🔗	COL-25		Activity Diagram	DONE	<input type="calendar"/> Oct 20, 2025	<input type="calendar"/> Oct 21, 2025		 Beshoy Gamal

☐		COL-26	Sequence Diagram	DONE	Oct 19, 2025	Oct 21, 2025		Youssef Sameh
☐	✓ ☒	COL-27	UI/UX Design	IN PROGRESS	Oct 22, 2025	Nov 11, 2025	Mobile Team	
☐		COL-28	Logo Design	DONE	Oct 22, 2025	Oct 24, 2025	Mobile Team	Youssef Sameh
☐		COL-29	User Interface Design	IN PROGRESS	Oct 25, 2025	Nov 3, 2025	Mobile Team	
☐		COL-30	User Experience Design	TO DO	Nov 4, 2025	Nov 11, 2025	Mobile Team	
☐	✓	COL-5	Development	TO DO	Nov 12, 2025	Apr 14, 2026		
☐	✓ ☒	COL-31	AI Development	TO DO	Nov 12, 2025	Dec 24, 2025	AI Team	
☐		COL-34	Implement Object & Color Detection Model	TO DO	Nov 12, 2025	Nov 20, 2025	AI Team	
☐		COL-35	Implement Clothing Color Matching Model	TO DO	Nov 21, 2025	Dec 2, 2025	AI Team	
☐		COL-36	Review Functional Specifications	TO DO	Dec 3, 2025	Dec 17, 2025	AI Team	
☐		COL-37	Review non-functional Specifications	TO DO	Dec 18, 2025	Dec 24, 2025	AI Team	
☐	✓ ☒	COL-32	Backend Development	TO DO	Nov 12, 2025	Jan 23, 2026	Backend Team	
☐		COL-38	Database Schema Implementation	TO DO	Nov 12, 2025	Dec 12, 2025	Backend Team	
☐		COL-39	API Implementation	TO DO	Dec 13, 2025	Dec 29, 2025	Backend Team	
☐		COL-40	API Testing & Debugging	TO DO	Dec 30, 2025	Jan 7, 2026	Backend Team	
☐		COL-41	Backend Security & Performance Optimization	TO DO	Jan 8, 2026	Jan 15, 2026	Backend Team	
☐		COL-42	Backend Deployment	TO DO	Jan 16, 2026	Jan 23, 2026	Backend Team	
☐	✓ ☒	COL-33	Mobile Development	TO DO	Jan 24, 2026	Apr 14, 2026	Mobile Team	
☐		COL-43	UI/UX Implementation	TO DO	Jan 24, 2026	Feb 10, 2026	Mobile Team	
☐		COL-44	State Management Design and Implementation	TO DO	Feb 11, 2026	Feb 27, 2026	Mobile Team	
☐		COL-45	API Integration	TO DO	Feb 28, 2026	Mar 14, 2026	Mobile Team	
☐		COL-46	AI Integration	TO DO	Mar 15, 2026	Mar 31, 2026	Mobile Team	

☐		COL-47	Mobile Deployment	TO DO	 Apr 1, 2026	 Apr 14, 2026	Mobile Team	
☐	▼ 	COL-6	Testing	TO DO	 Apr 15, 2026	 May 8, 2026	All Members	
☐	☒	COL-48	Validate Test Result Accuracy	TO DO	 Apr 15, 2026	 Apr 22, 2026		
☐	☒	COL-49	Unit Testing	TO DO	 Apr 23, 2026	 Apr 30, 2026		
☐	☒	COL-50	Integration Testing	TO DO	 May 1, 2026	 May 8, 2026		
☐	▼ 	COL-7	Deployment	TO DO	 May 9, 2026	 May 15, 2026	All Members	
☐	☒	COL-51	Publish the Application	TO DO	 May 9, 2026	 May 15, 2026		



 Create

Figure 2.3: ColorMate Jira Project

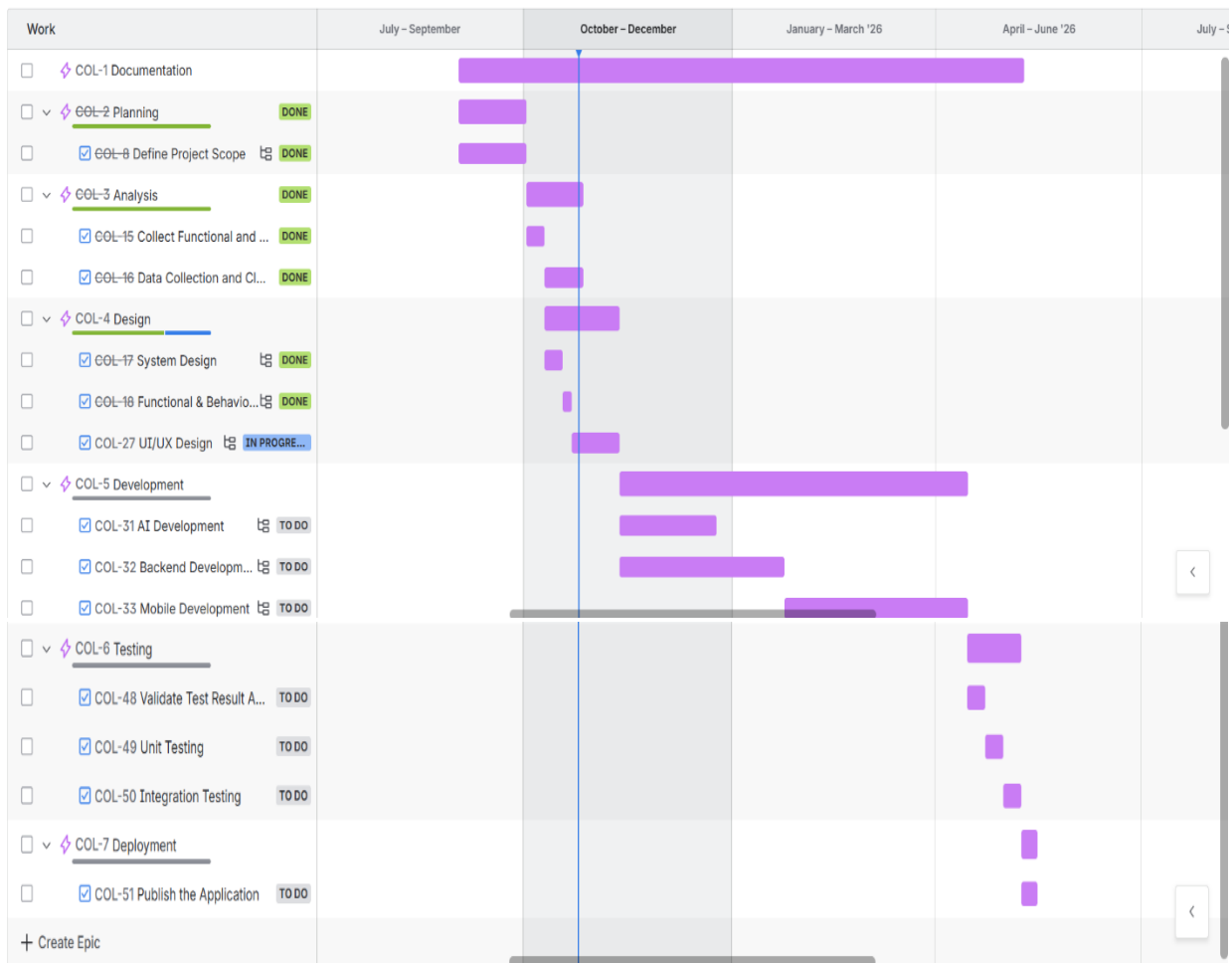


Figure 2.4: ColorMate Jira Project Graph

2.10 Conclusion

The project plan provides a clear roadmap for the successful implementation of the Color Blindness Detection App. Through careful allocation of resources, defined objectives, and continuous evaluation, the project aims to deliver an accessible and reliable tool that leverages AI to detect, simulate, and support individuals with color vision deficiencies. Effective planning ensures that the team can complete the project efficiently and achieve meaningful social and technological impact.

Related Work

Chapter three

This chapter reviews existing mobile applications developed to support individuals with color vision deficiency and highlights their main features and limitations. Applications such as ColorADD, Color Blind Test, and Color Blind Pal are analyzed to identify gaps in current solutions.

The chapter then introduces our application **ColorMate**, explaining its design goals and key features. A comparison between ColorMate and existing systems is presented to demonstrate its advantages. Finally, the datasets, related research work, and technologies used in developing the application are discussed.

3.1. Overview of Existing Platforms:

3.1.1 ColorADD [2]:

ColorADD is a mobile application developed to assist individuals with color vision deficiency (color blindness) in recognizing and differentiating colors in their daily environments. The main goal of the app is to promote accessibility and inclusivity by simplifying color identification through a straightforward mobile interface.

The app uses the device's built-in camera to detect the RGB values of an object and convert them into readable color names. Users can either scan colors in real time or upload an image from their gallery for analysis.

Key Features:

- **Object & Color Detection:** Instantly identifies the color of any object using the phone's camera.
- **Image-Based Recognition:** Detects dominant colors from uploaded images in the gallery.
- **Color Name Display:** Clearly shows the detected color name at the top of the screen (available in Arabic or English).
- **Basic Color Categorization:** Displays simple, general color names for easy recognition (e.g., red, blue, ivory).
- **Color History:** Stores recently detected colors for later reference.
- **Free and Accessible:** Completely free to download and use without subscription.
- **Cross-Platform Support:** Available for both Android and iOS devices.

- **Offline Functionality:** Operates without an internet connection once installed.

Limitations:

- **Lighting Dependence:** The accuracy of color detection is affected by poor lighting conditions or camera quality.
- **Limited Color Accuracy:** May provide approximate color names depending on reflections or ambient light.
- **No Type-Specific Support:** Does not distinguish between different types of color blindness (Protanopia, Deuteranopia, Tritanopia).
- **No Simulation or Correction:** Lacks features for simulating or correcting images based on color blindness types.
- **No Audio Feedback:** Displays color names visually only.

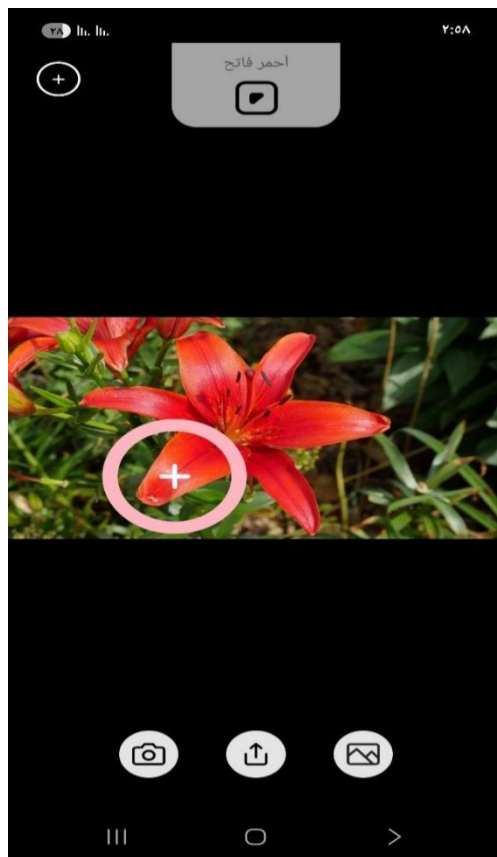


Figure 3.1: detecting the color of a specific area selected by the user within the image by **ColorADD** application [1]

3.1.2 Color Blind Test [3]:

Color Blind Test is a mobile application designed to evaluate and analyze color vision through multiple standardized tests. It helps users identify whether they have a color vision deficiency (CVD) and determine its type and severity. The app provides various test modes that simulate real ophthalmic assessments in a simplified, interactive format.

Key Features:

- **Multiple Testing Modes:** Includes several standardized color vision tests such as:
 - Ishihara Test for general color blindness screening.
 - Lantern Test to assess color perception under low-light conditions.
 - Red-Green Test to detect red-green color vision deficiencies.
 - Color Hue Test to evaluate the user's ability to arrange colors by hue.
- **Result Analysis:** Generates a detailed report indicating whether the user has normal vision or a specific color vision deficiency (Protanopia, Deuteranopia, or Tritanopia).
- **Simulation Mode:** Allows users to simulate how people with different types of color blindness perceive images or live camera views.
- **Educational Insights:** Provides explanations about types of color blindness and their impact on visual perception.
- **User-Friendly Interface:** Simple and intuitive design suitable for all age groups.
- **Offline Functionality:** Works without an active internet connection.
- **Free and Cross-Platform:** Available for free on both Android and iOS platforms.

Limitations:

- **Limited Medical Accuracy:** The results are approximate and should not replace professional medical evaluation.
- **No Correction or Enhancement Tools:** Does not include features to adjust or improve color perception after diagnosis.
- **Device-Dependent Accuracy:** Results may vary depending on screen brightness, color calibration, or lighting conditions.
- **No Test History Tracking:** The app does not save user results or progress across sessions.

- **No Audio or Accessibility Support:** Lacks voice feedback or accessibility features for visually impaired users.

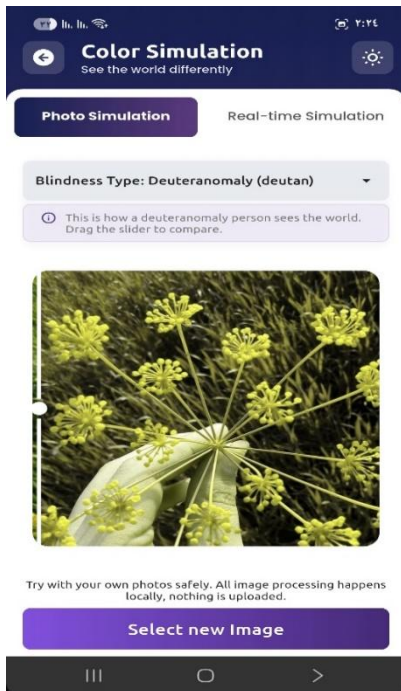


Figure 3.2: Color simulation of the image as perceived by a person with Deuteranomaly by **Color Blind Test** app [2]

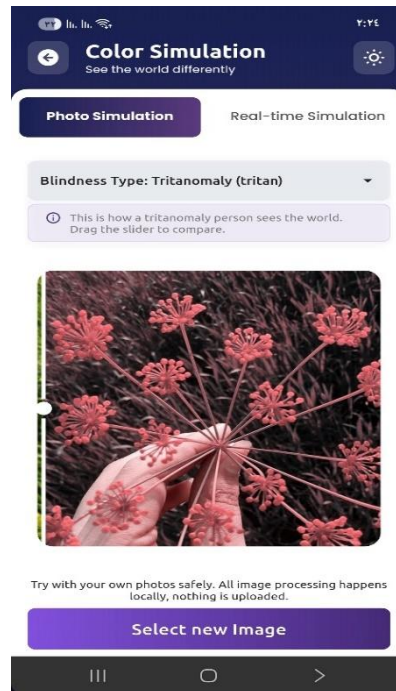


Figure 3.3: Color simulation of the image as perceived by a person with Tritanomaly by **Color Blind Test** app [2]

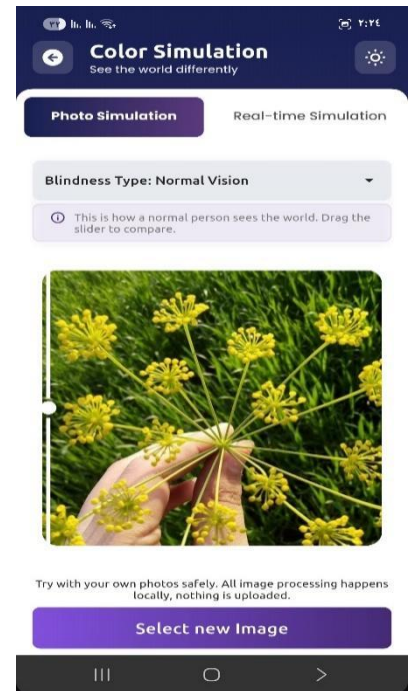


Figure 3.4: Original image under normal vision given to **Color Blind Test** app [2]

3.1.3 Color Blind Pal [4]:

Color Blind Pal is a free application available on iOS, Android, and macOS, designed to assist individuals with color blindness in identifying colors in their environment. It also allows users with normal vision to simulate the experience of color blindness. The app offers a user-friendly interface and interactive tools for daily use.

Key Features:

- **Color Identification:** Enables users to accurately determine color names, such as "Muted Green," along with HSV (Hue, Saturation, Value) and RGB values.
- **Color Blindness Simulation:** Allows simulation of various types of color blindness, including Deuteranopia, Protanopia, and Tritanopia, helping users understand how others perceive colors.
- **Filter Mode:** Provides a filter that adjusts colors through the camera, assisting color-blind users in distinguishing colors more effectively.
- **Simulation Mode:** Allows users with normal vision to see the world as perceived by individuals with color blindness, promoting empathy and understanding.
- **Cross-Platform Compatibility:** Available on iOS, Android, and macOS, ensuring broad accessibility.

Limitations:

- **Limited Accuracy in Certain Conditions:** Some users may experience difficulties obtaining precise color names under varied lighting conditions or with low-quality images.
- **User Interface Challenges:** Certain interface elements may not be intuitive initially, requiring users to spend time learning how to navigate the app effectively.
- **Dependence on Camera Quality:** Color recognition accuracy can be affected in low-light environments or when reflections are present.
- **Lack of Built-in Audio Support:** The absence of integrated audio guidance may pose challenges for users with additional accessibility needs.

3.2 ColorMate: A Smart Color Blindness Detection and Assistance App

ColorMate is an intelligent mobile application designed to support individuals with color vision deficiency (CVD) by providing tools for detection, simulation, correction, and education in a unified platform.

It integrates computer vision, artificial intelligence, and accessibility-focused design to help color-blind users identify colors, understand their visual condition, and interact confidently with the world around them.

The application aims to promote visual accessibility, color inclusivity, and user independence in daily life activities such as clothing selection, education, and object recognition.

Key Features:

- **Color Blindness Detection Test (Ishihara-Based):**
The app includes a scientifically recognized Ishihara color blindness test to identify color vision deficiencies. After completing the test, users receive clear visual feedback indicating whether their color perception is normal or impaired, and the results are stored locally for future reference.
- **Color Detection:**
Uses the device camera to identify and display the name of any color in the image.
- **Color Correction Mode:**
Enhances image colors by modifying contrast and color mappings to improve the visibility and distinguishability of visual elements for users with color vision deficiencies.
- **Object and Color Recognition:**
Converts detected colors into readable names for better accessibility.
- **Clothing Matching Assistant:**
Suggests suitable clothing color combinations for users with color vision deficiency, helping them make confident fashion decisions.
- **Educational and Awareness Section:**

Provides information on types of color blindness with visual examples to help users learn more about their condition, along with interactive lessons and color-learning activities for children.

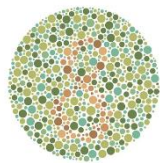
- **AI Chatbot Assistant:**

Integrated AI-powered chatbot helps users interpret their test results and provides answers to common color-related questions.

3.3 Why Our Application Is Better Than Existing Systems

This section presents a brief comparison between existing applications and our app, highlighting the key differences in features and explaining how our app offers more effective solutions tailored to the needs of color-blind users.

Table 3.1 Comparison between existing systems and our Application

Feature	ColorADD	Color Blind Pal	Color Blind Test	ColorMate
Logo				
Main Purpose	Helps identify colors for color-blind users	Helps identify colors for color-blind users	Tests users to detect type of color blindness	Detects, simulates, and assists color-blind users using AI
Color Detection	Yes	Yes	No	Yes (advanced color detection with object recognition using YOLO)
Color Blindness Test	No	No	Yes	Yes

Correction Mode	No	No	No	Yes (Image correction using color filters and ML)
AI & Machine Learning	No	No	No	Yes (AI-based recommendations and adaptive UI)
User Interface	Basic	Simple	Simple	Modern, accessible, and adaptive UI (Flutter)
Educational Section	Yes	No	No	Yes (Includes awareness content and training materials)
Reference	[1]	[2]	[3]	Our Application

3.4 Dataset Used:

3.4.1 COCO 2017 Dataset [\[5\]](#)

Title: Common Objects in Context (COCO) Dataset – 2017 Version

Dataset Link: Kaggle - COCO 2017 Dataset [\[5\]](#)

Key Information:

- **Size:** Over 200,000 labeled images
- **Data Fields:** Image ID, Object Labels, Bounding Boxes, Segmentation Masks, Captions
- **Details:**

- Purpose: The COCO 2017 dataset is widely used for training and evaluating computer vision models. It supports tasks such as object detection, instance segmentation, keypoint detection, and image captioning.
- Description: Each image contains multiple objects annotated with class labels and segmentation masks, enabling models to understand both object presence and spatial structure within real-world scenes.



Figure 3.5: Sample image from COCO 2017 dataset (clock and household objects)



Figure 3.6: Sample image from COCO 2017 dataset (kitchen scene)

3.4.2 Deep Fashion Images [\[6\]](#)

Title: DeepFashion-MultiModal – Comprehensive Clothing Understanding Dataset

Dataset Link: GitHub – Deep Fashion Images [\[6\]](#)

Key Information:

- **Size:** Over 44,000 full-body fashion images with rich annotations.
- **Data Fields:** Image files, Segmentation masks, Keypoints, DensePose data, Clothing attributes (shape, fabric, color), Textual descriptions.

- **Details:**

Purpose: Utilized for clothing attribute extraction and color analysis, enabling AI systems to identify clothing components and evaluate outfit color compatibility



Figure 3.7: Sample 1 DeepFashion dataset - Figure 3.8: Sample 2 DeepFashion dataset



Figure 3.9: Sample 3 DeepFashion dataset - Figure 3.10: Sample 4 DeepFashion dataset

3.4.3 Fruit and Vegetable Disease Dataset [\[7\]](#)

Title: Fruit and Vegetable Disease - Healthy vs Rotten Dataset

Dataset Link: Kaggle - Fruit and Vegetable Disease (Healthy vs Rotten) [\[7\]](#)

Key Information:

- **Size:** ~29,000 images of fruits and vegetables
- **Data Fields:** Image files categorized into Healthy and Rotten classes
- **Details:**
 - Purpose: Used for image classification tasks to train models that can identify whether a fruit or vegetable is healthy or rotten.
 - Applications: Supports computer vision and deep learning research in food quality detection and agricultural automation.



Figure 3.11: Healthy apple sample from Fruit and Vegetable Disease dataset.



Figure 3.12: Rotten apple sample from Fruit and Vegetable Disease dataset.

3.5 Related Papers:

Establishing Colour Harmony Evaluation and Recommendation Model for Clothing Colour Matching Based on Machine Learning and Deep Learning [\[8\]](#)

This research proposes a hybrid framework combining machine learning algorithms (SVM, Random Forest) with deep learning models (CNN, semi-supervised CNN, and GAN) to evaluate and recommend clothing color harmony. The system extracts color and visual features from clothing images and applies deep learning to classify harmonious and non-harmonious color combinations. The experiments were conducted using the DeepFashion dataset, achieving 98% accuracy in classification and 92% accuracy in regression tasks. This paper strongly supports ColorMate's color harmony analysis and recommendation feature, as it proves the effectiveness of AI-based models in understanding and evaluating color combinations.

Color Harmony Analysis and Outfit Recommendation [\[9\]](#)

This study presents a rule-based and color-theory-driven system for analyzing color harmony and recommending clothing combinations. It relies on HSL color space analysis and predefined harmony rules such as complementary, analogous, and triadic color relationships. This paper supports the logical foundation of ColorMate's color harmony evaluation layer, especially for building interpretable recommendation rules.

FASHION RECOMMENDATION: OUTFIT COMPATIBILITY USING GNN [\[10\]](#)

This paper presents an outfit recommendation system based on Graph Neural Networks (GNN) and Hypergraph Neural Networks (HGNN) to measure outfit compatibility. Each clothing item is represented as a node, while the relationships between items such as color, category, and style are modeled as edges. The model learns complex dependencies between fashion items to determine whether a given outfit is compatible or not. The system was trained and evaluated using the Polyvore dataset, achieving 77% accuracy in outfit compatibility prediction. This paper is directly related to ColorMate's outfit rating module, as it demonstrates how AI models can learn compatibility relationships between clothing items and colors. The concept of modeling outfits as connected components supports the development of ColorMate's intelligent color harmony and outfit evaluation system.

YOLOv8: A Cutting-Edge Object Detection Model [\[11\]](#)

This paper introduces YOLOv8, a state-of-the-art deep learning model designed for real-time object detection tasks. The model is based on a convolutional neural network architecture incorporating a CSPNet backbone and a feature aggregation strategy using FPN and PAN structures. YOLOv8 adopts an anchor-free detection mechanism, which improves localization accuracy and training efficiency compared to earlier YOLO versions. The model was trained and evaluated using benchmark datasets such as Microsoft COCO achieving up to 71.5% with the YOLOv8x variant. Although this work focuses on general object detection rather than fashion recommendation, it is relevant to ColorMate as it demonstrates robust visual feature extraction and object recognition capabilities. These capabilities can

be leveraged in ColorMate for accurate clothing item detection and segmentation, which form a critical foundation for downstream tasks such as color extraction, outfit analysis, and recommendation.

Evaluating pretrained CNNs for distinguishing fresh vs rotten fruits and vegetables [\[12\]](#)

This paper conducted a systematic evaluation of pre-trained convolutional neural networks (CNNs) for distinguishing fresh and rotten fruits and vegetables. The study employed a publicly available Kaggle dataset comprising images of multiple fruit and vegetable categories labeled as fresh or rotten. Several state-of-the-art pre-trained CNN architectures, including DenseNet, MobileNet, EfficientNet, and ShuffleNet, were analyzed using transfer learning techniques. Standard image preprocessing and k-fold cross-validation were applied to ensure reliable and unbiased performance assessment. The experimental results demonstrated that lightweight architectures, particularly ShuffleNet, achieved high classification accuracy while maintaining low computational complexity. These findings indicate that efficient CNN models can deliver competitive performance for freshness classification tasks. The study highlights the feasibility of deploying deep learning-based solutions for real-time food quality inspection systems and provides a strong benchmark for future research in automated fruit and vegetable freshness detection.

Learning color compatibility in fashion outfits [\[13\]](#)

This paper investigates predicting color compatibility in fashion outfits using machine learning models trained on deep visual features and color histograms. The experiments were conducted using the Polyvore outfits dataset, achieving 84% accuracy in compatibility prediction. This research validates the feasibility of learning color compatibility patterns from large-scale fashion datasets, which directly supports ColorMate's AI-based outfit evaluation and recommendation functionality.

A Physiologically-Based Model for Simulation of Color Vision Deficiency [\[14\]](#)

This paper presents a physiologically-based model for simulating color vision deficiency (CVD), including protanopia, deuteranopia, and tritanopia. The model is based on the human visual system and describes color perception through cone photoreceptor responses and color vision deficiency severity levels. The authors generated transformation matrices for different types and severities of color blindness, enabling realistic simulation of how individuals with CVD perceive images and colors. The proposed approach has become one of the most widely adopted methods for color blindness simulation due to its accuracy and computational efficiency. This paper directly supports ColorMate's color blindness simulation module, as the application uses Machado-based transformation matrices to simulate different CVD conditions and help users better understand color perception differences.

Evaluating Color Vision Deficiency Daltonization Methods Using a Behavioral Visual-Search Method [\[15\]](#)

This paper evaluates several color vision deficiency (CVD) daltonization techniques using a behavioral visual-search methodology rather than relying solely on mathematical color differences. The study investigates how effectively different daltonization methods improve color discrimination for individuals with color vision deficiencies while preserving the overall appearance of images. The authors compare multiple correction approaches and measure their impact on users' ability to identify visual targets under simulated color vision deficiency conditions. The results demonstrate that effective daltonization techniques can significantly enhance color distinguishability and visual accessibility for color-deficient observers. This paper supports ColorMate's color blindness correction module by providing a scientific foundation for evaluating and improving color correction methods that help users better distinguish colors and clothing combinations affected by color vision deficiencies.

Ishihara Color Vision Test Instructions and Assessment Methodology [\[26\]](#)

The Ishihara Color Vision Test is one of the most widely used and clinically validated methods for detecting congenital color vision deficiencies, particularly red-green color blindness. Developed by Shinobu Ishihara, the test consists of a series of pseudoisochromatic plates containing colored dots arranged to form numbers or paths that can be correctly identified by individuals with normal color vision but may be difficult or impossible to recognize for color-deficient observers. The test is designed to distinguish between different types and severities of red-green

color vision deficiencies, including protanopia, protanomaly, deuteranopia, and deuteranomaly. The authors explain the principles behind the test, the interpretation of responses, and the classification of color vision defects based on the user's performance. The Ishihara Test provides a fast, reliable, and non-invasive screening method for color blindness and has become a standard tool in both clinical and research settings. This work supports ColorMate's color blindness detection module by providing a scientifically established method for identifying users with color vision deficiencies before applying personalized color correction and accessibility enhancements.

Table 3.2 Related Work

Paper Reference	Method used	Year	Dataset	Accuracy
[8]	Machine Learning (SVM, RF) + Deep Learning (CNN, Semi-Supervised CNN, GAN, Regression)	2025	DeepFashion	98% (classification) 92% (regression)
[9]	Rule-based + HSL color theory + web-based color naming (algorithmic color harmony logic)	2025	9 fashion items with attributes	The paper does not report a standard accuracy

[10]	Node-wise GNN & Hypergraph Neural Network (HGNN)	2024	Polyvore dataset	77% (Compatibility) 40% (FITB)
[11]	Deep Learning – YOLOv8 Object Detection (CNN-based, CSPNet backbone + FPN+PAN neck, anchor-free)	2024	Microsoft COCO	55.2% (YOLOv8 overall) Up to 71.5% (YOLOv8x variant)
[12]	Deep Learning: Pre-trained CNNs (MobileNetV3) Adapted via Transfer Learning, and Evaluated with Cross-Validation	2024	Fruit and Vegetable Disease - Healthy vs Rotten Dataset	94.1482% (MobileNet V3)
[13]	ML with deep features, color histograms, compatibility classification + FITB	2020	Polyvore outfits dataset	84% (Compatibility) 59.8% (FITB)
[14]	Color Vision Deficiency Simulation using Physiologically-Based Color Transformation Matrices (Machado Model)	2009	Human color vision model / simulated visual perception	Not evaluated using classification accuracy
[15]	Evaluation of Color Vision Deficiency Daltonization Methods using	2014	Visual Search Test Images	Performance evaluated using behavioral visual-search metrics rather than

	Behavioral Visual Search			classification accuracy
[26]	Ishihara Pseudoisochromatic Plate Test	1917	14 color vision test plates	High sensitivity and specificity for red-green color blindness detection (no single standard accuracy reported)

3.6 Used technologies:

3.6.1 Flutter: [16]

Flutter is a popular cross-platform framework with a single code base, operating with Dart programming language. Launched only in 2018 by Google, Flutter has proven itself a convenient toolkit, easy for creating animations and quality UI components. Despite being Google's fresh take on cross-platform, Flutter provides smooth animations and user-friendly interface elements. With Flutter you can write high-quality apps with high performance that look like native iOS and Android apps

Flutter's best features:

1. Hot Reload

Makes it possible to see the changes in the code instantly reflected on the UI. This speeds up the process of working on the outlook of the application; moreover, it enables developers to correct errors and save cost and effort.

2. Widgets for fast UI coding

Flutter uses ready-made widgets and helps create a user interface utilizing these building blocks with the widgets it's much easier to develop even complex UIs.

3. Documentation and learning resources

Google created detailed and well-structured documentation. Apart from classic docs, you can watch video lessons from the Google team, and go through practical exercises on Codelabs.

3.6.2 ASP.NET Core: [17]

ASP.NET Core is a powerful, high-performance backend web framework developed by Microsoft, ideal for building APIs and server logic for mobile applications. It's widely used to build the backend for mobile apps (Flutter, React Native, Swift, Kotlin, etc.), providing fast communication, secure authentication, and stable business logic. ASP.NET Core is highly scalable and optimized for cloud and enterprise-grade applications.

Despite being a backend framework, ASP.NET Core enables mobile developers to deliver fast, secure, and reliable app experiences through RESTful APIs that mobile apps consume.

ASP.NET Core's Best Features

1. High Performance & Scalability

ASP.NET Core is known as one of the fastest backend frameworks. It can handle millions of users with low latency — ideal for real-time apps, mobile e-commerce, or streaming services.

2. Cross-Platform & Cloud Ready

Runs on Windows, Linux, or macOS, making it ideal for deploying mobile backends on cloud platforms like Azure, AWS, DigitalOcean, etc.

3. Secure Authentication & Authorization

Comes with built-in support for JWT Tokens, Identity, and supports mobile login systems (Email, Google, etc...) with enterprise-level security.

4. RESTful Web API Development

Mobile apps (built with Flutter, React Native, Swift, etc.) can easily communicate with ASP.NET Core backend via JSON APIs — perfect for mobile data exchange.

5. Dependency Injection & Clean Architecture

Helps developers write clean, maintainable, and testable backend logic — very useful for long-term mobile app projects.

3.6.3 Artificial Intelligence and Machine Learning Stack

1. Convolutional Neural Networks (CNNs) [\[18\]](#)

CNNs are deep learning models that automatically extract visual features from images and are widely used in image classification, object recognition, and feature extraction.

2. YOLO (You Only Look Once) – YOLOv8 [\[19\]](#)

YOLO is a fast, real-time object detection framework used to identify and locate objects in images and videos efficiently. Its single-stage architecture allows for both high accuracy and low-latency detection, making it suitable for various computer vision applications.

3. Python [\[20\]](#)

Python is a versatile programming language popular for software development, data analysis, and machine learning due to its simplicity and extensive libraries.

4. PyTorch [\[21\]](#)

PyTorch is a deep learning framework offering dynamic computation graphs and GPU support, widely used for building and training neural networks.

5. NumPy [\[22\]](#)

NumPy is a Python library for numerical computing, providing support for arrays, mathematical operations, and linear algebra for data analysis.

6. OpenCV [\[23\]](#)

OpenCV is a computer vision library used for image and video processing, including filtering, transformation, and object detection.

7. PIL (Pillow) [\[24\]](#)

Pillow is a Python library for basic image handling, allowing loading, saving, converting, and performing simple operations on images.

8. Matplotlib [\[25\]](#)

Matplotlib is a Python plotting library for creating graphs and visualizations to interpret and present data effectively.

3.7 Conclusion

The analysis of existing color blindness detection and assistance applications reveals that most current solutions, such as Color Blind Pal, Chromatic Vision Simulator, and EnChroma Test, address only limited aspects of color vision deficiency. These applications primarily focus on identifying or simulating color blindness, with few offering interactive or educational support. Moreover, they often lack intelligent features, real-time detection, and user engagement components, leaving a clear opportunity for improvement.

To bridge this gap, ColorMate utilizes modern technologies such as Flutter for cross-platform development and artificial intelligence (AI) for smart color detection and analysis. The integration of computer vision and AI allows the system to detect objects and identify their colors in real time, as well as simulate how colors appear to individuals with different types of color blindness.

Additionally, ColorMate includes interactive educational tools and an AI-powered chatbot that helps users learn about color vision, recognize different color categories, and improve daily interactions with colors in their surroundings.

Overall, ColorMate represents a next-generation assistive solution that combines AI, computer vision, and user-centered design to support individuals with color vision challenges. It provides a diagnostic, educational, and practical experience that enhances accessibility and helps users better perceive and understand the colorful world around them.

Chapter four

System analysis is a fundamental phase in the Software Development Life Cycle (SDLC) that focuses on understanding user needs and defining accurate system requirements. In this chapter, we introduce the main tools and techniques used in system analysis to ensure the system is well-structured, efficient, and aligned with user expectations.

4.1 System Requirements:

4.1.1 Hardware Requirements:

- A server infrastructure to host and run the application efficiently.
- Database server for storing user accounts, color blindness test results, processed images, outfit rating history, and user activity data.

4.1.2 Software Requirements

1- Backend Development

- ASP.Net Core for building robust and scalable RESTful APIs.
- Microsoft SQL Server as a reliable relational database system for managing application data.
- ASP.NET Identity for secure user authentication and role-based access control.

2- Frontend Development

- Flutter framework for cross-platform mobile app development, supporting both Android and iOS.

3- Security

- HTTPS to ensure secure data transmission between the client and server.

- Token-based authentication (e.g., JWT) to protect API access and maintain secure user sessions.

4- Network

- Stable internet connection for seamless user and admin access.
- HTTPS protocol to ensure secure and encrypted data transmission.

4.1.3 Functional Requirements

Functional requirements describe the specific actions, features, and behaviors that the software system must perform. They define what the system should do in response to various inputs or user interactions. In essence, these requirements outline the core objectives and expected outcomes of the system and the software cannot operate correctly if they are not fulfilled.

User Management

1- Login and Sign Up

- Users can create an account, log in, and manage their personal profile.

2- Profile Editing

- Users can update personal information.

3- Color Blindness Test

- Users take a color blindness test where the app displays test images and the user selects what they see.
- User gets their exact color-blind type detected and identified by the system (e.g., Protanopia, Deuteranopia, Tritanopia).
- User can re-take the color-blindness test if they want to update or confirm their result.

4- Outfit Rating

- Users uploads an image of their outfit (shirt and pants).
- User can receive a rating based on color compatibility and visibility.
- User can get color improvement suggestions if the score is low.
- User can save recommended outfit combinations for future use.

5- Color Correction

- Users uploads an image he struggles to see its colors properly.
- User sees the adjusted image with colors optimized for their specific color-blind type for clearer and more accurate color perception.
- User can save the filtered image to their history.
- User can view previously color corrected images in a saved history section.

6- Color Name Detection

- User can upload or capture an image to identify the colors of different objects in it.
- User receives a clear text description of the detected colors in the image.

4.1.4 Non-Functional Requirements:

Non-functional requirements define how the system should perform rather than what it should do. They cover qualities like security, performance, reliability, usability, scalability, and maintainability. These requirements guide system design and ensure overall efficiency, user satisfaction, and long-term sustainability.

1- Reliability:

- the system should be available all time.

2- Usability:

- The user interface should be intuitive, accessible, and color-blind-friendly, ensuring a smooth and user-friendly experience for both admins and users.

3- Accuracy:

- Accurate results for outfits Rating and suggestions.

4- Performance:

- The system should efficiently handle user actions such as login, data retrieval, and content submission with minimal response time.

5- Security:

- Secure user authentication should be implemented using token-based mechanisms such as JWT.
- All sensitive data — including user information, test results, and preferences — must be encrypted.

6- Scalability:

- The application must be capable of efficiently handling an increasing number of users and growing data volume without performance degradation.

7- Maintainability:

- The codebase should be well-structured and thoroughly documented to ensure updates and feature enhancements can be applied with minimal disruption.

4.2 Use Case Analysis

Use case analysis describes how users interact with the system to accomplish specific goals, such as performing a color vision test, receiving color-corrected results, or getting outfit rating and recommendations based on color harmony.

Use Case Representation

Use cases can be represented in two main formats:

1. **Use Case Diagram** — a visual representation of system interactions.
 2. **Use Case Specification (Textual Scenario)** — a detailed written description of the interaction.
-

Components of a Use Case Diagram

1. Actors:

- **Color blind user:** Takes the color blindness test and uploads images for processing — such as receiving outfit ratings, color-corrected images.

2. Scenario

A specific sequence of actions and interactions between the actor and the system to achieve a goal. It describes how the system behaves in response to a user action.

3. Goals

Define the objective of each interaction — such as allowing users to accurately determine their color blindness type, receive color-corrected visual output, or obtain outfit ratings and personalized color recommendations.

Characteristics of Use Case Diagrams

1. They are created during the early stages of system analysis and planning.
2. They represent system functionality from the user's perspective.
3. They specify who will use the system and how they will interact with it.
4. They illustrate the interaction between the system, external users, and other connected systems.

4.3 Use Case Scenario

A use case scenario represents a sequence of steps followed by a user to achieve a specific goal within the system. A single use case may include multiple possible paths or variations, and each of these paths is considered an individual use case scenario.

1- Use case name: Register

Table 4.1 Register use case analysis

Use Case Name Register	
Actor	User
Description	The user registers in the ColorMate system by providing the required personal information and account details. The system validates the entered data and creates a new account upon successful verification.
Events	User
	System
	Step 1: The user navigates to the registration page. Step 2: The user enters required information (e.g., name, email, password, and confirmation) or register by Google. Step 3: The user submits the registration form.
	Step 5: The system validates all input fields (e.g., email format, password strength, duplicate accounts). Step 6: If all information is valid, the system creates a new user account in the database. Step 7: The system displays a confirmation message or sends a welcome email. Step 8: The system redirects the user to the home page.
Precondition	• The user must not already have an existing account in the system. • The system must be online and accessible. • The registration page must be available and functioning properly.
Postcondition	• The user receives confirmation of successful registration (e.g., a welcome message or email). • The user is redirected to the home page or login page.
Assumption	• The user provides accurate and valid information (e.g., valid email format, strong password). • Internet connection is stable during the registration process. • The system has proper validation and error-handling mechanisms to detect invalid input.

2- Use case name: Login

Table 4.2 Login use case analysis

Use Case Name	Login	
Actor	User	
Description	The user accesses the ColorMate system by providing their login details. The system verifies the provided information and, upon successful authentication, directs the user to home page.	
Events	User	System
	Step 1: User enters login credentials (email/username and password). Step 2: User submits the login form.	Step 3: System validates the entered credentials Step 4: System checks if the user account exists in the database. Step 5: System verifies the entered password against the stored password. Step 6: If credentials are valid, the system logs in the user and redirects them to their respective home page. Step 7: If credentials are invalid, the system displays an error message.
Precondition	- The user must have completed the signup process. - The user must enter valid login credentials (email/username and password).	
Postcondition	- The user is successfully logged into the ColorMate system. - The system authenticates the user's identity and loads their Home Page.	
Assumption	- The user has an active account that is not banned or deactivated. - The user enters the correct login credentials.	

3- Use case name: Edit profile

Table 4.3 Edit Profile use case analysis

Use Case Name	Edit Profile	
Actor	User	
Description	The user updates their profile details, such as name, email, password, profile picture, and other information	
Events	User	System
	Step 1: User navigates to the Profile Settings page. Step 3: User updates profile fields (e.g., name, email, password, profile picture). Step 5: User clicks "Save Changes."	Step 2: System retrieves and displays the user's current profile information (name, email, preferred filter, etc.). Step 4: System validates the entered data (e.g., email format, password strength, valid filter selection). Step 6: If the data is valid, the system updates the user's profile in the database. Step 7: System confirms the update with a success. Step 8: If data is invalid, the system displays an error message and prompts the user to correct it
Precondition	• The user must be logged in. • The app must have permission to access the camera or gallery (if user changes the profile picture). • The user must have an existing profile in the system.	
Postcondition	• The user's profile and preferences are updated and saved securely. • The system logs the change and updates related accessibility settings immediately.	
Assumption	• The user must have the appropriate permissions to update their profile. • The user is required to enter valid and accurate information during the update process. • The system should allow the user to modify the selected profile details successfully.	

4- Use case name: Detect objects and their colors

Table 4.4 Detect Objects and their colors use case analysis

Use Case Name Detect Objects and Their Colors		
Actor	User	
Description	The user uploads or captures an image. The system uses AI (YOLO and color extraction algorithms) to detect objects and identify their dominant colors, then displays the labeled objects with their detected colors.	
Events	User	System
	Step 1: User opens the 'Color Detection' feature in the app. Step 3: User uploads an image from gallery or captures one using the camera. Step 5: User confirms detection request.	Step 2: System loads the YOLO model and color recognition modules Step 4: System processes the image and detects bounding boxes for each object. Step 6: System extracts each detected object region and immediately identifies the dominant RGB color from it. Step 8: The system converts the RGB values to readable color names and overlays labels (e.g., "Red Apple", "Blue Cup"). Step 10: System displays the annotated image and provides a text description for accessibility.
Precondition	• The user must have granted camera and storage permissions. • The device must have internet connection (if cloud model is used). • The object detection and color recognition models must be preloaded and functional.	
Postcondition	• The system successfully detects objects and displays their color labels on the image. • Results are shown visually for accessibility.	
Assumption	• The image provided contains detectable objects. • Lighting conditions in the image are sufficient for accurate color extraction. • The YOLO and color detection models are running without errors.	

5- Use case name: Identify Color Blindness Type

Table 4.5 Identify color blindness test use case analysis

Use Case Name Identify Color Blindness Type		
Actor	User	
Description	This use case allows the user to take the Ishihara color blindness test through the app. After the test is completed, the system analyzes the responses and determines the user's color blindness type.	
Events	User	System
	Step 1: Opens the app and selects “Color Blindness Test”. Step 3: Chooses test and taps “Start Test”. Step 5: Inputs the answer (e.g., the number seen in the image). Step 7: Submits answers.	Step 2: Displays Ishihara Color Blindness Test. Step 4: Loads test instructions and displays the first test image or question. Step 6: Moves to the next question until all items are completed. Step 8: Analyzes the responses using rule-based logic. Step 9: Determines whether the user has color blindness and its type. Step 10: Displays the result and offers options: Save Result, Retake Test, or Try Another Test.
Preconditions	- The user must be logged into the system. - The app database is available for analysis. - Test images or data are loaded correctly.	
Postconditions	- The user receives a result identifying the presence and type of color blindness (if any). - Test result stored in the user's history for future reference.	
Assumption	- The user's device displays colors accurately and supports touch input. - The user follows instructions and takes the test under proper lighting. - Users are assumed to provide honest and valid answers.	

6- Use case name: Perform Color Correction

Table 4.6 Perform color correction Profile use case analysis

Use Case Name	Perform Color Correction	
Actor	User	
Description	The user uses the Color Correction feature to upload an image and have it adjusted according to their color blindness type. The system retrieves the user's color vision test results and automatically applies the appropriate correction filter to enhance color visibility.	
Events	User	System
	Step 1: The user navigates to the Color Correction feature. Step 4: The user uploads the image to be corrected. Step 7: The user reviews the result and can choose to save or download the corrected image.	Step 2: The system checks whether the user has completed the Color Blindness Test. Step 3: If the test result is available, the system loads the user's color blindness type. Step 5: The system analyzes the image and applies the corresponding color correction filter based on the stored test result. Step 6: The system generates and displays the corrected image preview
Precondition	- The user must be logged into the ColorMate system. - The user must have previously completed the Color Vision Test (the system uses the stored result). - The system must have the necessary storage permissions to access and process uploaded images.	
Postcondition	- The system applies the correct filter according to the user's test result. - The corrected image is displayed and can be saved or downloaded. - User preferences (e.g., chosen filter intensity or adjustments) are saved for future use.	
Assumption	- The stored color blindness type is accurate and up to date. - The uploaded image is in a supported format (e.g., JPEG, PNG). - The system has predefined filters matching all major types of color blindness. - Internet or processing speed is sufficient for image correction.	

7- Use case name: Rate and Recommend color improvements

Table 4.7 Rate Outfit Sequence Diagram use case analysis

Use Case Name	Rate and Recommend color improvements	
Actor	User	
Description	This use case allows the user to upload or capture an outfit image. The system analyzes color harmony and provides a rating score along with suggestions to improve the outfit if poor color matching is detected.	
Events	User	System
	Step 1: Opens the app and selects “Rate Outfit.” Step 3: Uploads or captures a photo of an outfit. Step 4: The user uploads the image to be corrected. Step 7: User reviews score and optional recommended color changes.	Step 2: Displays upload options (Gallery / Camera). Step 4: Preprocesses the image (segment clothing items, detect dominant colors). Step 5: Evaluates color harmony using AI and generates a score (e.g., 7/10). Step 6: If harmony is low system generates recommendations such as “Change the shirt color to light blue for better contrast” or “Replace red pants with grey for balance.”
Precondition	• The user must be logged into the system. • The user has a clear image of an outfit stored on their device. • The AI model for color analysis is available (locally or via cloud).	
Postcondition	• The system displays the color harmony score and optional feedback (e.g., “Colors are complementary”). • The system provides recommendations for better color combinations. • The result can be saved for future reference or shared by the user.	
Assumption	• The AI model is trained to evaluate outfit color harmony accurately. • User uploads clear, well-lit images. • Internet is available if analysis is done online. • The result is advisory, not professional fashion guidance.	

8- Use case name: Ask the ChatBot Questions

Table 4.8 Ask Chatbot use case analysis

Use Case Name Ask the Chatbot Questions		
Actor	User	
Description	This use case allows the user to interact with an AI-powered chatbot to ask questions related to color blindness. The system understands the user's query using NLP/AI techniques and responds with accurate, clear, and user-friendly information.	
Events	User	System
	Step 1: User opens the chatbot interface. Step 3: User types a question related to color blindness (e.g., "What are the types of color blindness?"). Step 5: User waits for the chatbot to respond. Step 9: User reads the chatbot's reply.	Step 2: System initializes the chatbot session and greets the user. Step 4: System receives the question and processes it using Natural Language Processing (NLP). Step 6: System analyzes the intent and extracts relevant keywords from the question. Step 7: System retrieves the most relevant answer from its knowledge base or AI model. Step 8: System generates a clear and simplified response appropriate for the user's understanding level. Step 10: System displays the response and optionally provides related suggestions or links for further reading.
Precondition	- The user must be logged into the system. - The chatbot's AI/NLP model must be properly trained on color blindness information.	
Postcondition	- The chatbot provides a relevant and accurate answer to the user's question. - The user gains useful information about color blindness in an understandable manner.	
Assumption	- The user types a valid and understandable question in a supported language (e.g., English or Arabic). - The chatbot's AI model has access to up-to-date and verified information. - The internet connection is stable to allow smooth communication between user and system.	

9- Use case name: Logout

Table 4.9 Logout use case analysis

Use Case Name	Logout	
Actor	User	
Description	The user logs out of the ColorMate system to end their current session. The system processes the logout request, terminates the session, and redirects the user to the login or home page.	
Events	User	System
	Step 1: The user clicks the Logout button or link from their dashboard or navigation menu.	Step 2: The system receives the logout request. Step 3: The system invalidates the user's current session or authentication token. Step 4: The system clears any temporary user data stored during the session. Step 5: The system redirects the user to the Login or Home Page and displays a confirmation message (e.g., "You have successfully logged out").
Precondition	• The user must already be logged into the ColorMate system. • The system must have an active user session.	
Postcondition	• The user's session is terminated. • The system ensures no further access to authenticated pages without re-login. • The user is redirected to the login or home page.	
Assumption	• The system has proper session management implemented. • The user voluntarily initiates the logout process. • Internet connection remains stable during logout.	

4.4 Sequence Daigram:

A sequence diagram visually represents the interactions between objects in the order they occur. It illustrates how the objects in a system operate and the sequence in which these operations happen.

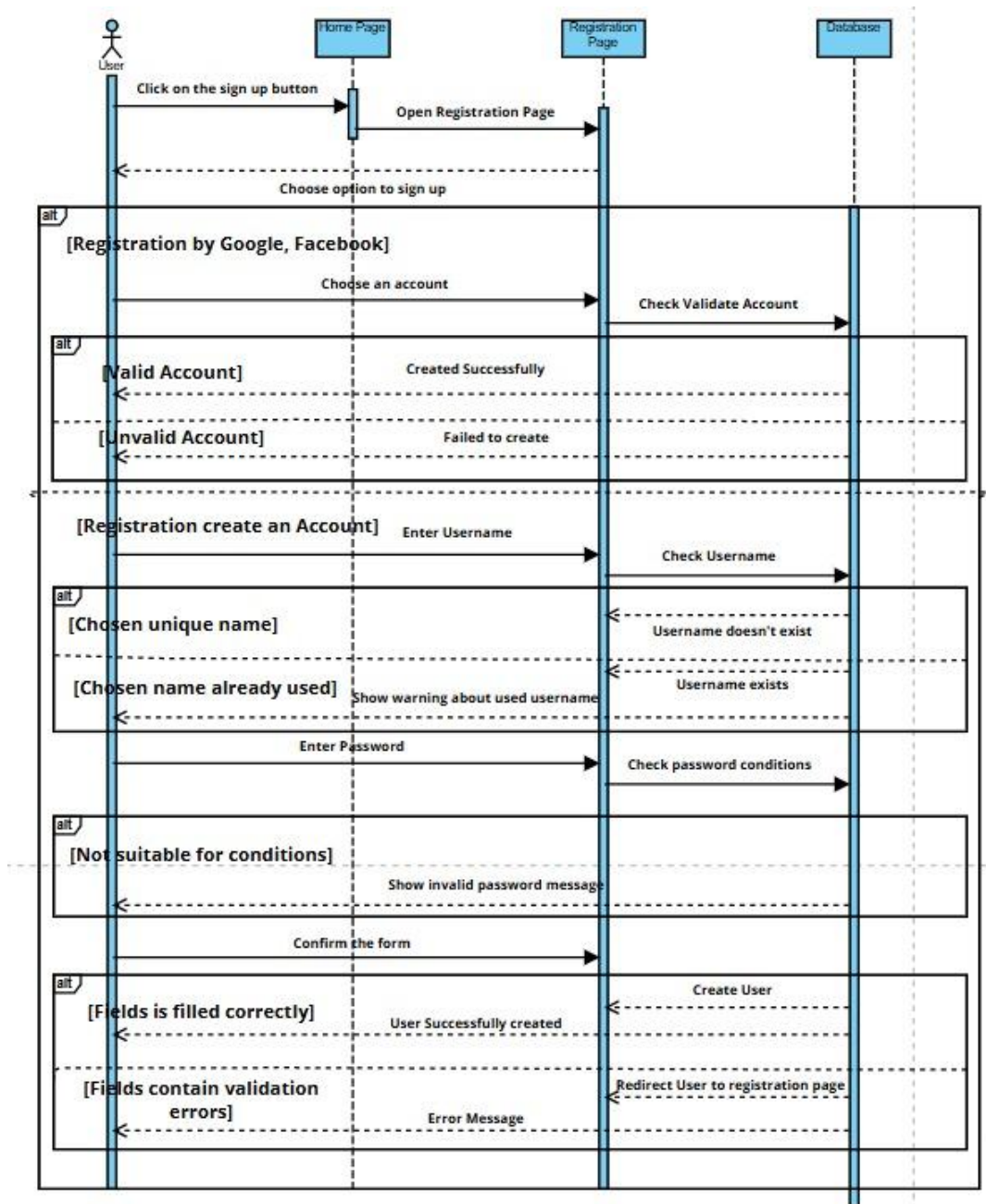


Figure 4.1: Register Sequence Diagram.

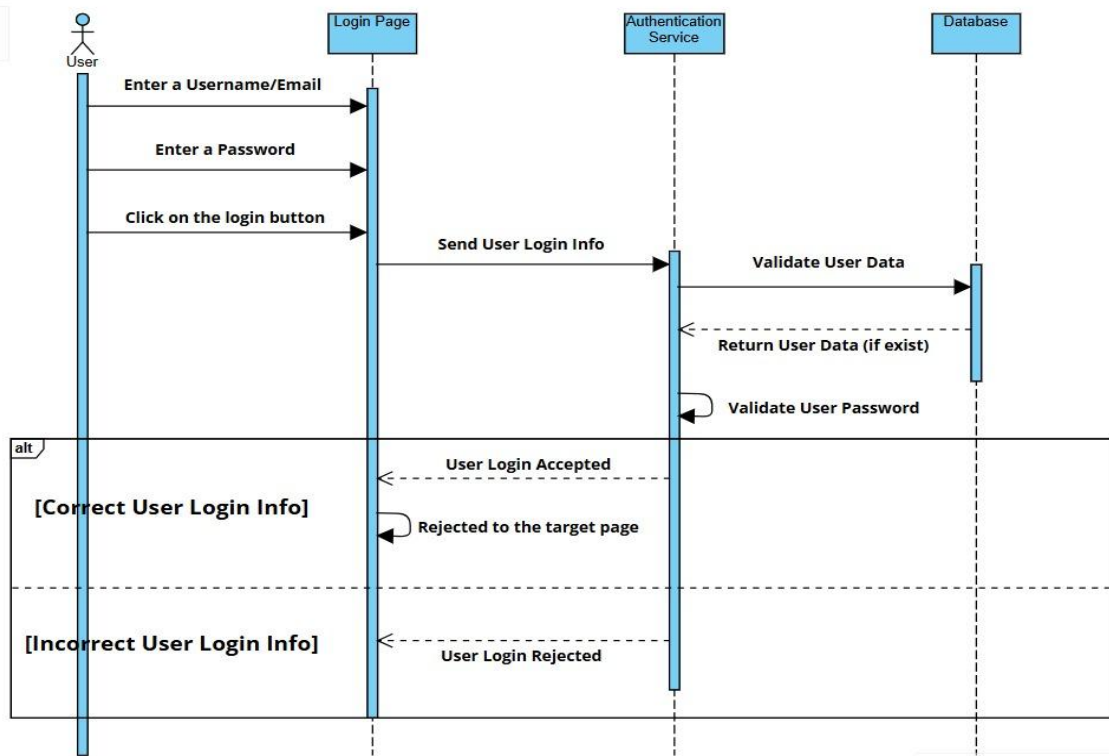


Figure 4.2: login Sequence Diagram.

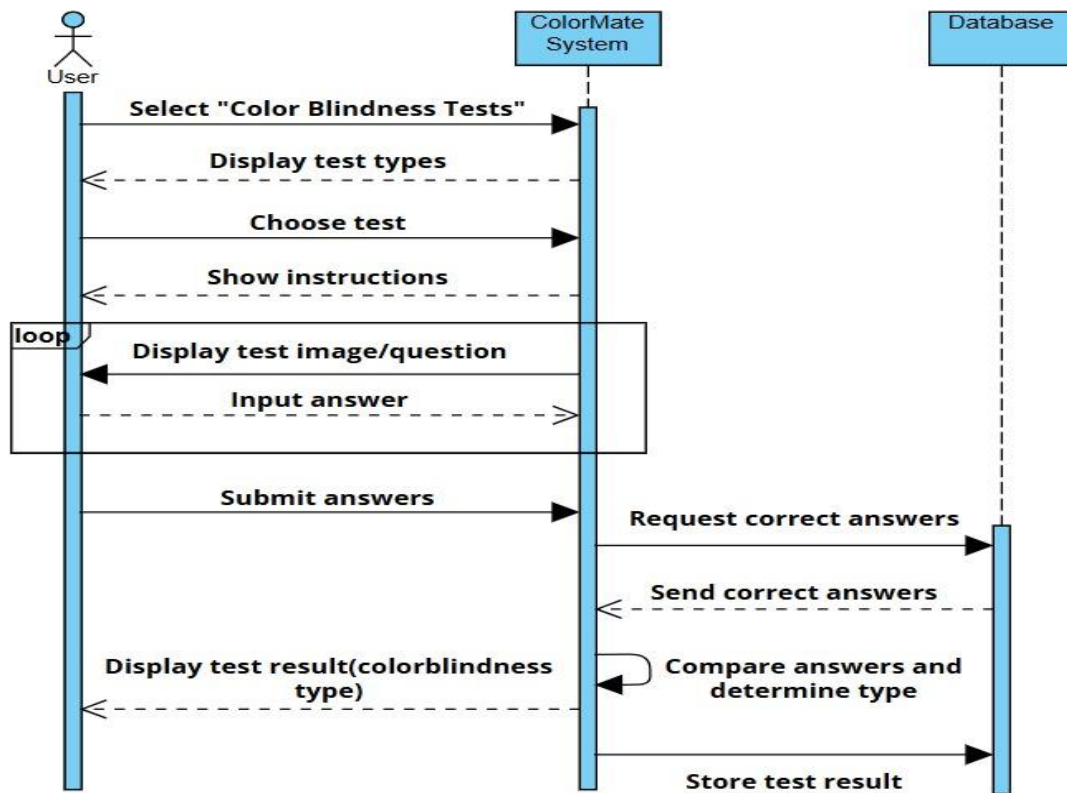


Figure 4.3: Take test Sequence Diagram.

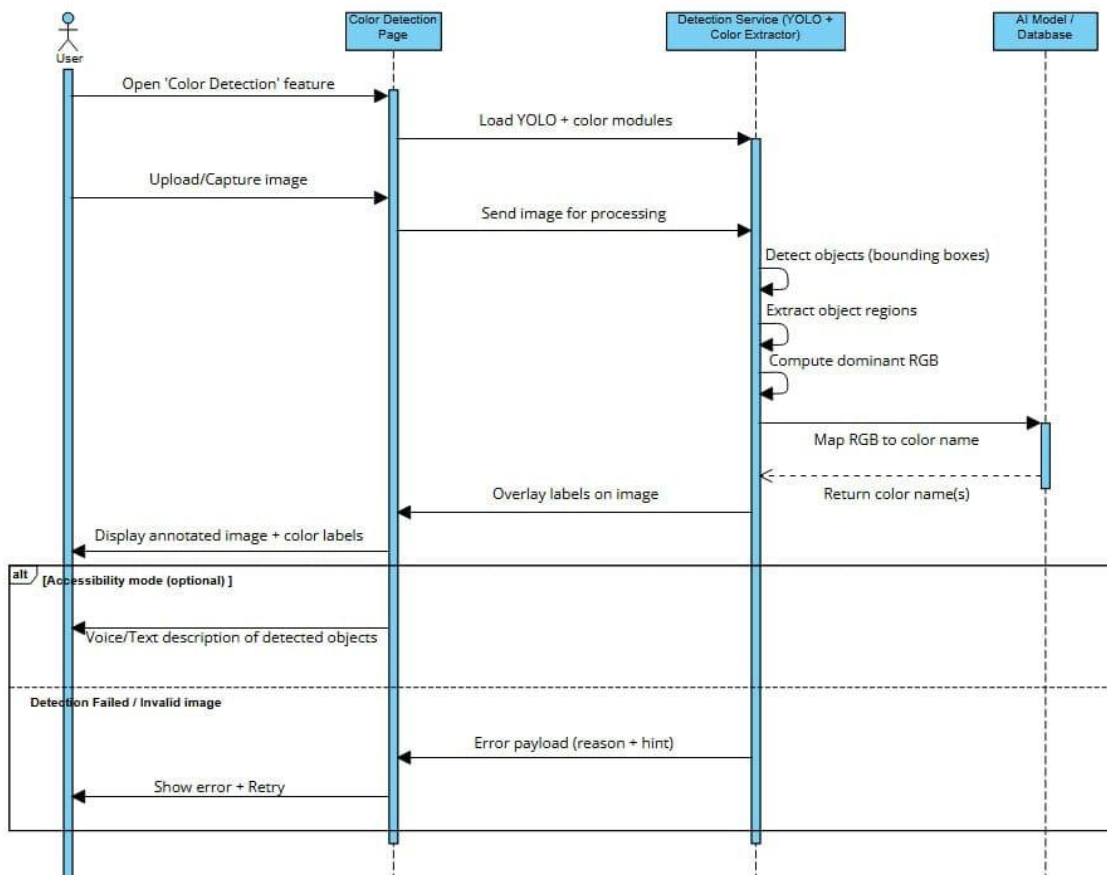


Figure 4.4: Detect objects and their colors Sequence Diagram.

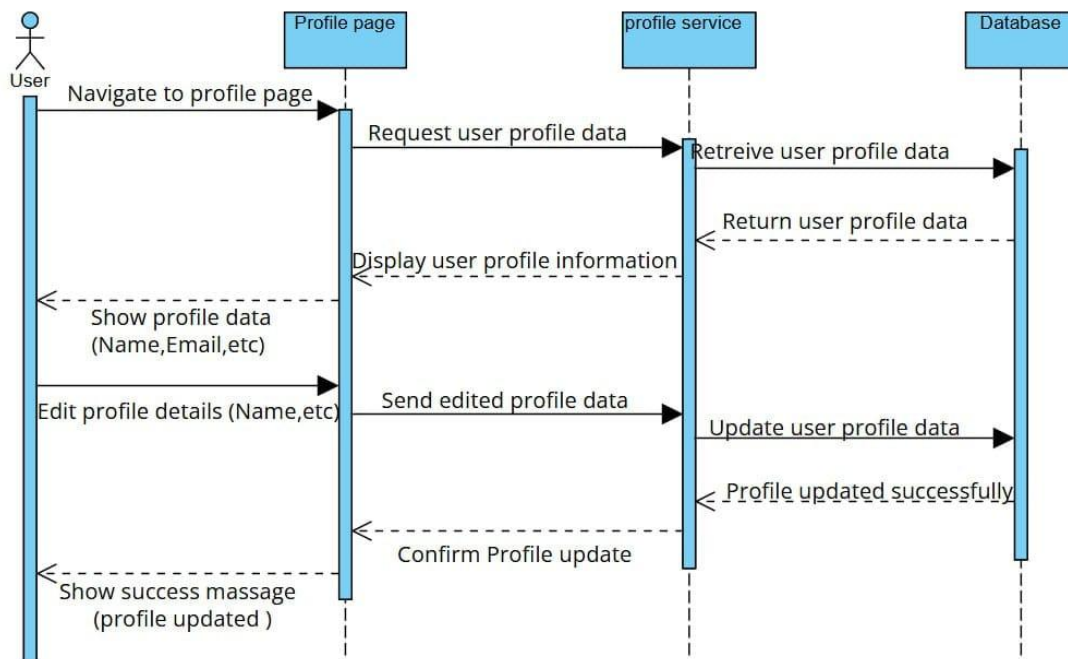


Figure 4.5: Edit profile Sequence Diagram.

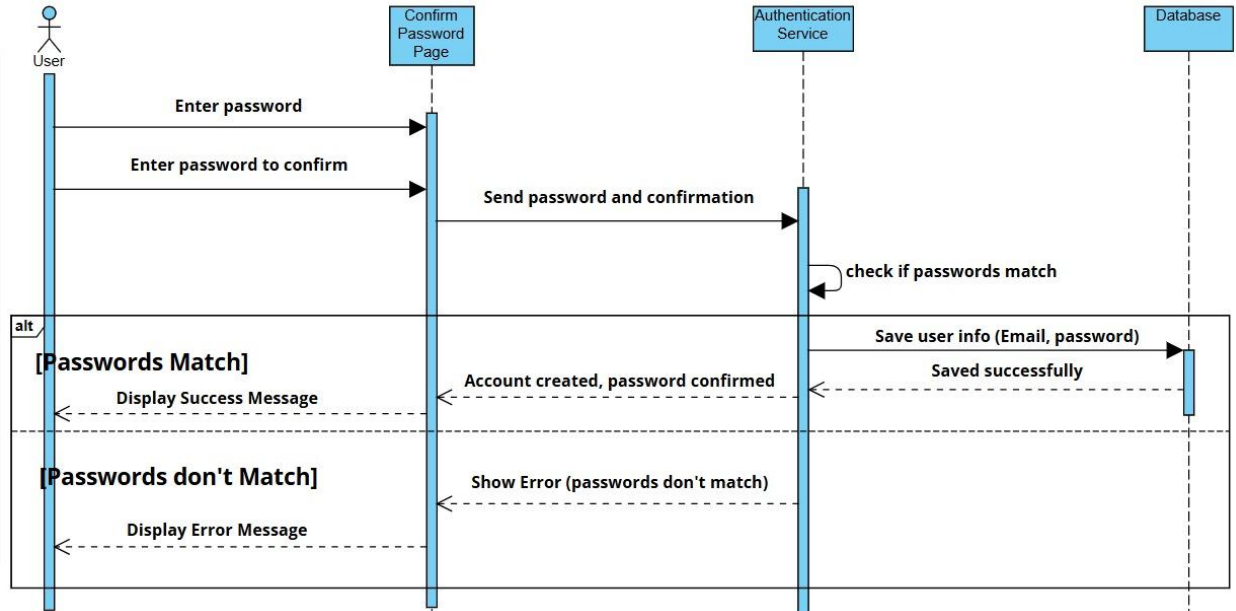


Figure 4.6: Confirm password Sequence Diagram.

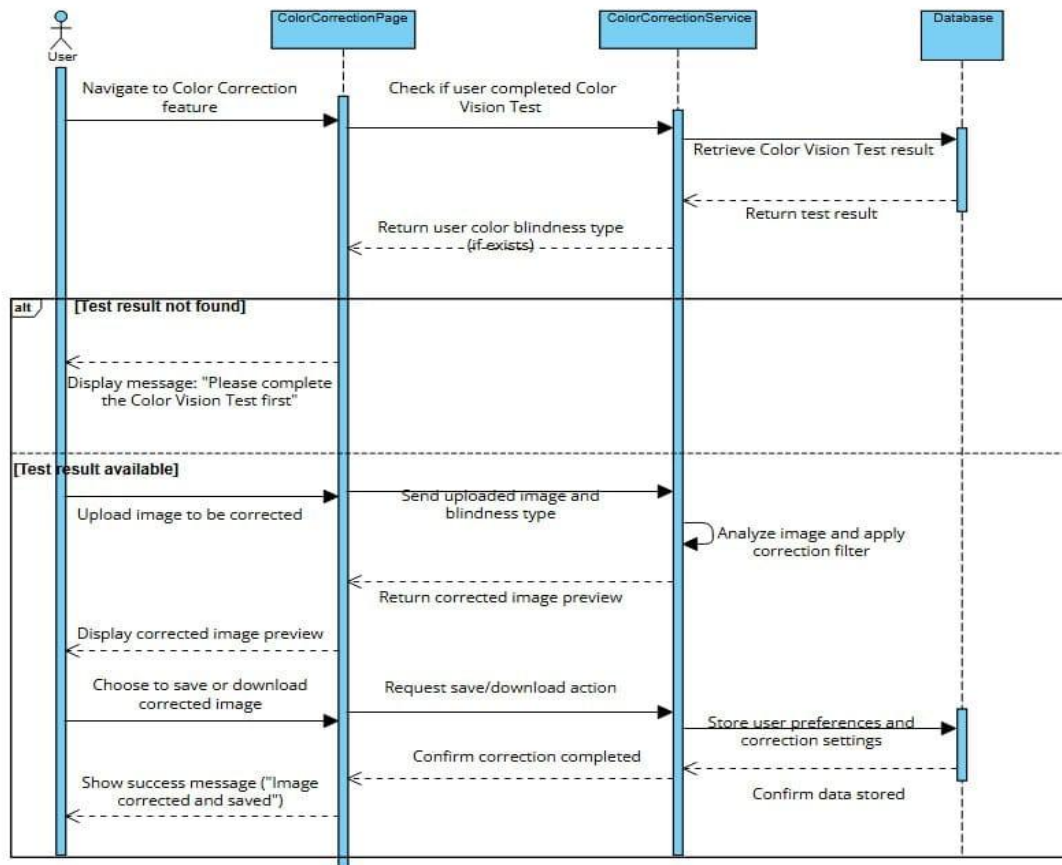


Figure 4.7: Perform color correction Sequence Diagram.

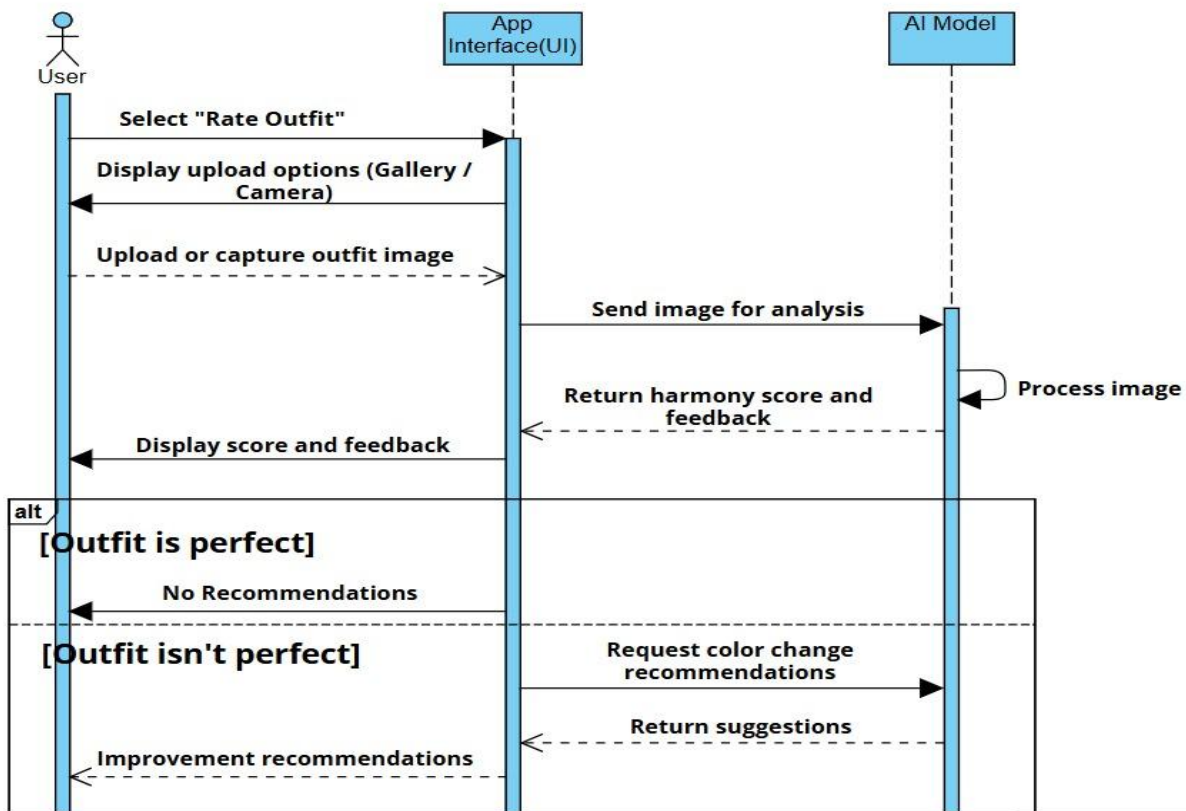


Figure 4.8: Rate outfit and recommend color improvements sequence daigr

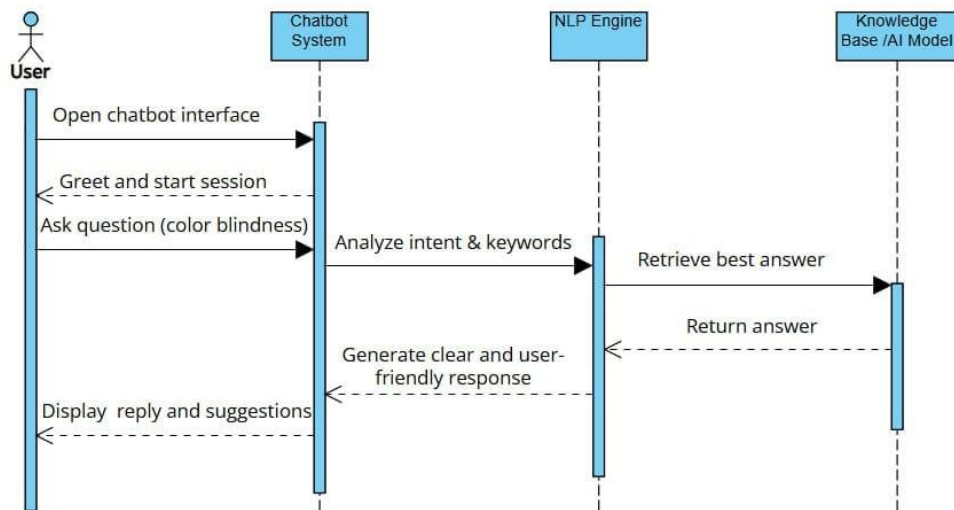


Figure 4.9: Ask Chatbot questions sequence daigram.

4.5 Use Case Daigram

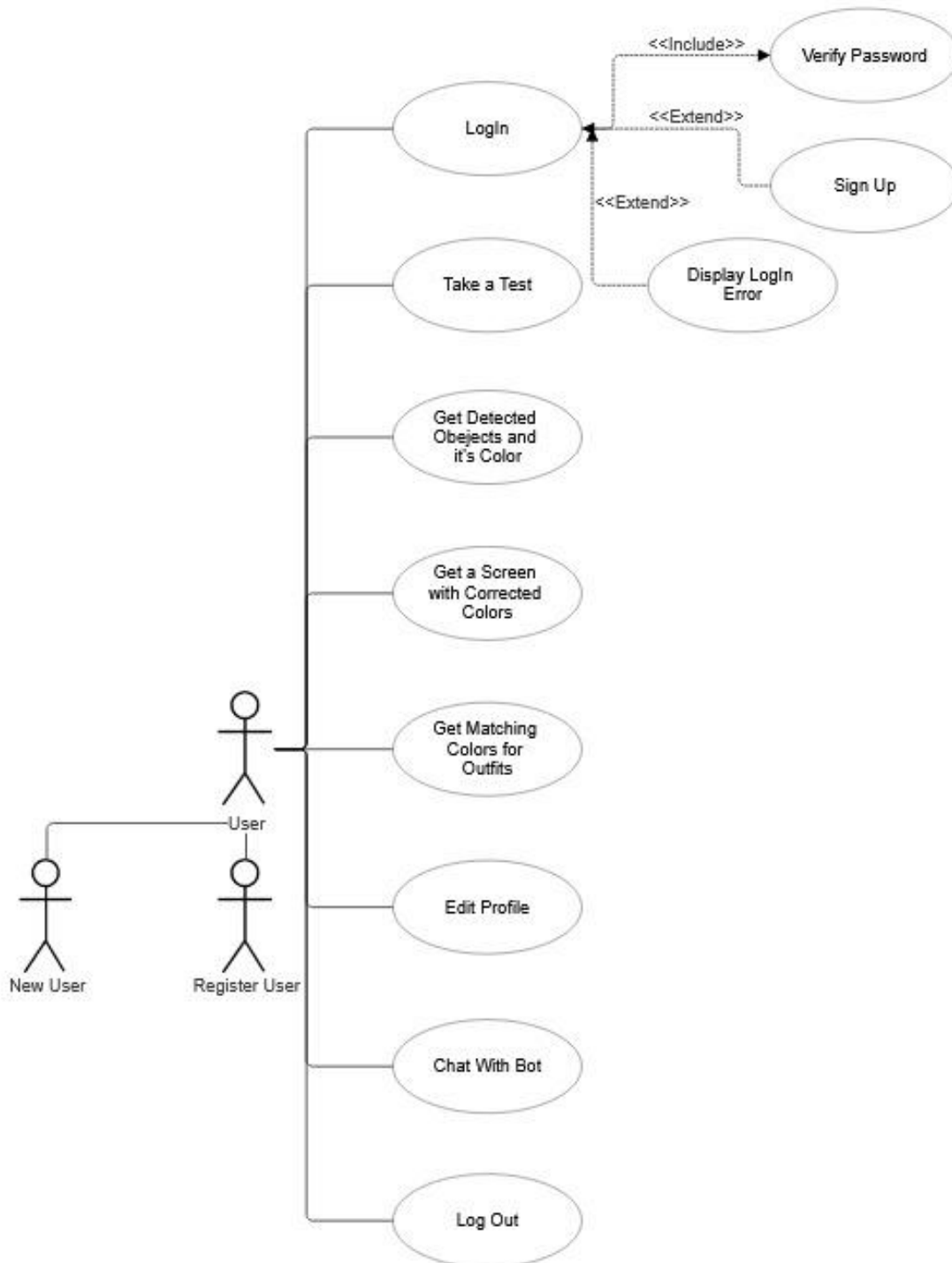


Figure 4.10: Shows the use case diagram for our application

4.6 Activity Daigram:

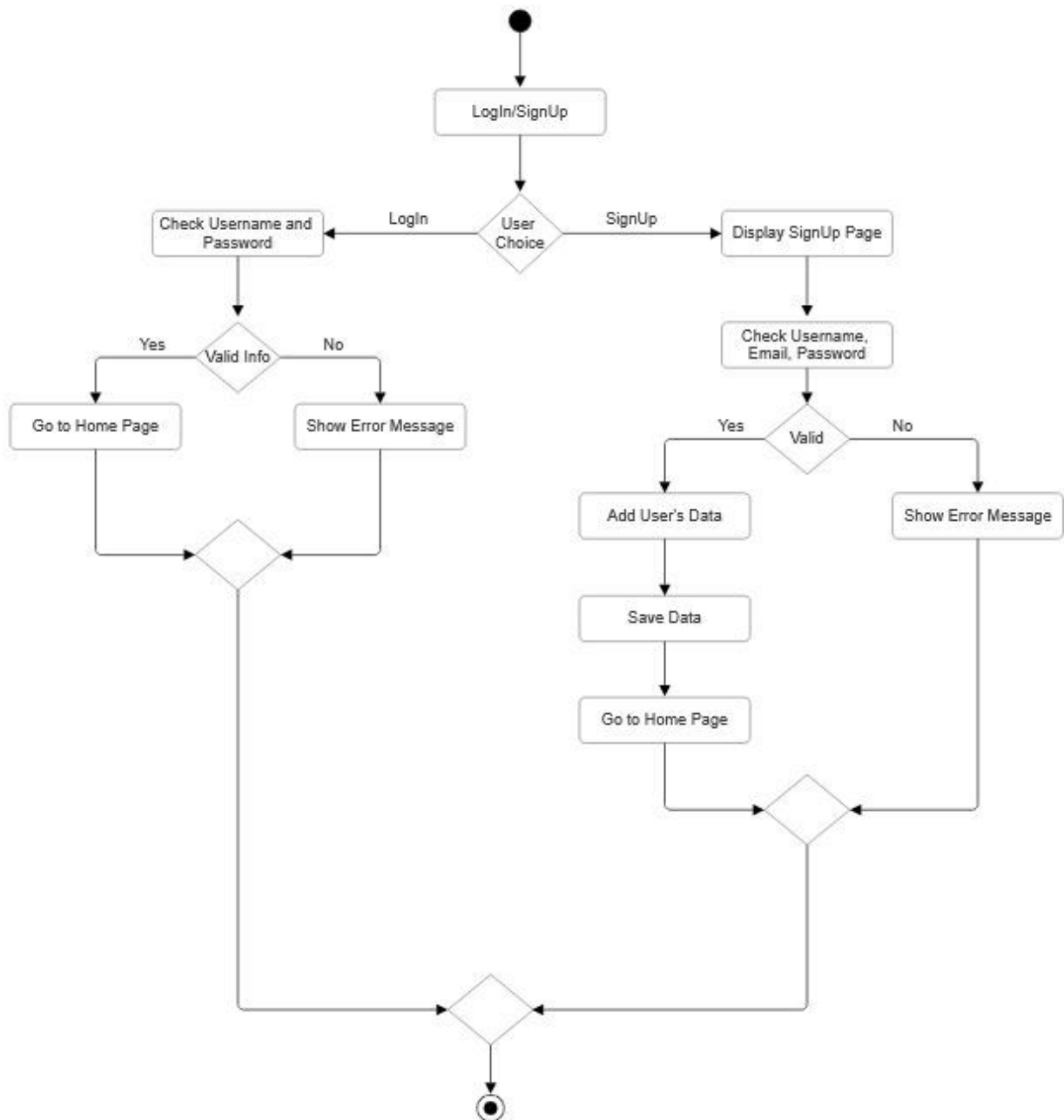


Figure 4.11: Login and sign-up Activity Diagram

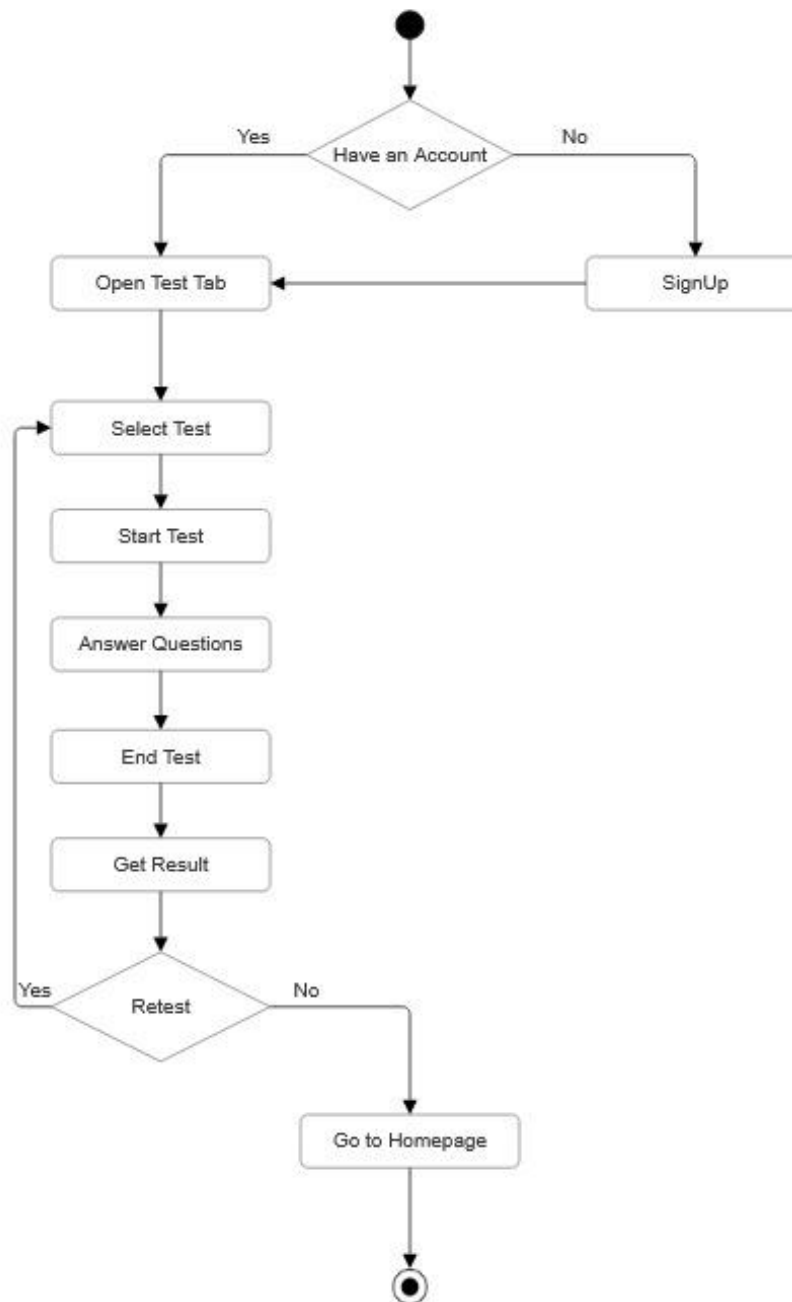


Figure 4.12: Take a test Activity Diagram

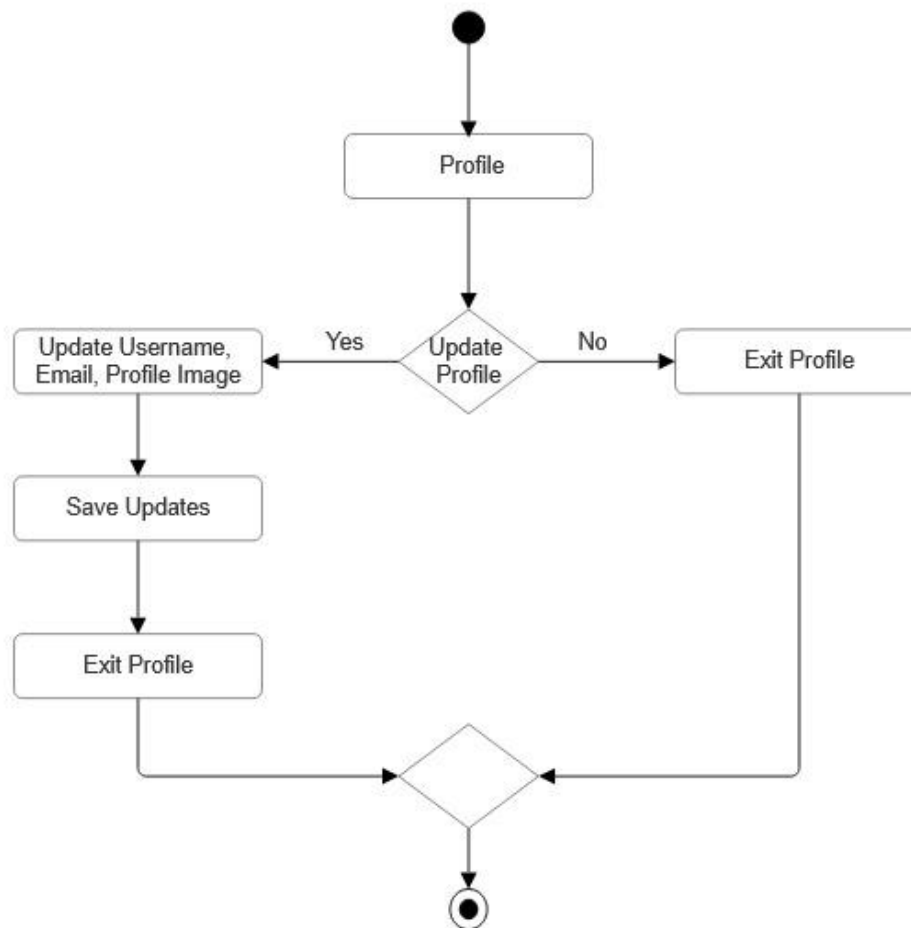


Figure 4.13: Edit profile Activity Diagram

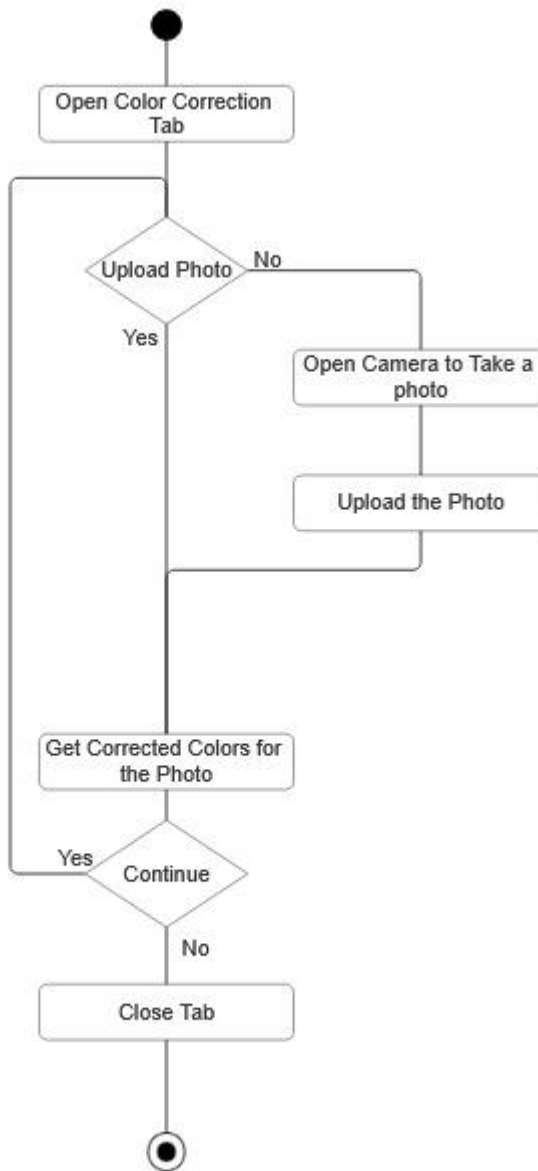


Figure 4.14: Perform color correction Activity Diagram.

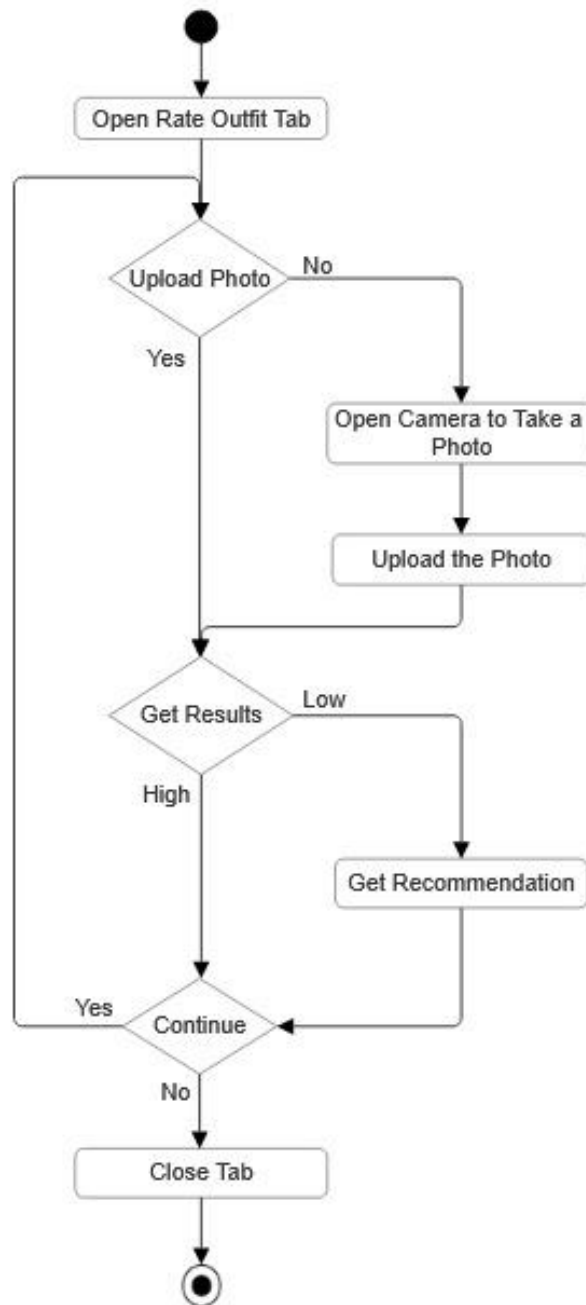


Figure 4.15: Rate outfit and recommend color improvements Activity Diagram.

4.7 Class Daigram

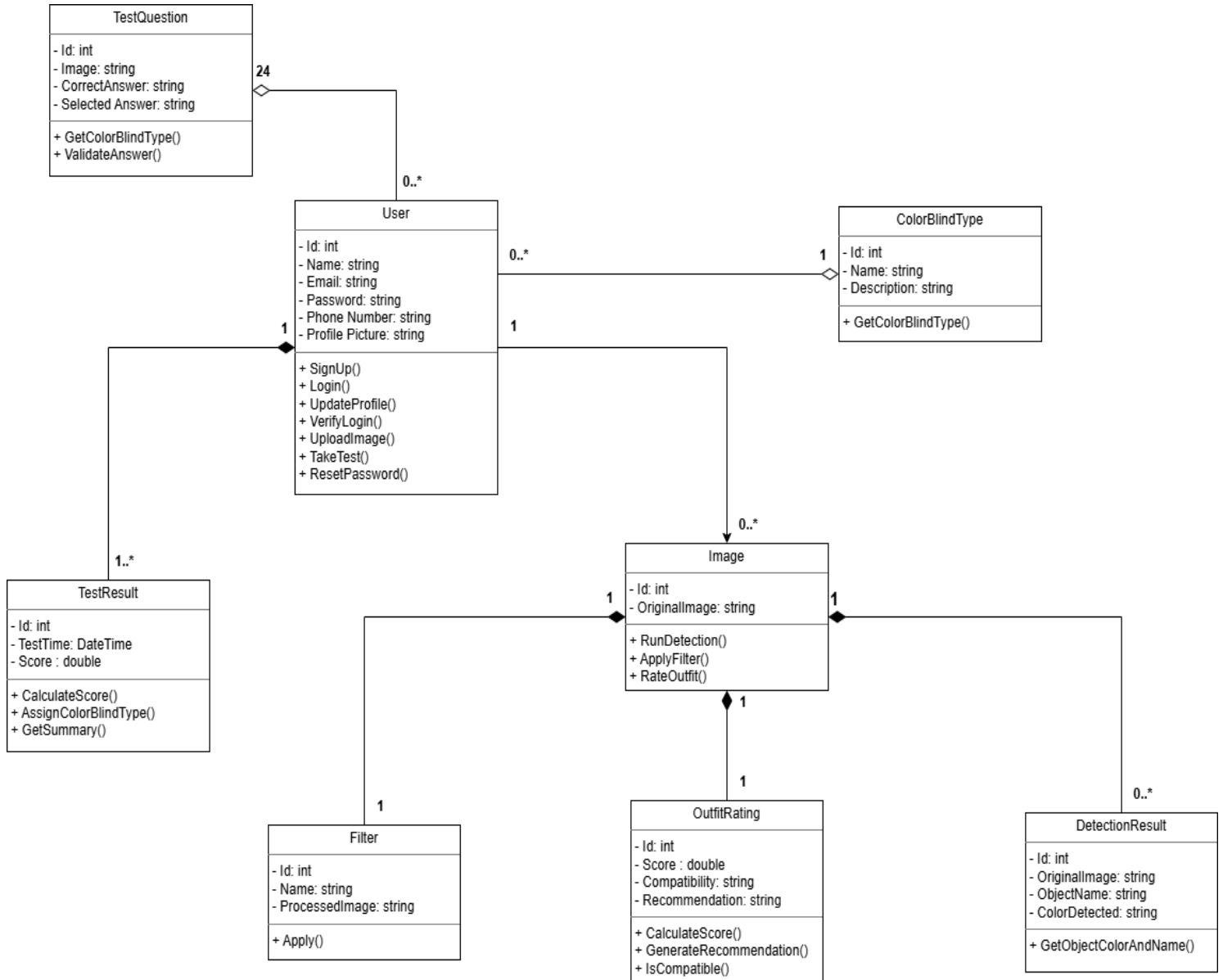


Figure 4.16: Class diagram.

4.8 Entity Relationship Daigram

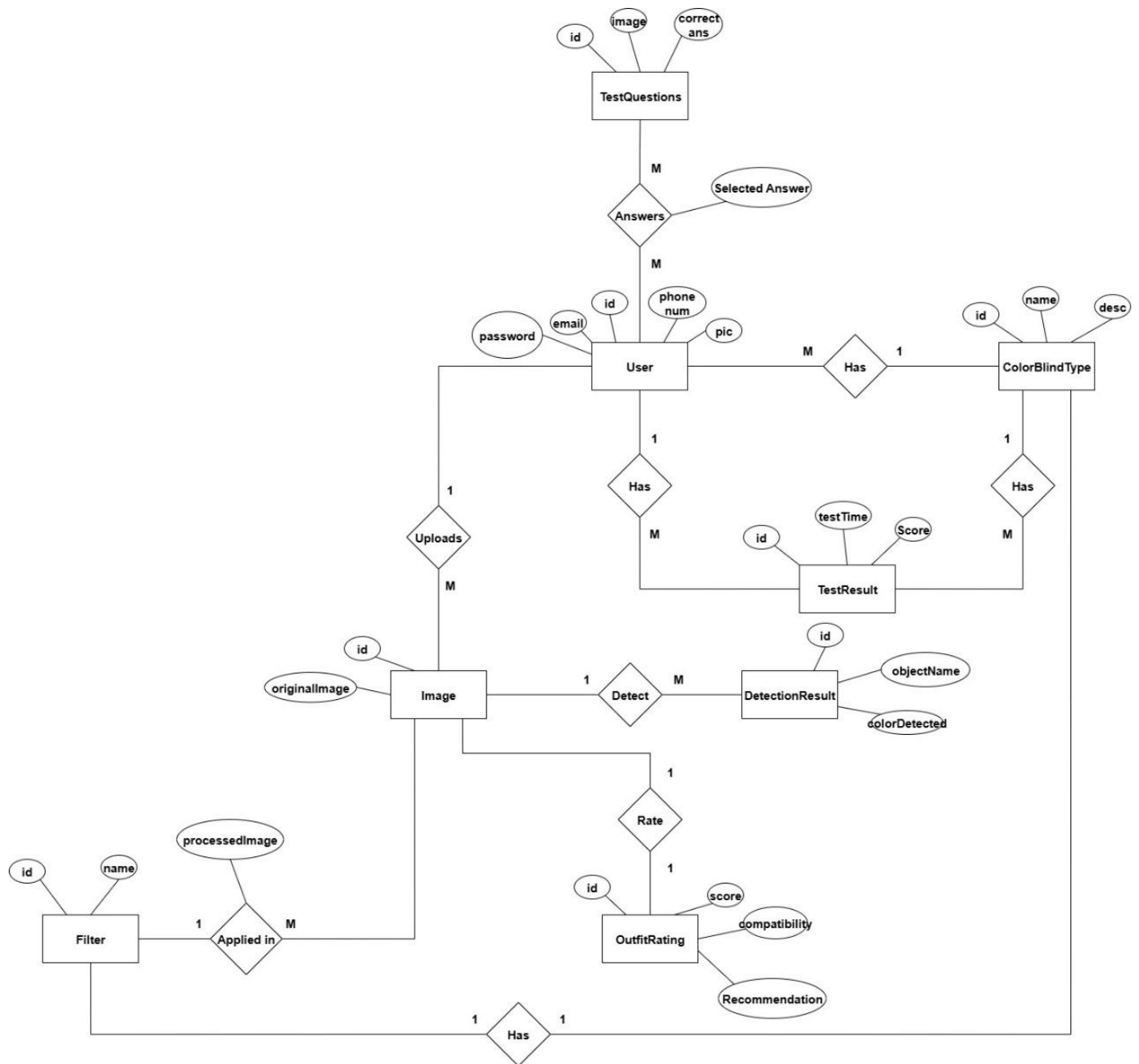


Figure 4.17: Entity Relationship Daigram

4.9 Context Daigram

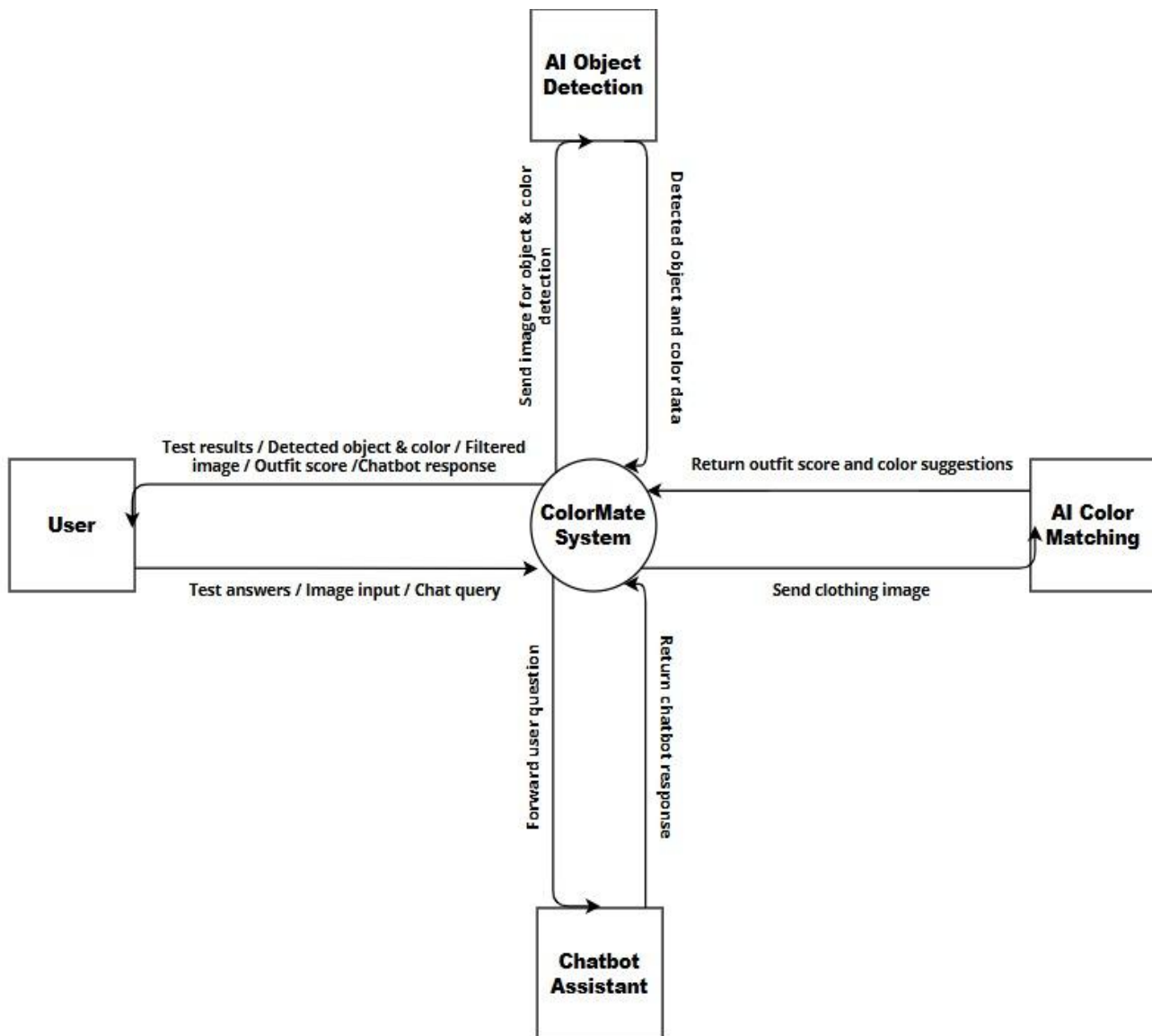


Figure 4.18: Context Daigram.

4.10 Conclusion

This chapter presented the system requirements and use case analysis for ColorMate, ensuring the development of a scalable, secure, and efficient e-learning platform. It detailed the hardware, software, and network specifications, highlighting the use of .NET Core, SQL Server, and Flutter to build a strong and reliable infrastructure. Key functionalities include user management, easy content access, discussion forums, feedback mechanisms, and AI-driven monitoring.

Additionally, the chapter defined the non-functional requirements, emphasizing performance, security, scalability, and usability to guarantee system reliability. The use case analysis illustrated real-world interactions to support smooth and engaging learning experiences. These foundational elements provide the groundwork for transforming ColorMate into a powerful and innovative digital learning platform.

Chapter five

5.1 Introduction

This chapter describes the high-level structural design of the ColorMate system. It outlines how various software and hardware components interact to provide a seamless experience for users with color vision deficiencies. The architecture is designed to be modular, scalable, and secure, ensuring efficient communication between the mobile client, backend APIs, and specialized AI microservices.

5.2 Logical Architecture (Three-Tier Architecture)

ColorMate follows a standard three-tier architecture to separate the user interface, business logic, and data storage.

- **Presentation Tier (Mobile Client):** Developed using the **Flutter** framework. This layer handles all user interactions, camera capturing for color detection, and the display of Ishihara test results.
- **Logic Tier (Application Server):** Built with **ASP.NET Core**. This layer processes requests from the mobile app, manages user authentication via **JWT**, and acts as an orchestrator that calls specialized AI models for object detection and fruit freshness analysis.
- **Data Tier (Database):** Uses **Microsoft SQL Server** to persistently store user profiles, historical test results, and saved outfit combinations.

5.3 AI & Computer Vision Architecture

The core "intelligence" of ColorMate is divided into specialized modules to handle different visual tasks.

- **5.3.1 Object and Color Detection Module:** This module utilizes **YOLOv8** for high-speed, real-time object recognition. Once an object (e.g., an apple) is detected, the system extracts the dominant color from the bounding box and maps it to a human-readable name using a color-naming algorithm.
- **5.3.2 Fruit Freshness Analysis Module:** This module uses a **Convolutional Neural Network (CNN)** trained on datasets of fresh and rotten produce. It analyzes surface textures and color patterns to provide a quality percentage to the user.
- **5.3.3 Outfit Compatibility Module:** Based on fashion compatibility research, this module evaluates color harmony. It uses machine learning logic

to determine if the colors of detected clothing items are compatible, providing a rating and suggestions for better matches.

- **5.3.4 Color Correction & Simulation Module:** This module applies mathematical transforms (Daltonization algorithms) to the live camera feed. It shifts "confusing" color frequencies (like Red/Green) into a spectrum that the specific user (Protanope or Deuteranope) can distinguish.

5.4 Chatbot Architecture

The **ColorMate Assistant** is a modular component designed for real-time support.

- **Backend:** A Python-based **Flask** microservice.
- **Model:** It leverages **Azure OpenAI (GPT-4.1)** to understand natural language queries about color blindness or app features.
- **Workflow:** The mobile app sends user text to the Flask API, which retrieves a response from the AI model and returns it to the UI in JSON format.

5.5 Security Architecture

To protect user data and ensure authorized access, ColorMate implements several security layers:

- **Authentication:** Uses **ASP.NET Identity** for secure account management.
- **Authorization:** Implements **JWT (JSON Web Tokens)**; every API request must carry a valid token to verify the user's identity.
- **Data Transmission:** All communications between the mobile app and the backend are encrypted using the **HTTPS** protocol.

5.6 Deployment Architecture

The system is designed for cloud deployment to ensure high availability.

- The **Web API** and **Database** are hosted on a cloud infrastructure (e.g., **Monster**).

- The **AI Models** (YOLOv8 and CNN) are optimized to run locally on the mobile device's GPU where possible to reduce latency, while more complex tasks (like Chatbot) are handled by remote cloud endpoints.

5.7 System Architecture Overview

The **ColorMate system architecture** is designed using a modular and scalable structure that integrates mobile application technologies with backend services and artificial intelligence models. The goal of this architecture is to ensure efficient processing of image-based tasks, secure communication between system components, and seamless interaction between users and the platform.

The system follows a **three-tier architecture model**, consisting of the presentation layer, application layer, and data layer. In addition, a dedicated **AI processing layer** is incorporated to handle computer vision and machine learning tasks.

The architecture enables different system components to work independently while maintaining smooth communication through well-defined APIs. This modular approach improves system maintainability, scalability, and performance.

The main components of the ColorMate architecture include:

- **Mobile Application (Flutter)**
- **Backend Services (ASP.NET Core API)**
- **AI Processing Modules (Python-based models)**
- **Database System (SQL Server)**

Each component plays a specific role in processing user requests, performing analysis tasks, and returning results to the user interface.

5.8 System Architecture Diagram

The system architecture consists of multiple layers that work together to provide intelligent color analysis services.

1. User Layer

- Represents the end users interacting with the mobile application.
- Users perform actions such as taking color blindness tests, capturing images, or analyzing clothing compatibility.

2. Presentation Layer

- Implemented using **Flutter**.
- Responsible for displaying the user interface and collecting user inputs.
- Handles image capture, data visualization, and chatbot interaction.

3. Application Layer

- Implemented using **ASP.NET Core Web API**.
- Processes user requests and handles system logic.
- Communicates with AI modules and the database.

4. AI Processing Layer

- Contains machine learning models responsible for performing image analysis and predictions.
- Includes object detection, color recognition, fruit freshness classification, and outfit compatibility analysis.

5. Data Layer

- Uses **SQL Server** to store user accounts, test results, historical analysis data, and system logs.

This layered architecture ensures separation of concerns, allowing each component to evolve independently without affecting other parts of the system.

5.9 API Communication Architecture

Communication between the mobile application and backend services is handled through a **RESTful API architecture**. The API acts as the central communication interface that connects the frontend application with the backend logic and AI processing modules.

When a user performs an action within the mobile application, such as uploading an image or requesting analysis, the application sends an HTTP request to the backend API. The API processes the request, invokes the required AI module if necessary, and returns the results in a structured JSON response.

Table 5.1: Key API endpoints

Endpoint	Method	Description
/api/auth/register	POST	Registers a new user account
/api/auth/login	POST	Authenticates users

/api/color/detect	POST	Detects objects and colors in an image
/api/fruit/analyze	POST	Determines fruit freshness
/api/outfit/analyze	POST	Evaluates outfit color compatibility
/api/test/result	GET	Saves color blindness test results

Using RESTful APIs allows the system to remain platform-independent and enables future integration with additional applications or services.

5.10 Summary

The system architecture of ColorMate integrates modern mobile technologies, backend services, and artificial intelligence models to create an intelligent color assistance platform. By adopting a layered architecture and modular AI components, the system achieves flexibility, scalability, and efficient processing of complex image analysis tasks.

The combination of Flutter-based mobile interfaces, ASP.NET Core backend services, and machine learning models enables the system to provide accurate analysis and helpful insights for users. Additionally, the use of RESTful APIs and cloud deployment strategies ensures that the platform remains scalable and adaptable to future enhancements.

Chapter six

6.1 Overview

This chapter discusses the practical steps and phases taken during the implementation of ColorMate. The system is built using a modular, scalable, and layered architecture that supports mobile-based interaction and efficient backend integration. The implementation involved developing a user-friendly mobile application designed specifically for individuals with color vision deficiencies, along with seamless integration with backend services and AI-powered modules.

6.2 System Architecture

The ColorMate application was designed using a multi-layer architecture that separates system components based on their responsibilities. This design approach enhances scalability, maintainability, and overall system performance.

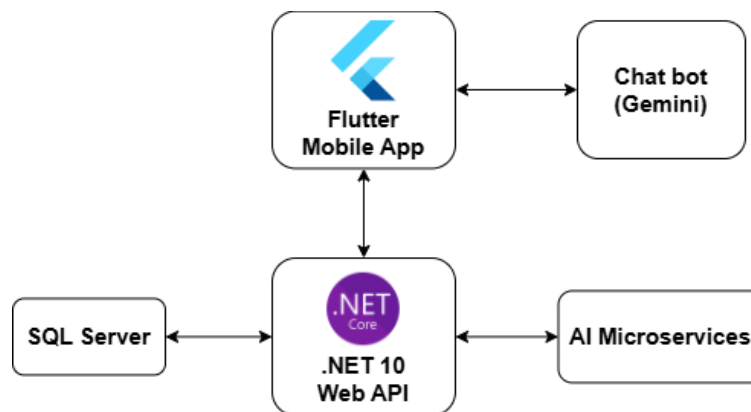


Figure 6.1: ColorMate System Architecture.

6.2.1 System Architecture Description

The architecture consists of the following layers:

- **Frontend Layer:**

Developed using Flutter to build a cross-platform mobile application that is intuitive, accessible, and user-friendly for individuals with color vision deficiencies. It provides interfaces for features such as color detection, color blindness testing, image correction, outfit analysis, and chatbot interaction.

- **Backend Layer:**

Implemented using ASP.NET Core Web API, it contains the main business logic, handles authentication and authorization, manages communication with the database, and coordinates requests between the mobile application and AI modules.

- **Database Layer:**

SQL Server is used to store structured data including user accounts, color blindness test results, detected images, outfit ratings, chatbot interactions, and user activity history.

- **AI and Computer Vision Layer:**

This layer is developed using Python and is responsible for integrating intelligent functionalities such as:

- Object using deep learning models.
- Outfit color harmony evaluation and recommendation.
- Fruit freshness detection using image classification models.

6.3 Technologies Used

The following table outlines the core technologies used in implementing ColorMate:

Table 6.1: ColorMate Technologies Used.

Technology	Purpose
Flutter & Dart	Mobile app UI/UX and logic
ASP.NET Core	Backend API development
Entity Framework Core	ORM for database operations
SQL Server	Relational database
Python	AI services (e.g., Object & Color Detection)
Postman	API testing and documentation
GitHub	Source control and collaboration
JWT Authentication	Secure access and role-based authorization

6.4 Backend Implementation Details

- **RESTful API:**

The backend exposes RESTful APIs following best practices (HTTP verbs, statelessness, etc.) and uses proper status codes for responses. Each API is secured using JWT-based tokens.

- **Dependency Injection:**

ASP.NET Core's built-in DI container was used to inject services like repositories, database contexts, and utility classes.

- **Exception Handling & Logging:**

A centralized exception handler is implemented using middleware to capture and log all runtime errors.

- **Modular Design:**

Services are separated into modules (UserService, TestService, ObjDetectionService, etc.), making the codebase maintainable and easy to scale.

6.5 Database Implementation

- **SQL Server** relational database
 - **ERD designed** for scalability
 - **Tables for:** Users, Questions, Results, Images, etc.
-

6.6 AI Modules Implementation

The ColorMate system is built around three core AI models, each responsible for a distinct task within the overall pipeline:

1. Fruit Freshness Classification Model (MobileNetV3Large) – a fine-tuned classifier that determines whether a fruit or vegetable is healthy or rotten, supporting automated freshness detection.
2. Perceptual Color Harmony System (PCHS) – a context-aware scoring framework that evaluates how well the colors in an outfit work together and generates recommendations to improve overall color harmony.
3. Object Detection Model (YOLOv8) – a pretrained model used to detect and localize objects and clothing items in real time, enabling the system to identify items before analyzing their colors.

Together, these three models allow ColorMate to detect items, assess produce quality, and evaluate fashion color compatibility within a single integrated system.

6.6.1 Fruit Model Architecture and Implementation:

Dataset [7]:

The model was trained on the Fruit and Vegetable Disease – Healthy vs Rotten Dataset mentioned in **Chapter 3 (3.4 Dataset Used)** , which includes labeled images divided into:

- Healthy
- Rotten

The dataset consisted of 28,645 images and was split using an 80/20 train-test ratio. Furthermore, 10% of the training portion was reserved for validation during model development. This resulted in 20,624 training images, 2,292 validation images, and 5,729 testing images as mentioned in **Table 6.2**. The validation set was used to monitor model performance and tune hyperparameters during training, while the independent test set was reserved for the final evaluation on unseen data. Class weights were computed and applied during training to compensate for the imbalance between classes (Healthy weight: 1.064, Rotten weight: 0.943).

Table 6.2 Fruit and Vegetable Dataset Classes

Dataset Classes	Number Of Images In Dataset
Healthy	13459
Rotten	15186

Table 6.3 Fruit and Vegetable Dataset Spilt

Dataset Subset	Percentage	Number of Images
Training	72%	20,624
Validation	8%	2,292
Testing	20%	5,729
Total	100%	28,645

To improve robustness and prevent overfitting, the following **data augmentation** techniques were applied to the training set:

- Random rotations (up to 20°).
- Width and height shifts (up to 15%).
- Shear transformations (up to 10%).
- Zoom (up to 20%).
- Horizontal flipping.
- Brightness adjustments (range: 0.8 to 1.2).

Fruit Model Architecture Overview:

As Shown in **Figure 6.3** the model accepts input images resized to $224 \times 224 \times 3$ (RGB). MobileNetV3Large is built upon bottleneck blocks that use hard-swish activations and squeeze-and-excitation modules, which improve feature expressiveness while keeping computation efficient. The pretrained base is extended with a custom classification head designed specifically for binary classification:

1- Input:

- Image format: RGB images
- Preprocessing:
 - (mobilenet_v3.preprocess_input) function, which normalizes pixel values to the range expected by the pretrained MobileNetV3Large weights.
 - Image Resize to : 224×224 pixels

2- MobileNetV3 Base (Feature Extractor) :

- Architecture: MobileNetV3-Large architecture pretrained on the ImageNet dataset.
- Layer Configuration: Utilizes the entire base network, spanning layers 0 through 187 (187 layers in total).
- Fine-Tuning: Rather than freezing the base network to use purely as a static feature extractor, all 187 layers are unfrozen and fine-tuned during

3- The classification head consists of:

- Global Average Pooling to reduce spatial feature maps into a single vector.
- Batch Normalization to stabilize activations and accelerate convergence.
- Fully Connected Layer (512 units, ReLU activation) with L2 regularization to capture complex patterns while preventing overfitting.
- Dropout (rate = 0.4) for regularization.
- Fully Connected Layer (128 units, ReLU activation) with L2 regularization.
- Dropout (rate = 0.3).
- Output neuron with Sigmoid activation producing a probability value for binary classification.

4- Output:

- A probability score in the range $[0, 1]$.
- Values below 0.5 are classified as Healthy; values at or above 0.5 are classified as Rotten.
- Final prediction is determined by thresholding the sigmoid output at 0.5.

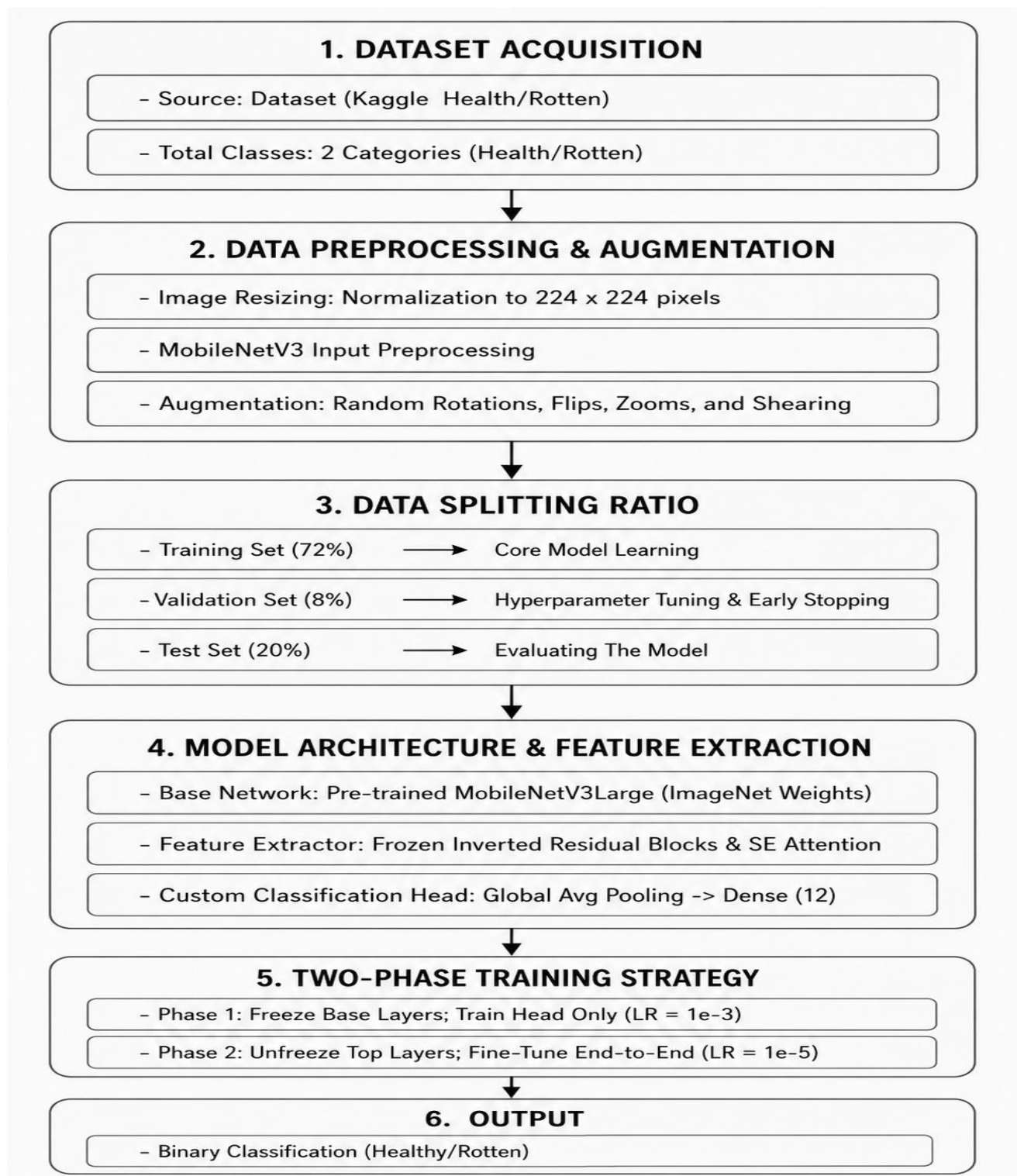


Figure 6.2: Fruit Model System Architecture.

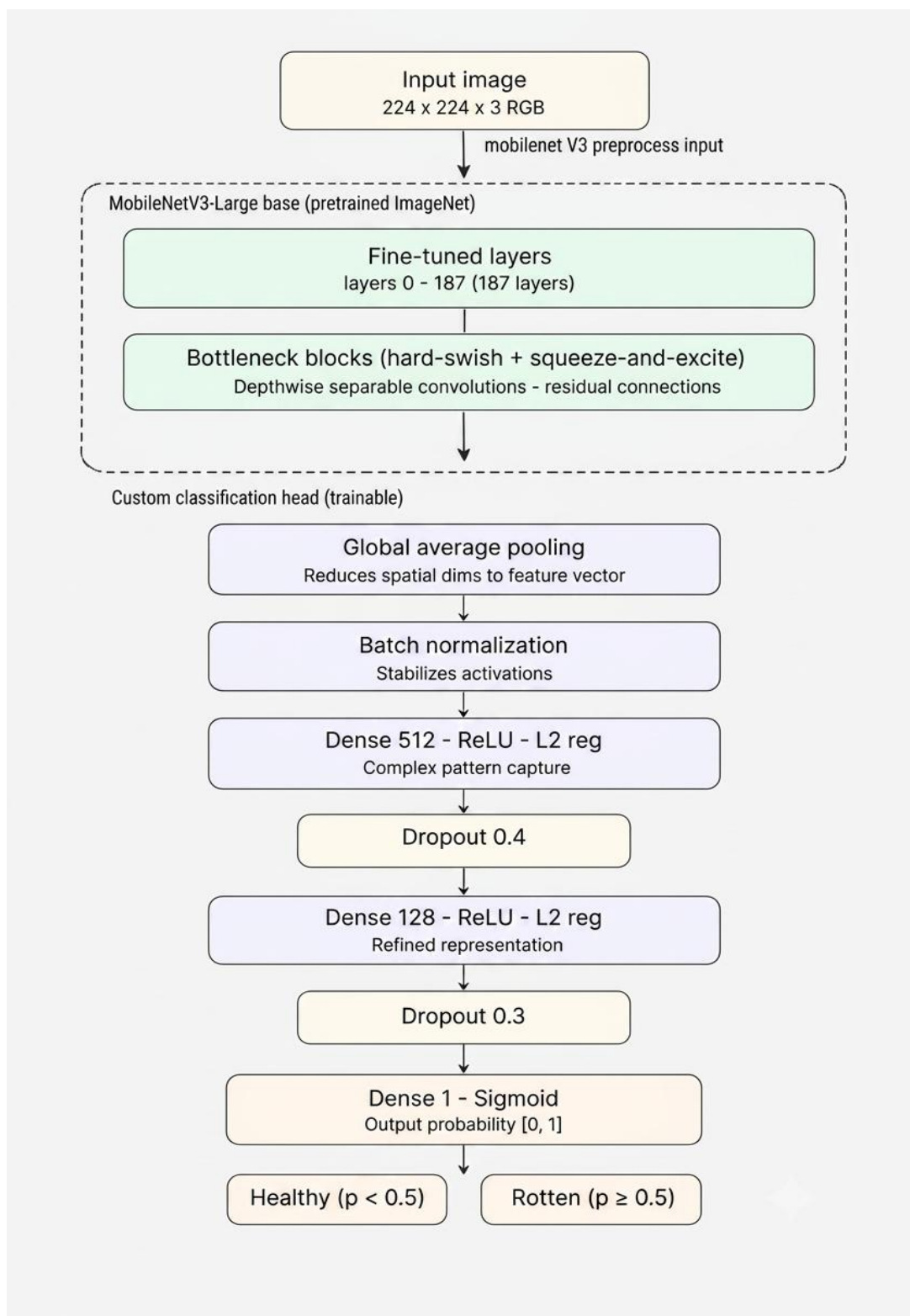


Figure 6.3: Fruit Model Architecture.

Model Training:

To enhance performance and reduce training time, a transfer learning strategy was adopted using a pretrained MobileNetV3Large model trained on the ImageNet dataset.

Training Configuration:

Table 6.4: Training Hyperparameters and Configuration

Hyperparameter	Value
Optimizer	Adam
Phase 1 Learning Rate	(1×10^{-3})
Phase 2 Learning Rate	(3×10^{-5})
EarlyStopping	Enabled
EarlyStopping Patience (Phase 1)	5
EarlyStopping Patience (Phase 2)	6
Loss Function	Binary Crossentropy + Label Smoothing (0.05)
Evaluation Metrics	Accuracy, AUC, Precision, Recall
Batch Size	32
EarlyStopping Patience	5 (Phase 1), 6 (Phase 2)
ReduceLROnPlateau Factor	0.3
ReduceLROnPlateau Patience	3

Training Strategy:

The model was trained in two sequential phases:

- **Phase 1—Head Training:** The MobileNetV3Large base model was fully frozen to retain its pretrained ImageNet weights. Only the custom classification head was trained. This phase used a learning rate of $1e-3$ with the Adam optimizer, running for up to

15 epochs with early stopping (patience = 5). The best model checkpoint achieved a validation accuracy of 93.98%.

- **Phase 2—Fine-Tuning:** The top layers of the base model (from 187 out of 187 layers) were unfrozen. The model was retrained end-to-end using a very low learning rate of $3e-5$ to carefully adapt the high-level features to the fruit classification task without destroying the pretrained representations. This phase ran for up to 30 epochs with early stopping (patience = 6).

This two-phase approach allowed the model to first establish a strong classification head before adapting the deep feature extractors to the specific domain.

Results:

After execution of the two-phase transfer learning framework and systematic hyperparameter tuning using the validation subset, the final fine-tuned MobileNetV3Large model was comprehensively evaluated using the completely unseen, independent test subset to verify generalization capabilities. The quantitative evaluation metrics achieved during testing are systematically organized and presented in **Table 6.5**:

Table 6.5: Fruit Model Results Table.

Metric	Healthy	Rotten	Overall
Precision	0.9757	0.9289	0.9523
Recall	0.9253	0.9770	0.9512
F1-Score	0.9498	0.9523	0.9511
AUC	—	—	0.9781
Accuracy	—	—	95.12%

Empirical Training and Optimization Trajectory

The system's optimization behavior across both distinct training phases (Head Training and Full-Base Fine-Tuning) is illustrated in **Figure 6.4 (Model Training Curves)**:

- **Loss Convergence:** The convergence profiles of both the training and validation loss curves illustrate a steady, synchronized decline without anomalous cross-over gaps or divergent trajectories. This proves that the combination of structured L2 regularization, applied dropout rates (0.4 and 0.3), and Label Smoothing (0.05) effectively suppressed overfitting.
- **Accuracy Stabilization:** The validation accuracy trajectory closely follows the training accuracy profile, reaching an operational plateau. This validates that the two-phase optimization schedule safely adjusted high-level spatial definitions within the deep convolutional layers without destroying the original foundational ImageNet weights.

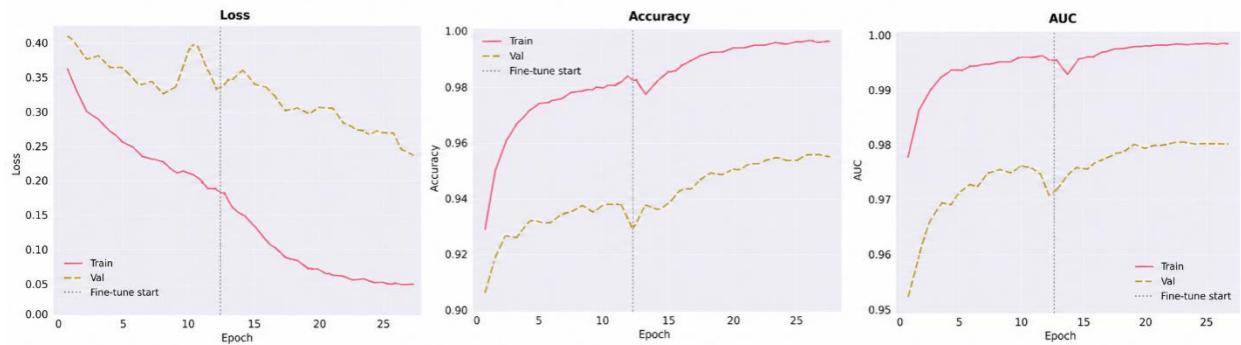


Figure 6.4: Fruit Model Training Curves.

Discriminative Power and Threshold Independence

The threshold-independent performance of the classification head is visually evaluated via **Figure 6.5 (ROC Curve)**:

- The Receiver Operating Characteristic (ROC) curve displays a sharp, optimal trajectory towards the upper-left coordinate.
- The model achieves a Macro **Area Under the Curve (AUC) of 0.9781**. This high value signifies an exceptionally low probability of false alarms. It indicates that the model's custom classification layers successfully separate the underlying latent distributions of healthy and rotten tissue features across varying ambient lighting conditions.

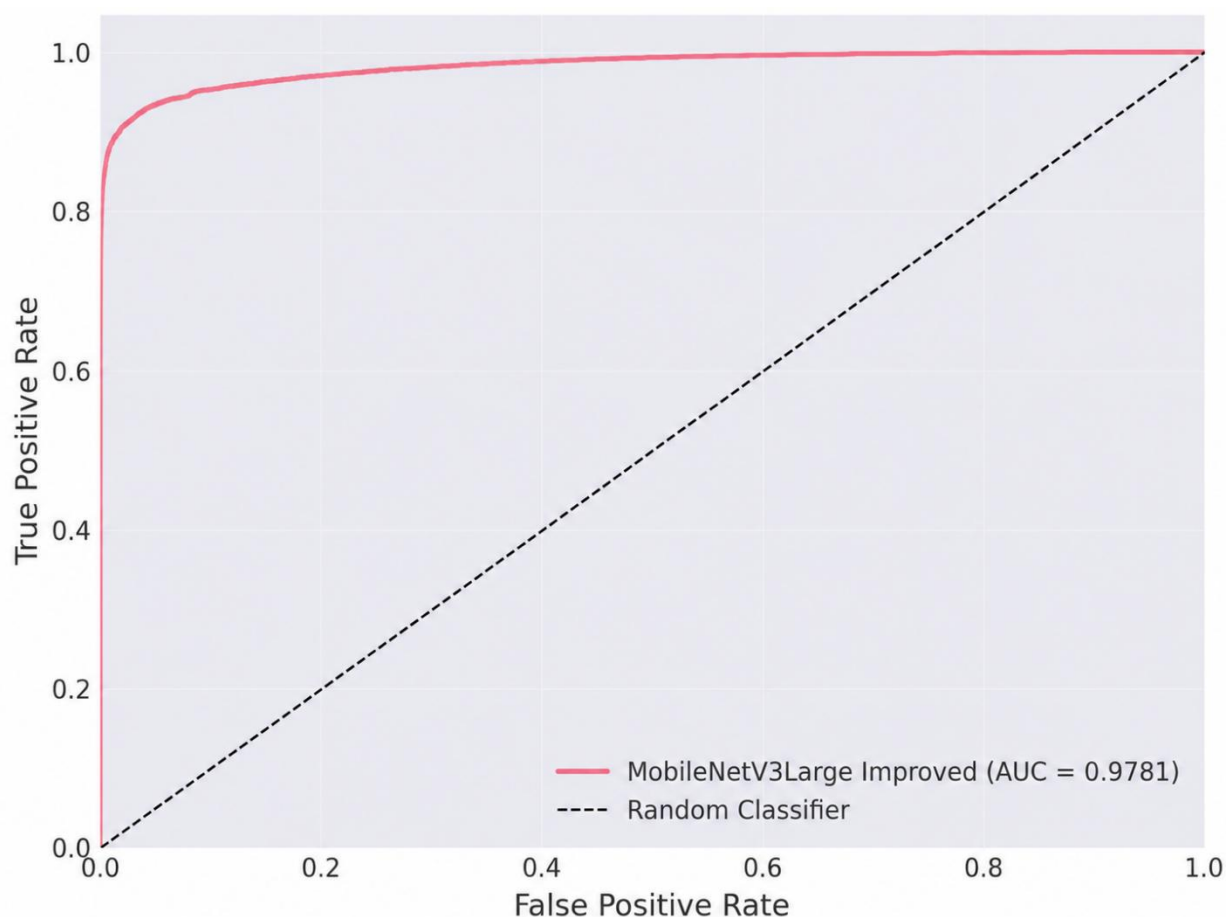


Figure 6.5: Fruit Model Roc Curve.

Class-Specific Misclassification Analysis

To examine the granular prediction behavior at the binary decision threshold (0.5), the actual versus predicted outputs were cross-tabulated in **Figure 6.6 (Confusion Matrix)**.

- **True Positive Rates (Sensitivity):** The model demonstrates an exceptional **Recall of 0.9770 for the Rotten class**, successfully capturing nearly all degraded produce items. This behavior minimizes dangerous False Negatives (passing a rotten fruit as healthy), which is vital for providing reliable real-world assistance to color-blind users.
- **Precision Trade-off:** The **Healthy class exhibits an outstanding Precision of 0.9757**, verifying that when the system flags a fruit as healthy, it is highly accurate. The slight trade-off observed (higher recall for rotten vs. higher precision for healthy) validates the utility of the applied class weights (Healthy: 1.064, Rotten: 0.943) in countering minor dataset imbalances

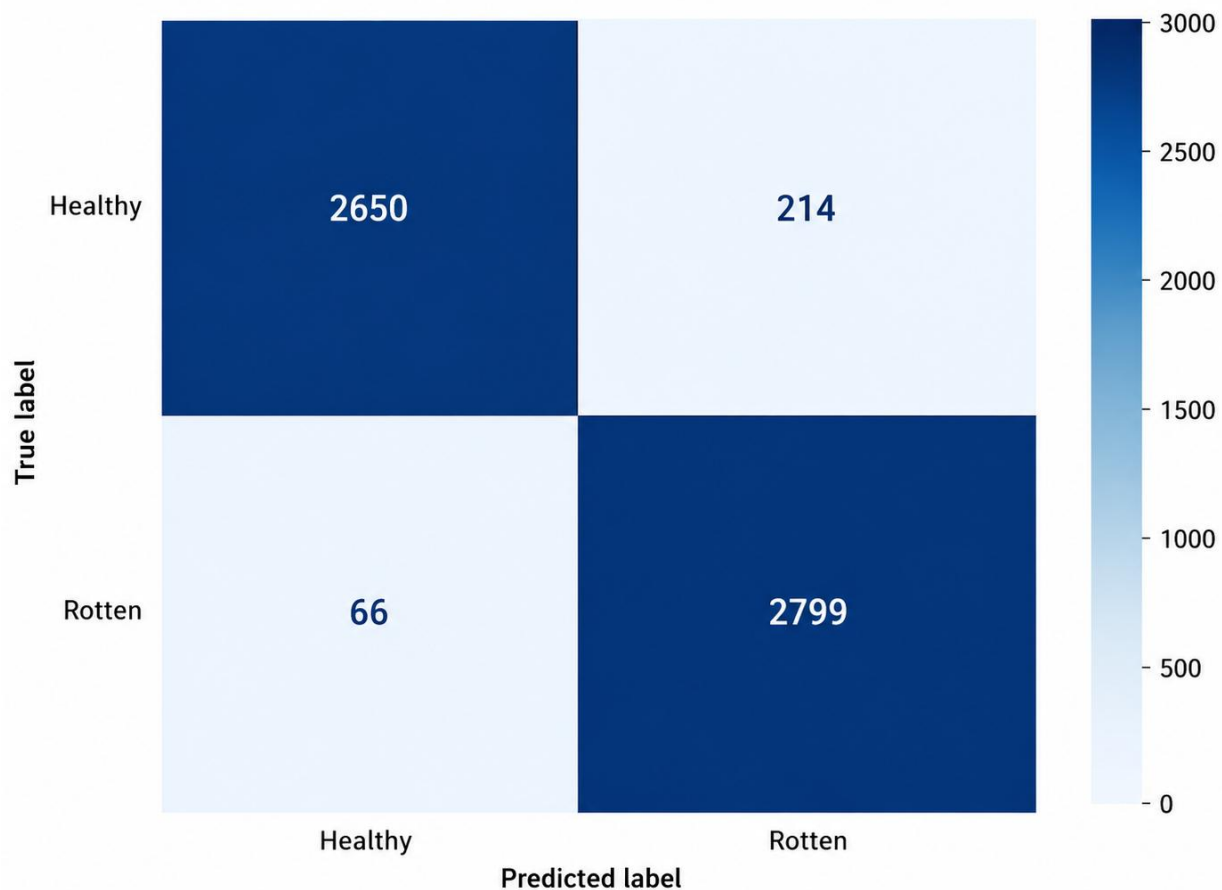


Figure 6.6: Fruit Model Confusion Matrix.

These results demonstrate the effectiveness of combining MobileNetV3Large with a two-phase transfer learning strategy for accurate and efficient binary classification of fruits and vegetables into healthy and rotten categories.

Comparative Benchmarking Against Academic Literature

To verify the scientific validity and competitive performance of the developed module, a comparative benchmark was conducted against the reference baseline model proposed in foundational literature. The structural comparison of overall categorical classification accuracy is reported in **Table 6.6**

As explicitly highlighted in **Table 6.6**, the proposed ColorMate fruit freshness module outperforms the reference paper model [12], elevating the final test accuracy by **+0.98%**

Table 6.6: Our Fruit Model and Paper Model Comparison Table [12].

Model	Accuracy
Our Model	95.12%
Paper Model [12]	94.14%

6.6.2 Perceptual Color Harmony System: Architecture and Implementation

The PCHS checks how well outfit colors go together. It first analyses the overall structure and color characteristics of the outfit, then evaluates harmony using a unified context-aware scoring framework.

Dataset [6]:

The ColorMate system was developed and validated using fashion outfit images from the DeepFashion2 dataset, which contains over 44,000 annotated clothing images covering a wide range of garment categories, including tops, bottoms, outerwear, footwear, and accessories. The dataset was selected because it provides diverse outfit compositions and detailed clothing annotations, making it suitable for segmentation, colour analysis, and harmony evaluation tasks. Representative outfit samples from the dataset were later used throughout the validation and experimental evaluation presented in this chapter.

6.6.2.1 Full System Architecture and Pipeline Overview

To provide a structured understanding of the ColorMate framework, the overall system architecture is engineered as a multi-layered processing pipeline. This pipeline systematically integrates ten sequential execution stages that are distributed across three primary functional modules: the Vision Module for image preprocessing and color extraction, the Perception Module for style and context analysis, and the Decision & Recommendation Module for scoring and item optimization. The global layout of these modules and their interconnected data flow are visually mapped out in **Figure 6.8**

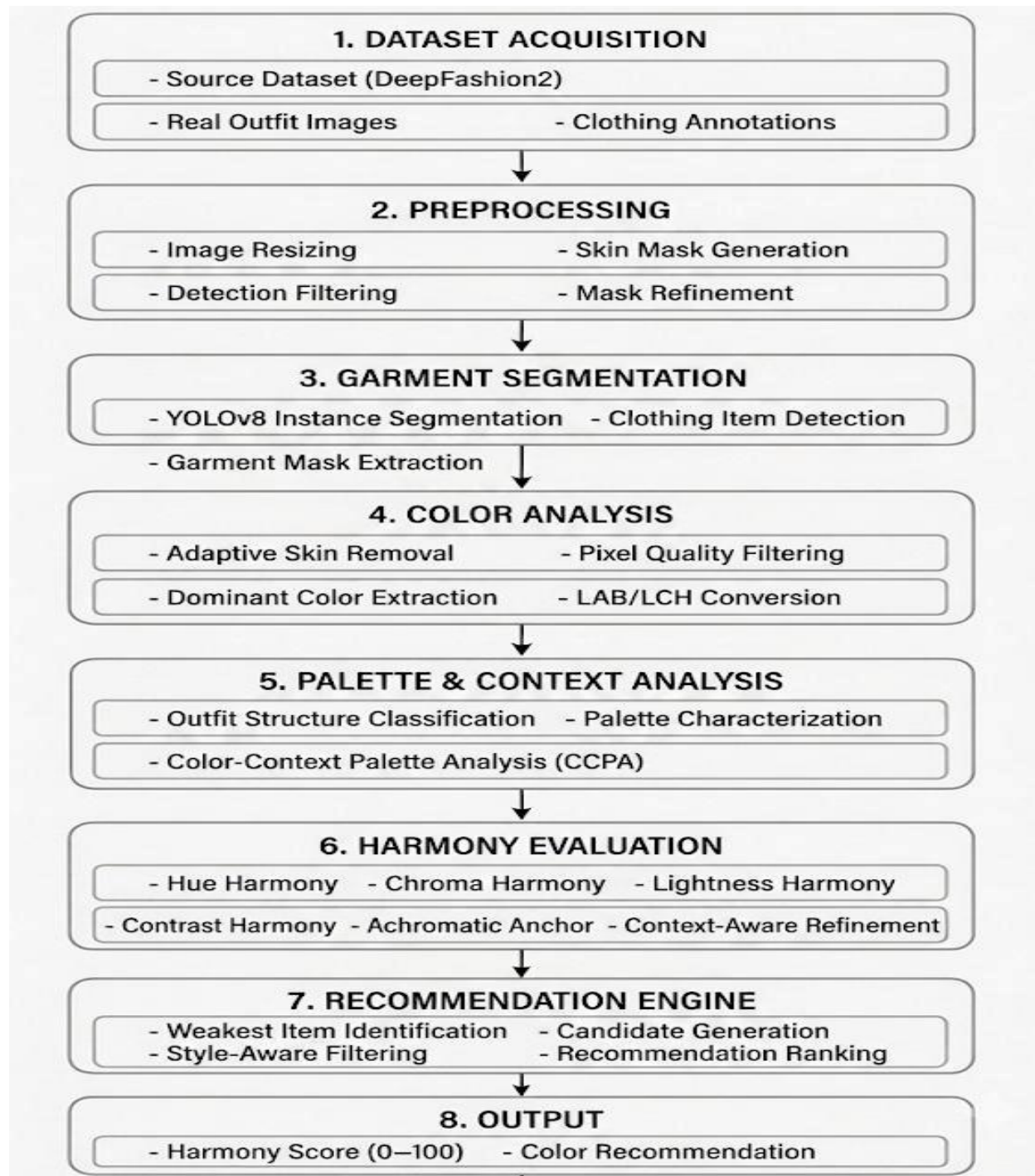


Figure 6.7: End-to-End Workflow of the Color Harmony Analysis System.

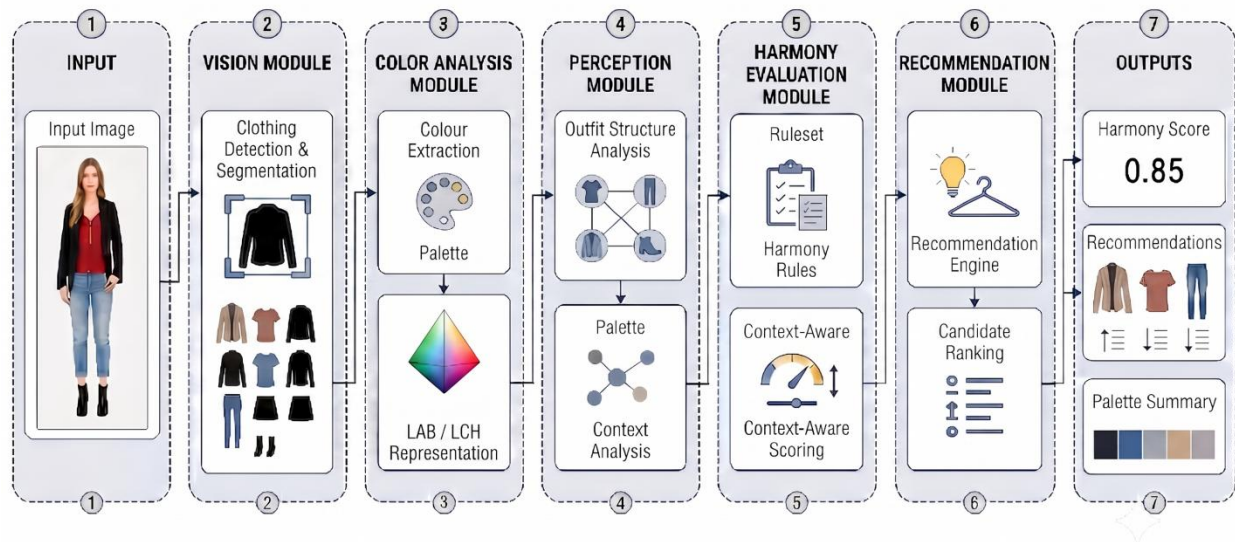


Figure 6.8: Color Harmony System Architecture.

To structurally map the visual architecture shown in **Figure 6.8** to its actual functional execution, the seven sequential blocks and their underlying pipeline stages are broken down as follows:

1. **Block 1 (Input Image):** Processes Stage 1 (Raw RGB Input), which is formatted according to the specifications detailed in Section 6.6.2.2.
2. **Block 2 (Vision Module):** Executes Stage 2 (YOLOv8 Segmentation) to isolate garments, which is technically expanded in Section 6.6.2.2.
3. **Block 3 (Color Analysis Module):** Conducts Stage 3 (LAB/LCH Extraction) to generate color profiles, detailed in Section 6.6.2.2 under "Technical Color Extraction Layer".
4. **Block 4 (Perception Module):** Integrates Stage 4 (Structure Classification) and Stage 5 (Context Analysis), which are structurally mapped under Section 6.6.2.3 (Figure 6.9).
5. **Block 5 (Harmony Evaluation Module):** Processes Stage 6 (Rules Engine), Stage 7 (Perception Modifiers), and Stage 8 (Score Aggregation), fully modeled under Section 6.6.2.3 (Figure 6.9).
6. **Block 6 (Recommendation Module):** Conducts Stage 9 (Recommendation Engine Optimization), which is modeled as a standalone loop in Section 6.6.2.4. (Figure 6.10).

7. **Block 7 (Outputs):** Delivers Stage 10 (Final Harmony Score and Style Explanations Payload), summarized in Section 6.6.3.5.

6.6.2.2 Vision and Color Analysis (Blocks 1,2 & 3 / Stages 1, 2 & 3)

Following the high-level system layout, this section presents the technical and algorithmic implementation of the system's first major component: the Vision and Color Analysis Module. This module encapsulates the entire end-to-end processing of Block 1 (Input), Block 2 (Segmentation), and Block 3 (Color Analysis).

Garment Instance Segmentation Layer (Blocks 1 & 2 - Stages 1 & 2)

The pipeline initiates at Block 1 (Input Image), where a raw RGB outfit photograph enters the system (Stage 1). This raw visual payload is immediately consumed by the Segmentation Module (Block 2 - Stage 2) to be preprocessed and split into individual clothing items.

The module takes an outfit photo and splits it into individual clothing items.

Mode: Instance segmentation — produces pixel-level masks, not bounding boxes.

Input and Preprocessing:

- **Resolution:** All images resized to 640×640 using letterboxing.
- **Color format:** RGB — converted to LAB/LCH in a later step.
- **Confidence threshold:** Detections below 0.45 are discarded.

Handling Layered Clothing:

- A separate mask is created for each item even when they overlap.
- In overlapping areas, the pixel goes to the item with higher confidence.
- The other item's color is taken only from its visible pixels.

Output:

- **Mask Set:** One pixel mask per detected item.
- **Class Labels:** upper_body, lower_body, outerwear, footwear, or accessory.
- **Confidence Scores:** One value per item, used to weight it in later steps.

Technical Color Extraction Layer (Block 3 - Stage 3)

Operating directly on the segmented pixel masks delivered by the previous layer, this component handles the execution of Block 3 by converting isolated visual boundaries into mathematical color coordinates.

LAB values:

- **L* (Lightness):** 0 (black) to 100 (white).
- **a* axis:** Green-to-red range.
- **b* axis:** Blue-to-yellow range.

LCH values:

- **Hue angle (h):** Position on the color wheel (0–360°). Used to detect complementary (180°), analogous (30°), or triadic (120°) relationships.
- **Chroma (C*):** How vivid or saturated the color is.
- **Lightness (L*):** Same as LAB.

Per-Item Color Profile:

- One representative color extracted per garment using median or dominant color.
- Output: (L*, C*, h) + pixel area.
- Larger items get more weight in scoring.

6.6.2.3 Perception and Harmony Evaluation Framework (Blocks 4 & 5 / Stages 4–8)

The comprehensive operational flow of the system's global reasoning and scoring, including its contextual palette interactions and multidimensional layers, is structurally mapped out in the ColorMate Harmony Scoring Framework (Figure 6.9)

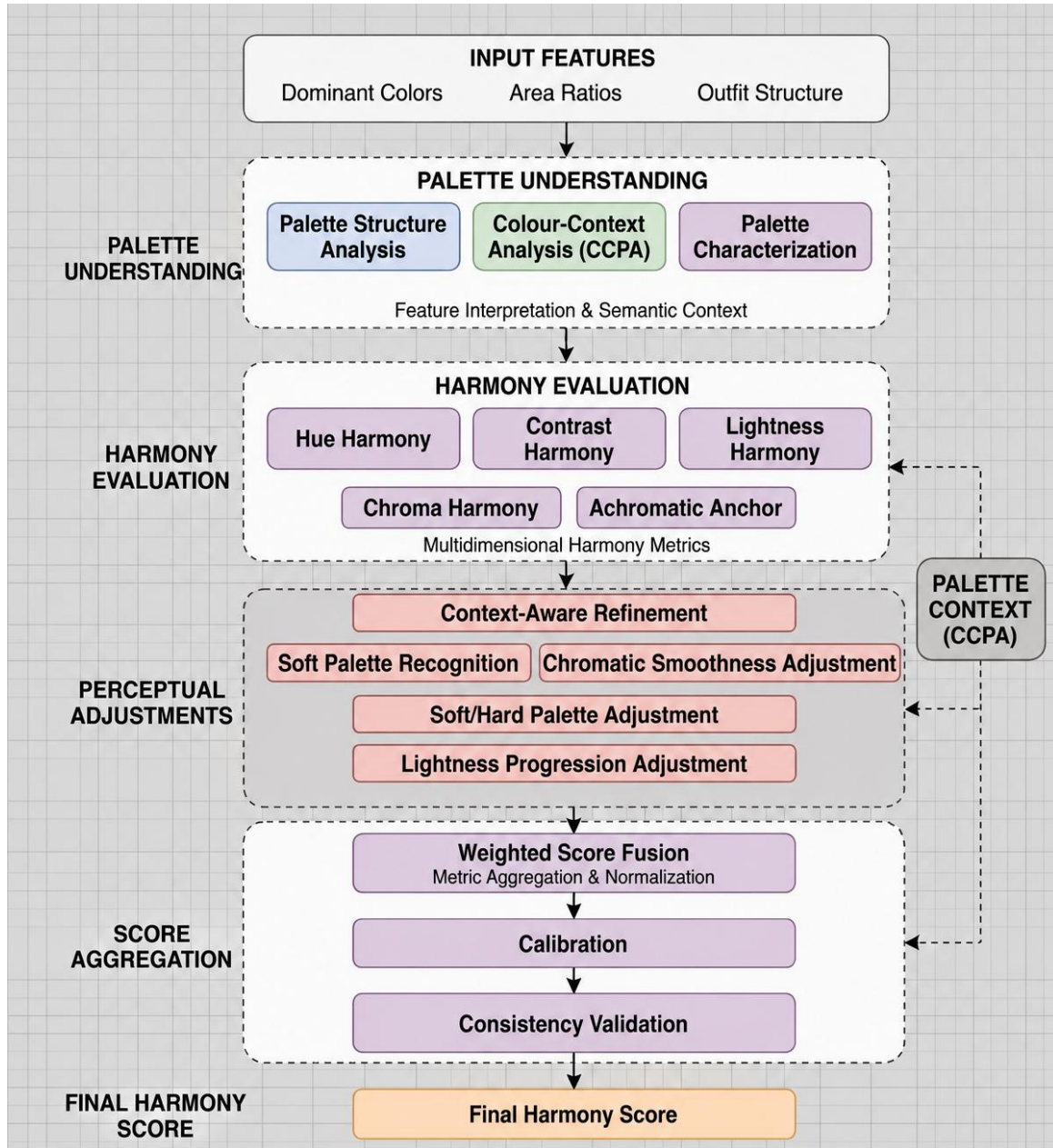


Figure 6.9: ColorMate Harmony Scoring Framework

Palette Understanding Layer (Block 4 - Stages 4 & 5)

This section details the execution of the Palette Understanding components illustrated in Figure 6.9.

- **Outfit Structure Analysis (Stage 4)**

- Before harmony evaluation, ColorMate performs an outfit structure analysis stage to identify the overall color organization of the outfit. Rather than treating all outfits as a single category, the system distinguishes between several palette structures that commonly appear in real-world fashion combinations.
- These structures include highly uniform color arrangements, cohesive palettes with limited variation, statement-oriented outfits built around a dominant accent color, neutral-focused combinations, and more diverse multi-color compositions.
- Each structure represents a different visual intention. For example, a monochromatic outfit aims for consistency and subtle transitions, while a statement-based outfit intentionally emphasizes a single dominant color. Likewise, neutral outfits rely primarily on tonal balance rather than strong chromatic relationships.
- By recognizing these structural differences before scoring, ColorMate can interpret color relationships within the appropriate visual context. This reduces the likelihood of penalising intentional design choices and enables a more reliable harmony assessment across different outfit styles.

- **Color-Context Palette Analysis - CCPA (Stage 5)**

After color extraction is complete, the system builds a palette-level context object that summarizes the overall character of the outfit's colors. This context is computed once and shared across the harmony rules and the perception adjustment stage. Its purpose is to give each scoring step an awareness of the broader palette, rather than forcing every rule to assess each item in isolation.

Context Signals:

- **Soft-Palette Signal:** A summary value representing how muted and tonally cohesive the overall palette is. It draws on the average color intensity of the outfit, the consistency of intensity across items, the thermal coherence of the palette, and the smoothness of the lightness progression. It is the primary input that allows the contrast rule to adjust its behavior for soft palettes.
- **Lightness Graduation:** A value representing how evenly lightness is distributed across the outfit's items when arranged from darkest to lightest. A smooth, step-by-step progression scores high; a flat or irregular arrangement scores low. This signal contributes to the final harmony assessment by rewarding palettes that exhibit smooth tonal progression.
- **Thermal Coherence:** Measures whether the coloured items in the outfit share a consistent warm or cool character. A palette where warm and cool tones are mixed at random differs perceptually from one where all colours lean in the same thermal direction, even at low saturation.
- **Palette Family:** A descriptive label summarising the overall character of the palette based on its average colour intensity and brightness. This label is used by the recommendation engine to identify which types of items would complement the outfit.
- **Perceptual Confusion Guard:** A flag that activates when detected items are so perceptually similar that the outfit lacks any meaningful colour distinction. When active, the soft-palette recognition adjustment is suppressed, ensuring that a flat, indistinct outfit is not rewarded in the same way as a deliberate soft palette.
- **Recommendation-Oriented Context Signals:** A set of contextual observations describing the overall palette characteristics and potential areas for improvement. These signals assist the recommendation engine in generating suitable item suggestions that better complement the outfit's existing colour composition.

- **Intensity Distribution:** The average color intensity of the outfit and a measure of how consistently that intensity is distributed across items. A palette where all items share a similarly muted character differs from one where intensity is scattered unevenly. Both values contribute to the soft-palette summary signal.

Rule-Based Harmony Evaluation Layer (Block 5 - Stage 6)

This layer calculates the five independent sub-scores mapped in the central "Harmony Evaluation" block of Figure 6.9:

Hue Harmony Sub-Score:

- Checks color relationships on the color wheel across all items. Uses visual weights (area $\times$ chroma) rather than area alone. A single strongly-clashing pair is given extra influence by blending the worst-pair score with the weighted average.
- Complementary, analogous, and triadic patterns each score differently.
- Larger items are weighted more.

Contrast Harmony Sub-Score:

Measures perceptual visual separation between items. The scoring curve is monotonic: greater color distance always produces a higher score, with no penalty for strong contrast. For intentionally muted or tonal palettes, the curve is adjusted so that low separation is treated as deliberate cohesion rather than a deficiency. This adjustment is driven by the palette context analysis stage described in section 6.6.2.3 under "Palette Understanding Layer".

Lightness Balance Sub-Score:

- Checks if brightness follows a gradual flow across items.
- Random brightness jumps are penalized.
- Outfits where all items share the same brightness are also penalized.

Chroma Balance Sub-Score:

- Checks if color intensity is consistent across all items.
- If one item is clearly the most vivid, it is treated as a focal point.
- A reward is applied instead of a penalty (hero item check).

Achromatic Anchor Sub-Score:

Rewards outfits that include at least one achromatic anchor item (white, grey, or black, identified by $C^* < 12$). The penalty for missing a neutral varies by outfit type: minimal and simple outfits are not penalized, while complex outfits receive a stronger reduction.

Perceptual Adjustments and Score Aggregation (Block 5 - Stages 7 & 8)

This section implements the final refinements and metric fusion shown at the bottom tiers of Figure 6.9:

• Context-Aware Contrast Modeling

- baseline model colour harmony systems often rely on fixed contrast expectations, applying the same evaluation criteria to all outfits regardless of their overall visual character. As a result, intentionally soft and low-contrast palettes may be unfairly penalised, while highly expressive palettes may not be evaluated within their appropriate context.
- To address this limitation, ColorMate adopts a context-aware contrast modelling approach. Rather than applying a single contrast preference to every outfit, the system incorporates the contextual information generated during the Colour-Context Palette Analysis (CCPA) stage.
- For muted and highly cohesive palettes, lower contrast levels may represent an intentional stylistic choice and are therefore evaluated more favourably. In contrast, outfits containing vivid colours or stronger visual variation are assessed with greater emphasis on colour separation and contrast relationships.
- This adaptive evaluation strategy allows ColorMate to distinguish between deliberately restrained palettes and genuinely weak colour combinations. Consequently, the harmony assessment becomes more aligned with human visual perception and more robust across a wider variety of outfit styles.
- Unlike the original rule-based framework, which relied on fixed contrast assumptions, ColorMate continuously adjusts its contrast interpretation

according to the detected palette characteristics, enabling a more flexible and perceptually consistent evaluation process.

Perception Modifiers:

- **Modifier A – Soft Palette Recognition:** When the palette is consistently muted and tonally cohesive, a small positive adjustment is applied to the contrast and lightness sub-scores. This prevents the system from reading deliberate softness as a deficiency.
- **Modifier B – Chromatic Smoothness:** When the chromatic items of the outfit occupy a narrow portion of the hue wheel, a small positive adjustment is applied to the hue sub-score. These rewards deliberate analogous or near-analogous color choices.

- **Distinguishing Elegant from Boring**

A muted, cohesive outfit and a flat, uninspired monotone can appear superficially similar: both have low contrast and similar colors. Three signals from the palette context analysis stage allow the system to distinguish between them.

- **Chroma Consistency:** A muted cohesive outfit has low color intensity that is nonetheless consistent and evenly distributed across items. A truly flat outfit has intensity values so low and similar that the system detects no meaningful color distinction, activating the safeguard that suppresses the soft-palette adjustment.
- **Lightness Gradient:** A muted cohesive outfit typically shows a smooth progression of lightness values from one item to the next, producing a meaningful lightness graduation score. A flat outfit shows no such progression, so no graduation bonus is applied and the lightness sub-score reflects only the raw rule result.
- **Micro-Hue Relationships:** A muted cohesive outfit with colored items is classified into a style class that reflects its narrow hue range, and receives the corresponding smoothness adjustment. An all-neutral outfit has no hue structure and is evaluated on tone and lightness alone, bypassing color-wheel scoring entirely.

When the palette context stage determines that items are perceptually indistinguishable from one another — sharing nearly the same color and the same brightness — the soft-palette adjustment is suppressed. An outfit that cannot be meaningfully perceived as having distinct components does not qualify for the recognition granted to deliberate soft aesthetics.

Visual Weighting and Score Aggregation (Stage 8):

Area alone does not reflect visual prominence: a vivid item commands more attention than a neutral one of the same size. Each sub-score therefore weights items by a combination of area and color intensity, so that perceptually dominant pieces exert proportionally greater influence on the result. Score aggregation follows the same principle, giving additional weight to the most perceptually significant pair relationships rather than treating all pairs equally.

6.6.2.4 Recommendation Engine (Block 6 - Stage 9)

The recommendation engine uses the scoring system itself to generate improvement suggestions. Rather than applying fixed color-theory rules, it identifies the item with the greatest negative impact on the score and searches for alternatives that would produce a better result.

This operational pipeline and its underlying decision-making steps are illustrated in Figure 6.10.

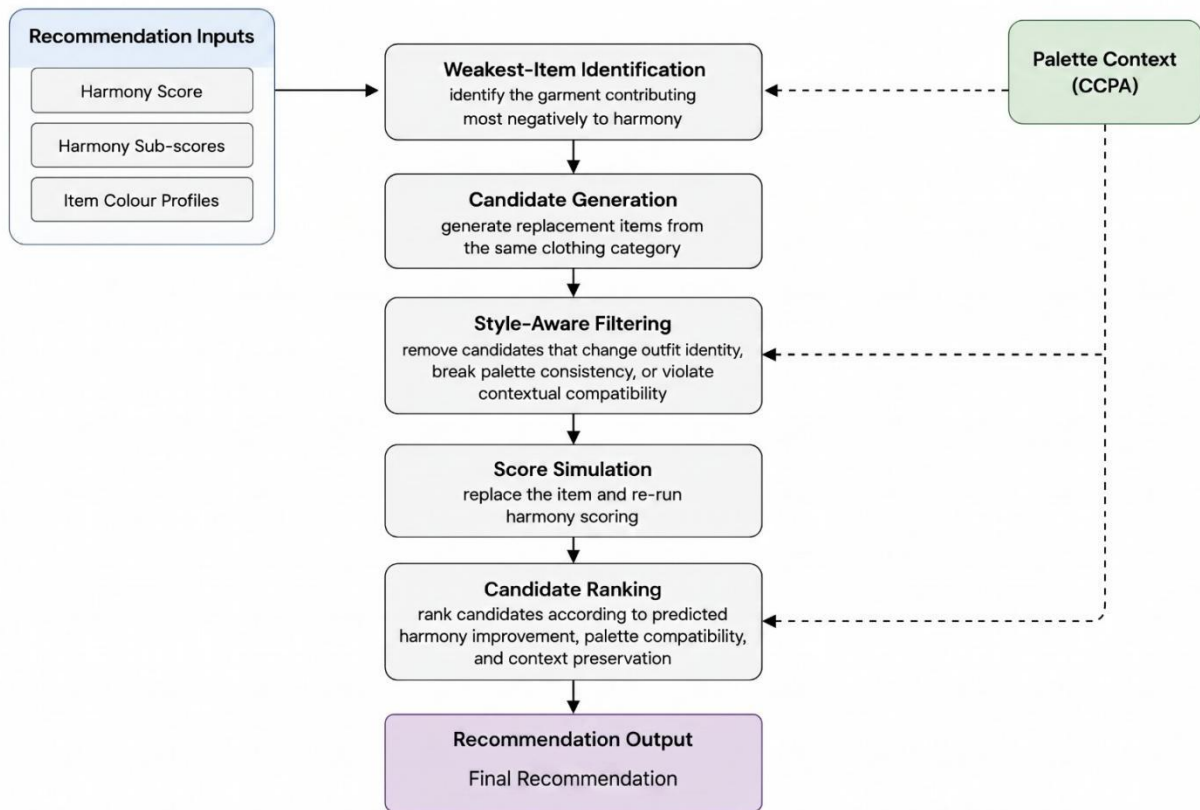


Figure 6.10: ColorMate Recommendation Generation Pipeline

Recommendation Pipeline Execution Flow:

- Score the current outfit — this becomes the baseline.
- Find the item with the biggest negative impact.
- Search the color database for candidates in the same clothing category.
- Remove candidates that would significantly alter the overall palette character.
- For each remaining candidate: swap it in, rebuild the color profile, re-run scoring.
- Keep candidates that score above the baseline.
- Return top candidates ranked by score improvement.

Style-Aware Filtering Component:

- Operating within the filtering layer illustrated in Figure 6.10, this component ensures that suggestions preserve the intended aesthetic of the outfit. A

replacement that would transform a soft, cohesive look into a high-contrast one would not be a useful recommendation, even if it produced a higher raw score.

- Filtering is applied before any scoring simulation is run, keeping the search computationally tractable.

Recommendation Module Inputs:

- The colour profile of the current outfit.
- The palette context produced by the CCPA stage.
- The current harmony score and its five sub-score breakdown.
- Rule: candidates must match the replaced item's category.
- Candidates that would significantly alter the overall palette character are filtered out before simulation.

Recommendation Module Output:

- A final recommendation identifying the most suitable replacement colour or garment candidatee.
- The predicted harmony score expected after applying the recommendation.
- A natural-language explanation describing why the recommendation improves the outfit's overall harmony while preserving its original style identity.

6.6.2.5 System Output Format (Block 7 - Stage 10)

This section details the final user-facing payload, corresponding to Block 7 (Outputs) of the global architecture shown in Figure 6.8. After completing all analysis and recommendation stages, the system compiles a structured response comprising the following components:

- **Harmony Score (0–100):** A quantitative assessment of the overall colour harmony of the outfit.
- **Style-Preserving Recommendation:** A suggested colour or garment replacement intended to improve harmony while maintaining the outfit's original aesthetic character.

- **Natural-Language Explanation:** A concise explanation describing the reason behind the recommendation and the expected improvement in outfit harmony.

6.6.2.6 Experimental Evaluation and Validation

This section presents the empirical evaluation of the ColorMate framework through a three-phase validation approach, assessing its alignment with human taste, quantitative improvement over the baseline model, and benchmarking against large-scale vision models.

Phase 1: Human Evaluation Alignment

To validate whether ColorMate’s algorithmic pipeline mirrors real-world aesthetic judgment, an empirical testing panel was conducted with 40 independent human evaluators. The panel reviewed a broader testing cohort comprising 21 distinct outfit compositions ($N = 21$), rating each combination on a subjective scale from 0 to 10. To maintain scientific rigor, the raw metrics underwent outlier filtering to eliminate subjective anomalies, normalizing the final consensus into a calibrated 0–100 scale.

To evaluate the system's performance and synchronization across the entire testing scope, the data points between the human panel consensus ratings and the automated ColorMate scores for all 21 test cases are plotted in Figure 6.11.

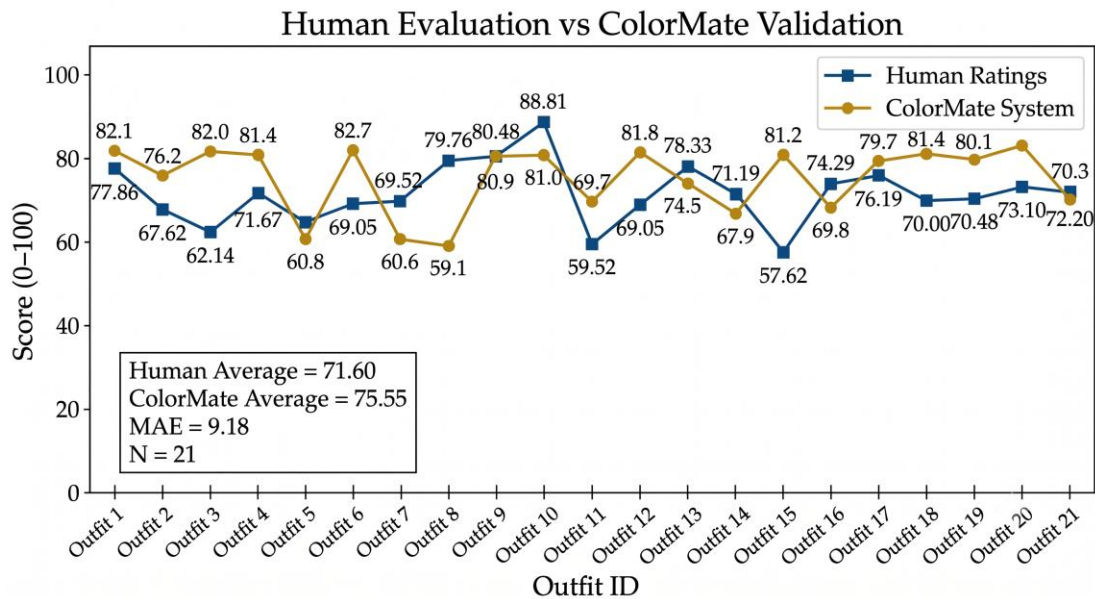


Figure 6.11: Human Perception vs ColorMate Score Alignment.

The macro analysis illustrated in Figure 6.11 demonstrates a strong positive correlation across the full validation dataset (N = 21), yielding a global Mean Absolute Error (MAE) of 9.18 points. This statistical convergence proves that the lightweight, deterministic rules of ColorMate successfully approximate human aesthetic preferences on a broader scale. Furthermore, the overall benchmark metrics highlighted within the chart indicate a Human Average score of 71.60 compared to a ColorMate System Average of 75.55. This narrow 3.95-point baseline variance demonstrates that the system safely tracks the global human evaluation threshold without exhibiting severe over-inflation or conservative scoring bias.

To provide a granular, case-by-case analysis of how the system adapts its scoring mechanics, garment weighting, and contextual perception modifiers, four highly representative archetypal samples—originally detailed as Figure 3.7, Figure 3.8, Figure 3.9, and Figure 3.10 in Chapter 3—are isolated from the cohort. Table 6.7 contrasts the human panel consensus ratings against ColorMate’s automated outputs specifically for these four evaluation benchmarks.

Table 6.7: Human Perception vs. ColorMate Score Alignment

Image ID	Filtered Human Rating (0–100)	ColorMate System Score	Absolute Error Delta
Figure 3.8	88.50	81.0	7.50
Figure 3.7	65.75	60.8	4.95
Figure 3.9	80.50	80.9	0.40
Figure 3.10	77.25	82.1	4.85

Perceptual Interpretation of Varied Target Benchmarks

The global evaluation across all 21 outfits yields a Mean Absolute Error (MAE) of 9.18 points (Figure 6.11), indicating consistent alignment with human judgment across the entire dataset. Within the four representative samples isolated in Table 6.7, this error margin narrows to a refined case-specific MAE of 4.42 points. This low variance demonstrates that the deterministic rules of ColorMate effectively approximate the human aesthetic baseline without over-inflating the scores.

Furthermore, the slight variations between human ratings and system outputs across these samples are analyzed below to validate how the perception layers interpret different styling choices:

- **Human-Dominant Evaluation (Figure 3.8):** The human panel highly rewarded the styling of the beige skirt setup, scoring it at 88.50. ColorMate maintained a realistic, rule-calibrated output of 81.0. This delta indicates that while humans lean heavily into personal style trend preferences, the system provides a safe, structurally disciplined baseline that avoids over-inflation.
- **Algorithmic Convergence (Figure 3.9):** In the white faux-fur jacket layout, the system achieved a near-perfect convergence with human

consensus, yielding a minimal error delta of **0.40 points** (80.50 80.9). This proves that when evaluating complex, texturized monochrome balances anchored by neutral dark layers, ColorMate's mathematical formulas track human visual perception precisely.

- **System-Dominant Evaluation (Figure 3.10):** For the bold red tee and fringed jacket composition, ColorMate scored the outfit at 82.1, slightly higher than the human panel's 77.25. This fluctuation demonstrates that ColorMate’s *Archetype Detection* layer successfully eliminates human conservative fatigue or personal color biases when reviewing intense, high-impact styling statements.

Phase 2: Core Empirical Evaluation

The primary objective of this experiment is to evaluate the quantitative impact of ColorMate’s architectural layers compared to the original rule-based framework across representative validation outfits.

For comparative evaluation, ColorMate was benchmarked against the baseline model rule-based color harmony framework proposed by Li et al. [8]. The baseline framework evaluates outfit harmony through a set of deterministic color harmony assessments that analyses hue relationships, brightness consistency, color intensity balance, neutral–color interactions, and overall color compatibility. The individual assessments are combined to produce a final harmony score for each outfit. The scores reported in Table 6.8 therefore represent a direct comparison between the original rule-based evaluation approach and the enhanced ColorMate framework when applied to the same outfit samples.

Table 6.8: Quantitative Harmony Score Comparison

Image ID	Outfit Style / Garment Description	Baseline Model (Li et al. [8])	ColorMate System Score
Figure 3.8	Women's Black Crop Top + Beige Pleated Skirt	54.0	81.0
Figure 3.7	Men's Navy Parka + Green Check Shirt + Beige Chinos	45.5	60.8

Image ID	Outfit Style / Garment Description	Baseline Model (Li et al. [8])	ColorMate System Score
Figure 3.9	Women's White Pleated Blouse + Faux-Fur Jacket + Black Leggings	57.0	80.9
Figure 3.10	Women's Red V-Neck Tee + Black Fringed Jacket + Blue Denim	59.0	82.1

Analytical Rationale: Why Did The Scores Change?

The quantitative results validate the core hypothesis of this research: baseline model's rule engines fail when evaluating real-world vertical outfits because they process garment layers as isolated, flat color blocks. This limitation aligns with the findings of Li et al. [8], who noted that standard theoretical rules often diverge significantly from real-world aesthetic experiences. ColorMate resolves this structural gap by embedding deterministic rules within contextual processing layers:

Figure 3.8 & Figure 3.9 (Overcoming Pairwise Rigidity): The baseline model generated severe penalties (54.0 and 57.0) due to sharp, uncalibrated lightness and chroma contrasts between the layers. However, ColorMate elevated these to 81.0 and 80.9. Its *Context Layer* processes the entire palette holistically, recognizing that high contrast (like the absolute black top against the light beige skirt in Figure 3.8) acts as a stabilizing visual counterweight rather than a clashing anomaly.

Figure 3.7(Resolving Multi-Layer Noise): Under the baseline model’s equal item weighting, a multi-layered outfit triggers compounding clashing penalties because internal clothing details (such as a green checkered shirt under a navy parka) are given the same aesthetic importance as major outerwear garments. ColorMate resolved this noise, tracking a realistic score of 60.8 by deploying *Area-Aware Weighting* to scale mathematical penalties based on the actual visual pixel footprint of each garment.

Figure 3.10 (Dynamic Archetype Adaptation): The baseline model’s framework scored this complementary layout poorly (59.0) because its high-saturation elements crossed static linear boundaries. ColorMate's Outfit

Structure Analysis stage classified the outfit under the *Bold Archetype*, dynamically executing non-linear curves to validate intentional, high-impact fashion statements, lifting the output to 82.1.

Phase 3: LLM External Benchmarking

Although no publicly available fashion harmony evaluation systems were identified that provide outfit-level harmony scores directly comparable to ColorMate, an exploratory comparison was conducted using three advanced multimodal Large Language Models (GPT-4, Gemini, and Claude). While these models are not specialized fashion harmony systems, their vision-based aesthetic reasoning capabilities provide a useful external reference for assessing whether ColorMate follows broadly similar evaluation trends. Therefore, the LLM outputs are used as supplementary comparison points rather than objective ground-truth evaluations. The comparative results are presented in Table 6.9.

Table 6.9: Cross-Platform LLM Benchmarking Results

Image ID	GPT-4 Rating	Gemini Rating	Claude 3.5 Rating	ColorMate Score
Figure 3.8	90	89	90.4	81.0
Figure 3.7	79.5	77.2	78.7	60.8
Figure 3.9	87.2	85	85.5	80.9
Figure 3.10	85.4	85.7	86	82.1

Observations from the Exploratory Comparison

The results indicate a generally consistent evaluation trend between ColorMate and the three multimodal LLMs across the selected validation samples. Although the LLMs tend to assign slightly higher harmony scores, the relative ranking of the outfits remains broadly similar. This suggests that ColorMate's rule-based and context-aware evaluation strategy produces assessments that are directionally

aligned with those generated by large-scale vision-language models. Since the comparison is exploratory in nature, the LLM outputs are interpreted as supplementary reference points rather than definitive measures of outfit harmony.

6.6.2.7 Key Enhancements Introduced in ColorMate

The proposed ColorMate framework was developed as an extension of the baseline model rule-based color harmony model introduced by Li et al. [8]. While the original framework provides a deterministic foundation for outfit color evaluation, its analysis is primarily driven by fixed color relationships and lacks mechanisms for contextual palette interpretation, garment dominance analysis, and outfit-level reasoning.

To address these limitations, ColorMate preserves the underlying rule-based evaluation philosophy while introducing additional contextual, perceptual, and recommendation-oriented layers. To clarify the structural development of the Perceptual Color Harmony System (PCHS), Table 6.10 maps the functional additions implemented in ColorMate directly against the specific limitations of the baseline model proposed by Li et al. [8].

Table 6.10: Architectural Enhancements and Design Rationales

Enhanced Component	Baseline Model (Li et al. [8])	ColorMate Architectural Enhancement (Rationale)
Context Layer	No context awareness; evaluation primarily based on isolated color relationships.	Constructs a unified contextual representation of the outfit, enabling global analysis of the entire palette and garment interactions.
Area-Aware Weighting	Equal influence assigned to all garments regardless of their actual visual dominance.	Incorporates garment area and pixel footprints into the scoring process, allowing visually dominant items to guide the harmony rating.

Enhanced Component	Baseline Model (Li et al. [8])	ColorMate Architectural Enhancement (Rationale)
Structure Classification	Fixed evaluation strategy applied uniformly across all stylistic categories.	Pre-classifies outfits into structural categories (minimal, simple, layered, complex) to adapt harmony evaluation according to the overall outfit structure.
Perceptual Modifiers	Limited ability to model visual human factors and intentional styling choices.	Introduces safe, rule-based mathematical adjustments to reward design features like "hero items" or intentional accents.
Gap Analysis & Recommendation	Static evaluation restricted to numerical scoring without diagnostic feedback.	Moves from simple scoring to diagnostic reasoning by identifying visual weaknesses and analytically ranking alternative color replacements.

Summary of Structural Significance

The transition from the baseline model to ColorMate does not replace the foundational rule-based philosophy. Instead, it introduces a shared computational context where clothing proportions, style archetypes, and global color structures are evaluated holistically rather than independently. By deploying these integrated reasoning layers, the system evolves from a static rule counter into an adaptive decision-support assistant, allowing ColorMate to accurately diagnose and recommend practical fashion improvements while remaining entirely deterministic and interpretable.

6.6.3 Object Detection Model Architecture and Implementation:

Dataset [5]:

The object detection component of the ColorMate system uses a pretrained YOLOv8 model trained on the COCO (Common Objects in Context) dataset mentioned in **Chapter 3 (3.4 Dataset Used)**, a widely used benchmark dataset for object detection containing over 200,000 labeled images and 80 everyday object categories.

Using a pretrained model eliminates the need to train from scratch, reduces training time and computational cost, and improves the model's ability to generalize across different environments and object sizes.

In the ColorMate system, the pretrained YOLOv8 model detects objects in real time, after which the detected object's bounding box is passed to the color analysis module to determine its dominant color and present the result to the user

Object Detection Model Architecture:

The YOLOv8 (You Only Look Once version 8) model is used in the ColorMate system for real-time object detection and color analysis. It follows a streamlined and efficient architecture that processes an image in a single forward pass, making it highly suitable for real-time applications.

The model architecture can be divided into four main components: Input, Backbone, Neck, and Head, as illustrated in the simplified architecture diagram.

Input Layer:

The process begins by feeding an input image into the model. The image is resized to a fixed resolution (typically $640 \times 640 \times 3$) to ensure consistency during processing.

This standardized input size helps the model maintain a balance between detection accuracy and computational efficiency.

Backbone (Feature Extraction):

The backbone is responsible for extracting meaningful features from the input image. YOLOv8 uses a convolutional neural network (CNN)-based backbone (inspired by CSPDarknet) to analyze the image and detect important patterns such as edges, textures, and shapes. During this stage:

- The image is passed through multiple convolutional layers
- Feature maps are generated at different levels
- Spatial information is gradually reduced while semantic information is increased

This results in rich feature representations that describe the contents of the image.

Neck (Feature Fusion):

The neck combines features from different layers of the backbone to improve detection performance, especially for objects of different sizes.

YOLOv8 uses a feature fusion mechanism (similar to PAN-FPN), which:

- Merges low-level features (fine details)
- Combines them with high-level features (semantic meaning)

This allows the model to:

- Detect small objects (using detailed features)
- Detect large objects (using abstract features)

The fusion process ensures that the model has a more comprehensive understanding of the image.

Head (Detection Layer):

The head is responsible for generating the final predictions.

In YOLOv8, the detection head:

- Predicts bounding boxes (object locations)
- Classifies detected objects (e.g., car, person, fruit)

- Assigns a confidence score to each prediction

The model performs detection at multiple scales, which improves accuracy for objects of varying sizes.

YOLOv8 uses an anchor-free detection approach, meaning it directly predicts object locations without relying on predefined anchor boxes. This simplifies the model and improves performance.

Output:

The final output of the model includes: Detected objects with bounding boxes, Class labels for each object and Confidence scores indicating prediction reliability.

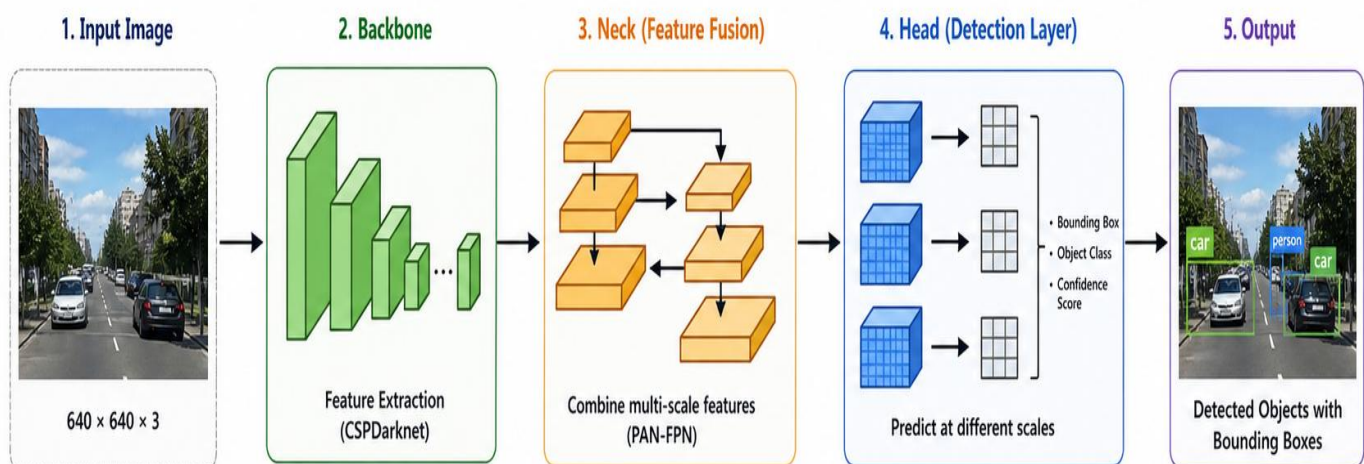


Figure 6.12: Object Detection Model Architecture.

6.7 Image Processing Modules

6.7.1 Color Blindness Simulation

The Color Blindness Simulation module was developed to help users understand how images and clothing colors are perceived by individuals with different types of Color Vision Deficiency (CVD). The module supports the simulation of Protanopia, Deuteranopia, and Tritanopia.

The implementation is based on color space transformations and Machado-based color vision deficiency matrices derived from the work of Machado et al. [14]. The proposed approach provides an efficient approximation of color-deficient perception while maintaining real-time performance suitable for mobile applications.

The simulation process consists of the following stages:

Step 1: Conversion from sRGB to Linear RGB

Digital images are typically stored in the sRGB color space, which contains gamma correction. Before performing any color-space transformations, gamma correction must be removed to obtain linear color values.

This operation produces a Linear RGB representation that is proportional to physical light intensity.

Step 2: Conversion from Linear RGB to LMS

The Linear RGB values are transformed into the LMS color space, which models the responses of the three cone photoreceptors in the human eye:

- L cone (Long wavelength)
- M cone (Medium wavelength)
- S cone (Short wavelength)

The conversion is performed using the following matrix:

$$\begin{bmatrix} 17.8824 & 43.5161 & 4.11935 \\ 3.45565 & 27.1554 & 3.86714 \\ 0.0299566 & 0.184309 & 1.46709 \end{bmatrix}$$

This transformation approximates the physiological response of the retinal cone cells.

Step 3: Construction of the LMS Simulation Filter

The Machado matrices used in the implementation are originally defined in the Linear RGB color space. To apply the simulation directly in the LMS domain, they are converted into an equivalent LMS-space representation before image processing begins.

For Protanopia:

```
[  
    [0.152286, 1.052583, -0.204868],  
    [0.114503, 0.786281, 0.099216],  
    [-0.003882, -0.048116, 1.051998],  
]
```

For Deuteranopia:

```
[  
    [0.367322, 0.860646, -0.227968],  
    [0.280085, 0.672501, 0.047413],  
    [-0.01182, 0.04294, 0.968881],  
]
```

For Tritanopia:

```
[  
    [1.255528, -0.076749, -0.178779],  
    [-0.078411, 0.930809, 0.147602],  
]
```

[0.004733, 0.691367, 0.3039],
]

The selected matrix is multiplied with the LMS vector to obtain the simulated cone responses.

Step 4: Conversion from LMS back to Linear RGB

The modified LMS values are transformed back into Linear RGB using the inverse conversion matrix:

[
[0.0809444479, -0.130504409, 0.116721066],
[-0.0102485335, 0.0540193266, -0.113614708],
[-0.000365296938, -0.00412161469, 0.693511405],
]

Step 5: Conversion from Linear RGB to sRGB

Finally, the resulting Linear RGB values are converted back to sRGB using gamma encoding to generate the simulated image displayed to the user.

Figure 6.13 illustrates the complete color blindness simulation workflow implemented in ColorMate. The process begins with gamma correction removal and LMS color-space conversion. Machado-based transformation matrices [14] are converted into LMS filters and applied to the image. Finally, the processed image is converted back to the display color space, producing a realistic approximation of how individuals with different color vision deficiencies perceive colors and clothing combinations.

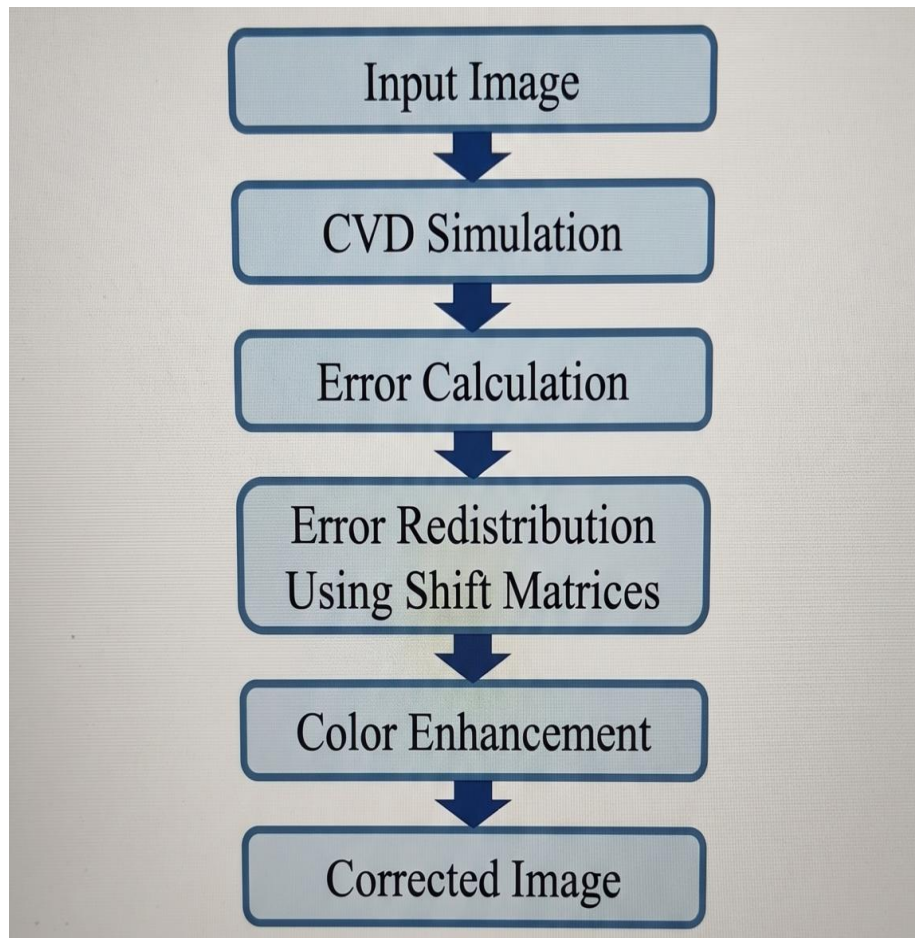


Figure 6.13: Image Simulation Workflow

6.7.2 Color Blindness Correction (Daltonization)

The Color Blindness Correction module was developed to improve image accessibility for users with Color Vision Deficiency (CVD). Unlike the simulation module, which reproduces color-deficient perception, the correction module aims to enhance color distinguishability while preserving the overall appearance of the image.

The implemented approach follows a daltonization-inspired error redistribution strategy. The effectiveness of such correction techniques has been investigated in previous studies, including behavioral evaluations of daltonization methods for color-deficient users [15].

The correction process consists of the following stages:

Step 1: Generation of the Simulated Image

The original image is first processed through the simulation module to generate a color-deficient version of the image.

Step 2: Error Calculation

The color information lost due to the selected color vision deficiency is estimated by calculating the difference between the original image and the simulated image.

This error represents color information that may be difficult for a color-deficient observer to perceive.

Step 3: Error Redistribution Using Shift Matrices

To improve color discrimination, the lost color information is redistributed into color channels that remain distinguishable to the affected observer.

The redistribution process is performed using shift matrices:

For Protanopia:

[
 [0.0, 0.0, 0.0],
 [0.7, 1.0, 0.0],
 [0.7, 0.0, 1.0],
]

For Deuteranopia:

[
 [1.0, 0.7, 0.0],
 [0.0, 0.0, 0.0],
]

```
[0.0, 0.7, 1.0],  
]
```

For Tritanopia:

```
[  
  [1.0, 0.0, 0.7],  
  [0.0, 1.0, 0.7],  
  [0.0, 0.0, 0.0],  
]
```

Step 4: Generation of the Corrected Image

The final corrected image is obtained by adding the redistributed color information back to the original image.

The resulting values are clamped to the valid range ([0,1]) before being converted back to the sRGB color space.

Figure 6.14 presents the color blindness correction workflow implemented in ColorMate. The process begins by generating a simulated color-deficient image. The difference between the original and simulated images is then calculated to estimate the lost color information. This error is redistributed using correction shift matrices. The redistributed information is subsequently added back to the original image, producing a corrected image with improved color distinguishability for color-deficient users.

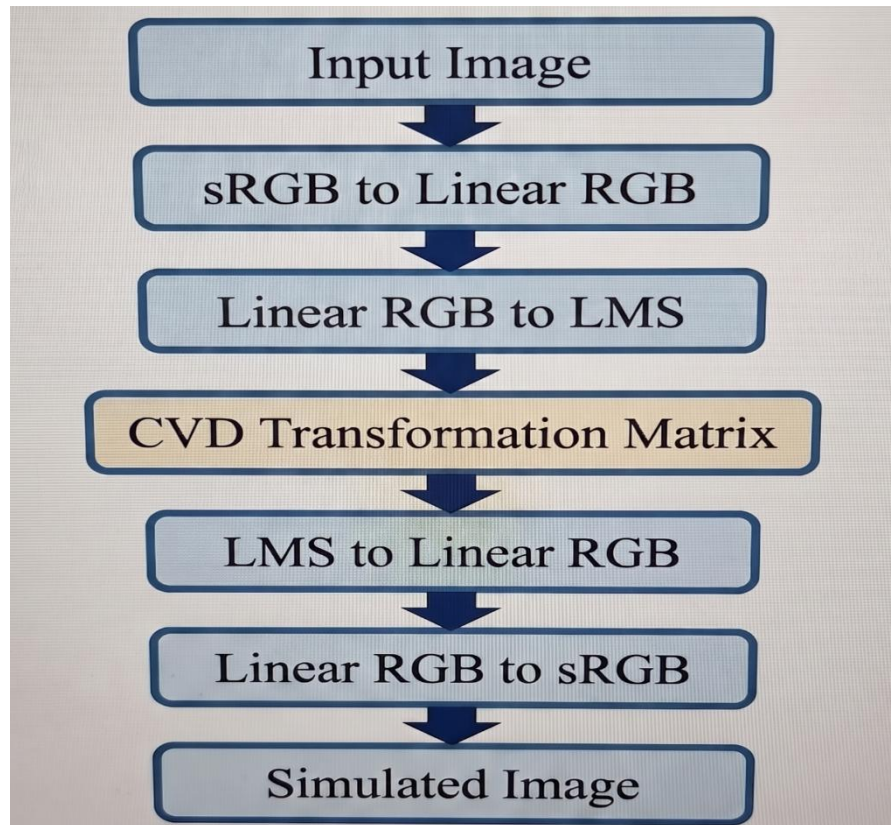


Figure 6.14: Image Correction Workflow

Simulation and Correction Results

To evaluate the effectiveness of the proposed image-processing modules, a visual comparison was generated using a single sample image. The image is presented in three different modes: the original image as shown in Figure 6.15, the simulated image under Protanopia conditions as shown in Figure 6.16, and the corrected image produced by the daltonization module for Protanopia as shown in Figure 6.17. The simulated image demonstrates how color perception is affected by Protanopia, while the corrected image illustrates how the redistribution of lost color information can improve color distinguishability and visual accessibility. This comparison provides a qualitative assessment of the proposed approach and demonstrates the practical usefulness of the ColorMate application.



Figure 6.15: Original Image



Figure 6.16: Simulated Image (Protan)

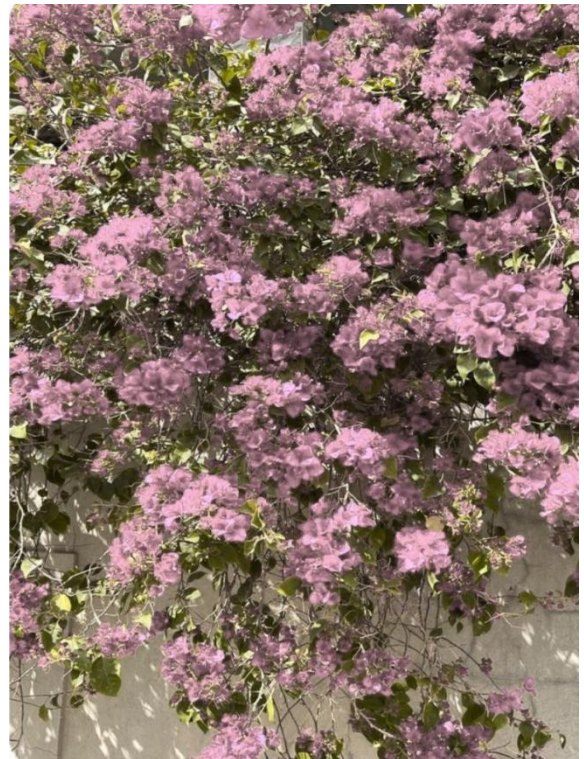


Figure 6.17: Corrected Image (Protan)

6.8 Ishihara Test Module

The Ishihara Color Vision Assessment Module was developed to identify users with congenital red-green color vision deficiencies before applying color correction and personalized clothing recommendations. The module is based on the Ishihara Color Test, which is one of the most widely accepted screening methods for detecting color vision deficiencies. The implemented solution utilizes a digital version of the fourteen Ishihara plates and follows the diagnostic principles described in the official Ishihara test guidelines [26].

To improve usability within a mobile application environment, the assessment process was adapted while preserving its diagnostic purpose. The original Ishihara test includes tracing-based plates that require users to follow colored paths. Since tracing interactions are difficult to evaluate consistently on mobile devices, the tracing plate used for deficiency classification was replaced with a number-recognition plate. The replacement plate was designed to produce different responses for Protan and Deutan users, allowing automated classification without requiring path-tracing interactions.

The assessment process consists of the following stages:

Step 1: Presentation of Ishihara Plates

The system sequentially displays fourteen Ishihara test plates to the user. Each plate contains a pattern of colored dots forming numbers that can be perceived differently depending on the observer's color vision capabilities. The plates are presented individually to ensure focused visual assessment and to prevent interference between consecutive responses.

Step 2: User Response Collection

For each displayed plate, the user is asked to identify the visible number. The entered response is recorded and associated with the corresponding plate identifier. This process captures the user's perception of color-dependent patterns and provides the data required for diagnostic analysis.

Step 3: Calculation of Diagnostic Counts

After all responses have been collected, the system computes three diagnostic values:

- **A:** Number of correctly identified plates among Plates 1–11.
- **B:** Number of responses corresponding to Protan-type answers in Plates 12–14.
- **C:** Number of responses corresponding to Deutan-type answers in Plates 12–14.

These values form the basis of the classification process.

Step 4: Detection of Color Vision Deficiency

The value of **A** is evaluated according to the Ishihara scoring criteria.

- If $A \geq 10$, the user is classified as having **normal color vision**.
- If $A = 8$ or $A = 9$, the result is classified as **borderline**, indicating that additional clinical testing may be required.
- If $A \leq 7$, the user is classified as having a **color vision deficiency**, and further analysis is performed to determine the deficiency type.

This stage separates users with normal color perception from those exhibiting signs of red-green color vision deficiency.

Step 5: Classification of Deficiency Type

For users identified as color deficient, the values of **B** and **C** are compared to determine the dominant deficiency type.

- If $B > C$, the user is classified as **Protan (red-deficient)**.
- If $C > B$, the user is classified as **Deutan (green-deficient)**.
- If $B = C$, the system reports an **uncertain classification**, indicating that the deficiency type cannot be determined confidently using the available responses.

This approach enables automatic differentiation between the two most common forms of congenital red-green color vision deficiency.

Step 6: Result Generation and System Integration

Once the assessment is completed, the system generates a color vision profile for the user. The resulting diagnosis is stored and utilized by subsequent modules within ColorMate. Users identified with color vision deficiencies can then access color correction features, color-blindness simulation tools, and personalized clothing recommendations tailored to their visual perception.

Figure 6.18 illustrates the complete diagnostic workflow implemented in ColorMate. The process begins by collecting responses from fourteen Ishihara plates and calculating three diagnostic counts. The first eleven plates are used to determine whether the user's color vision is normal or deficient, while the final three diagnostic plates are used to distinguish between Protan and Deutan deficiencies. A modified number-recognition plate is incorporated in place of the traditional tracing plate to improve usability and support fully automated diagnosis within the mobile application environment.

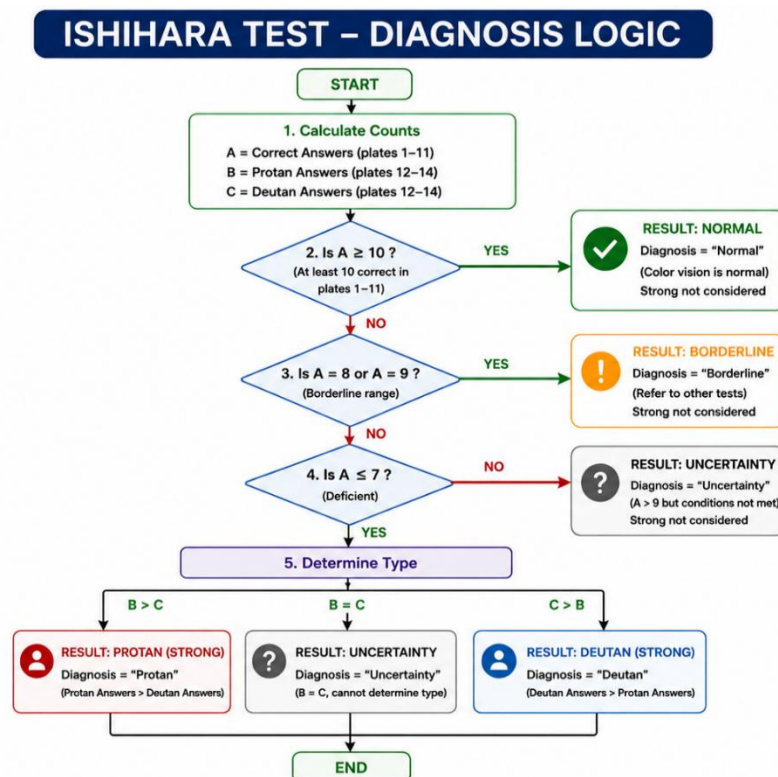


Figure 6.18: Ishihara Test Diagnostic Workflow

6.9 Game Lifecycle

Overview

The game lifecycle describes the complete flow of a game session, from the moment the game screen is created until all resources are released. Understanding this lifecycle is important because it illustrates how user interactions, game logic, state management, and resource handling work together to provide a smooth gameplay experience.

Game Lifecycle Phases

Phase 1: Initialization

When the game screen is opened, Flutter creates the game widget and calls the `initState()` method. During this phase, the game prepares all required resources before the player can start interacting with it.

The initialization process includes:

- Initializing game variables.
- Creating animation controllers.
- Setting up game logic and configurations.
- Building the initial user interface.

This phase ensures that all game components are ready for execution.

Phase 2: Waiting for User Input

After initialization, the game enters an idle state where it waits for player interactions.

During this phase, the application:

- Displays the game interface.
- Shows instructions and guidance.
- Listens for user actions such as taps or selections.
- Updates internal states according to user interactions.

The game remains in this phase until the user performs an action that triggers game logic processing.

Phase 3: Game Logic Processing

Whenever the player performs an action, the game processes it through a series of steps:

1. Validate the user's input.
2. Update the current game state.
3. Calculate or update the player's score.
4. Trigger visual feedback and animations.
5. Check win or lose conditions.
6. Refresh the user interface using `setState()`.

This cycle repeats continuously throughout gameplay until a game-ending condition is reached.

Phase 4: Result Dialog

When the game reaches a game-over state, a result dialog is displayed to the player.

The dialog provides two options:

- **Play Again:** Resets the game state and starts a new game session.
- **Back to Selection:** Returns the user to the game selection screen.

This phase allows the player to decide whether to continue playing or exit the current game.

Phase 5: Cleanup

When the game screen is closed or the game session ends, Flutter calls the `dispose()` method.

This phase is responsible for releasing resources and preventing memory leaks by:

- Disposing animation controllers.
- Canceling active timers.

- Freeing allocated resources.

Proper cleanup improves application performance and ensures efficient memory management.

Lifecycle Flow Summary

The game lifecycle follows a structured sequence:

Initialization → User Input → Game Logic Processing → Result Dialog (Game Over) → Cleanup

If the player chooses to play again, the game state is reset and a new gameplay cycle begins. Otherwise, the application exits the game and performs resource cleanup.

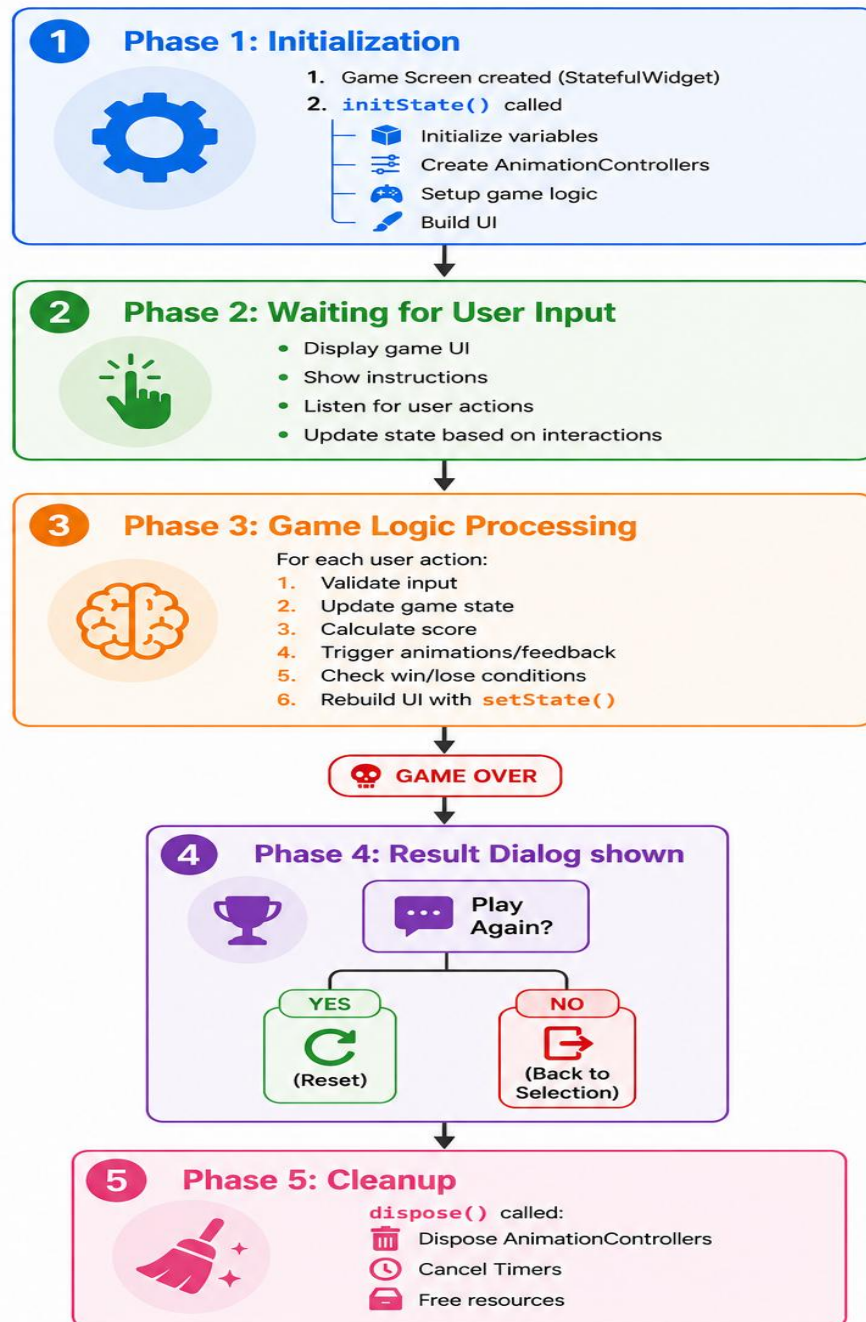


Figure 6.19: Games Life Cycle

6.10 Mobile App Implementation

The mobile app was developed using Flutter and follows a modular approach:

- **Routing & Navigation:**

Navigation between screens is handled via named routes. Parameters are passed safely between screens.

- **State Management:**

Provider and setState were used for small-scale state management; large states (e.g., profile data) are cached locally using Shared Preferences and Cubit.

- **API Integration:**

The mobile app connects to the backend using HTTP requests and handles JSON responses.

- **Localization:**

The app supports English interfaces.

- **Form Validation & Error Handling:**

All input forms include client-side validation and error messages for better UX.

- **Object Color Detection Modul**

This module was developed using Flutter to determine the colors of objects in user-uploaded images. The system analyzes image pixels to identify the dominant color of each object.

- **AI Chatbot Integration**

ColorMate includes an AI chatbot powered by the Gemini API. The chatbot assists users with outfit recommendations, color matching suggestions, and application-related questions through a real-time conversational interface.

- **Educational Mini-Games Module**

The application provides educational mini-games designed to help children learn and recognize colors. These interactive activities improve color identification skills while creating an engaging learning experience.

6.11 System Modules Description

Each module operates as an independent unit:

- Authentication Module
 - Color Blindness Test Module (Ishihara Test) Module
 - Object & Color Detection Module
 - Outfit Rating Module
 - Fruit Freshness Detection Module
 - AI Chatbot Module
-

6.12 Security Implementation

Security features include:

- JWT Token-Based Authentication
 - Refresh Tokens
 - HTTPS Enforcement
 - CORS Configuration
-

6.13 Deployment & Hosting

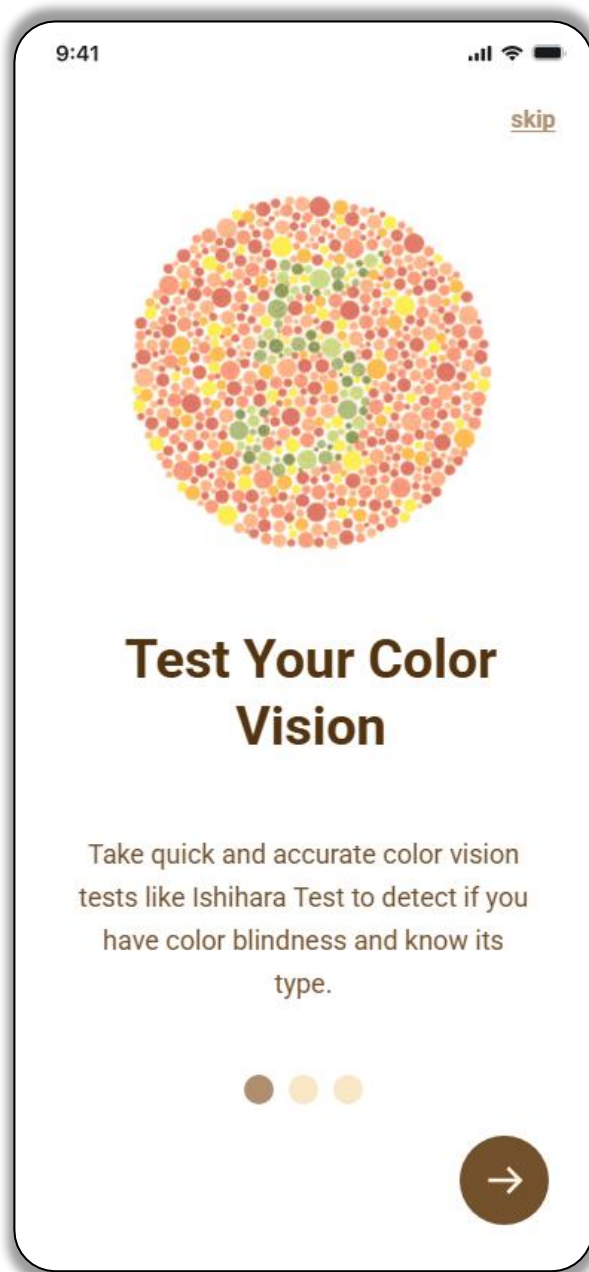
- ASP.NET API, SQL Server and Python AI services deployed in Monster
 - Mobile app is deployed via APK.
 - OpenAI API keys and sensitive configs stored securely using protected environments.
-

6.14 Application Interface

This section presents the graphical user interface of the ColorMate mobile application. The screenshots demonstrate the main screens and features available to users and illustrate how the application supports color analysis, accessibility, outfit recommendations, and educational activities.

- **Application Intro Screens**

The introductory screens are displayed when the application is launched for the first time. These screens provide a brief overview of the application's objectives and key features, helping users understand the services offered by ColorMate before proceeding to the main application as shown in Figure 6:20.



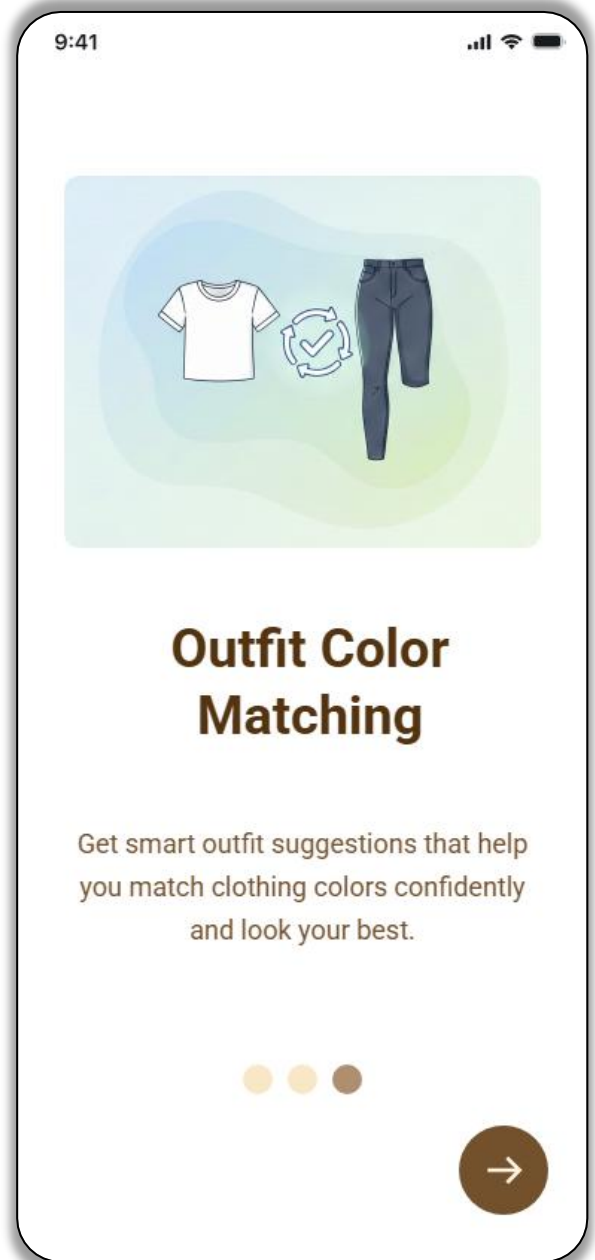
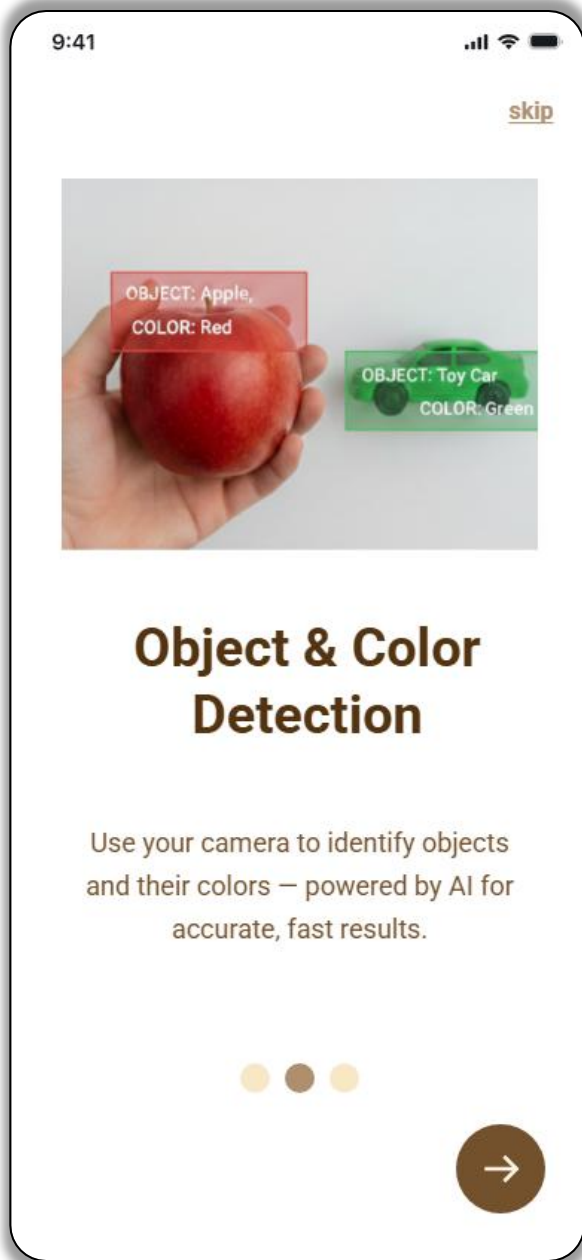
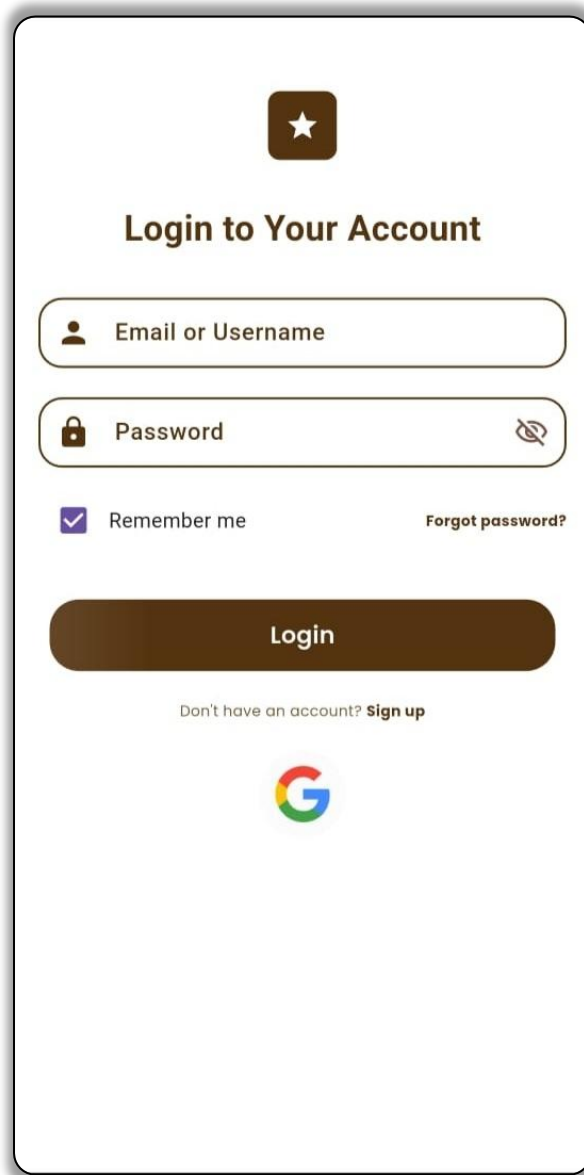


Figure 6.20: Application Intro Screens.

- **Authentication Screens**

The authentication interface allows users to create a new account, log in securely, and access personalized application features. These screens provide user-friendly forms for account registration and authentication.

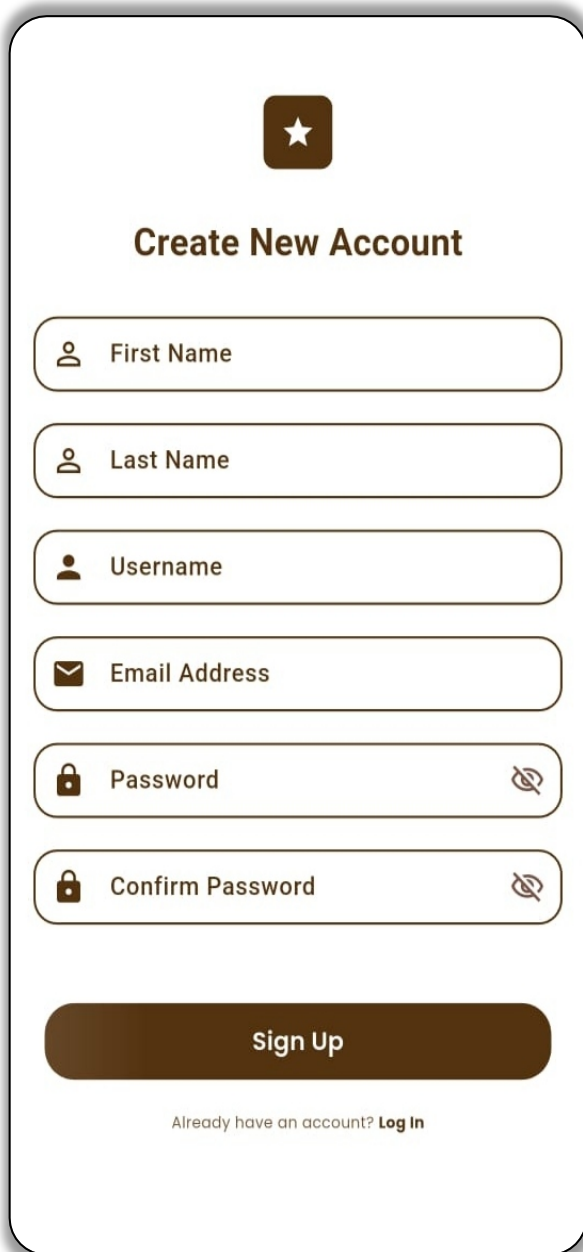
- Login Screen



A mobile app login screen mockup. At the top is a dark blue square icon with a white star. Below it is the title "Login to Your Account" in bold black text. There are two input fields: the first is labeled "Email or Username" with a person icon, and the second is labeled "Password" with a lock icon and a toggle eye icon. Below the password field are two links: "Remember me" with a checked checkbox and "Forgot password?". A large blue rounded button labeled "Login" is centered below these links. At the bottom, there is a link "Don't have an account? Sign up" and a Google Assistant-style "G" logo.

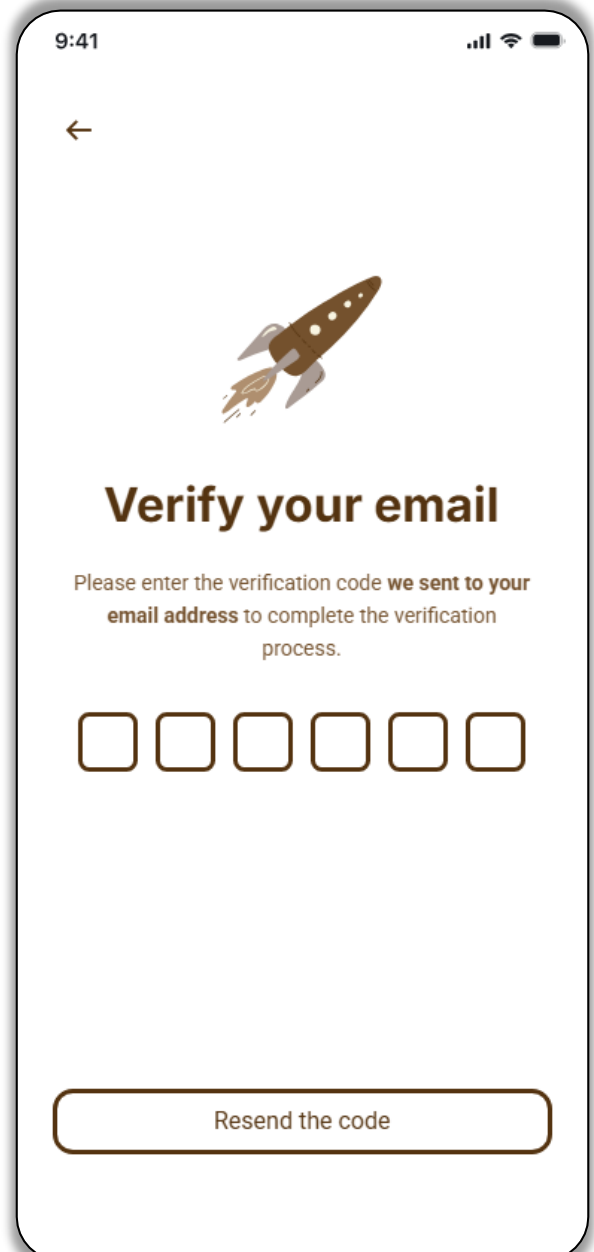
Figure 6.21: Login Screen.

- Register Screens



The Sign-Up screen features a dark blue star icon at the top. Below it, the title "Create New Account" is centered. The form consists of six input fields, each with a dark blue icon on the left: "First Name" (person icon), "Last Name" (person icon), "Username" (person icon), "Email Address" (envelope icon), "Password" (lock icon), and "Confirm Password" (lock icon). Each password field has a dark blue eye icon on the right to toggle visibility. At the bottom, there is a large dark blue "Sign Up" button and a link "Already have an account? Log In" in dark blue text.

Figure 6.22: Sign-Up Screen.



The OTP Verify screen shows the time "9:41" and signal/battery status at the top. A back arrow is in the top left. A rocket icon is centered above the title "Verify your email". Below the title, a message reads: "Please enter the verification code we sent to your email address to complete the verification process." There are six empty square boxes for the code. At the bottom is a "Resend the code" button.

Figure 6.23: OTP Verify Screen.

- Forget Password Screen

9:41



Forgot Password?

Don't worry! Enter your email address and we'll send you a link to reset your password.

Email Address

Send reset link

Remembered your password? [Login](#)

Figure 6.24: Forget Password Screen.

- **Home Screen**

The home screen serves as the central navigation point of the application. It provides quick access to all major features, including outfit analysis, color blindness tools, color detection, chatbot assistance, and educational games.

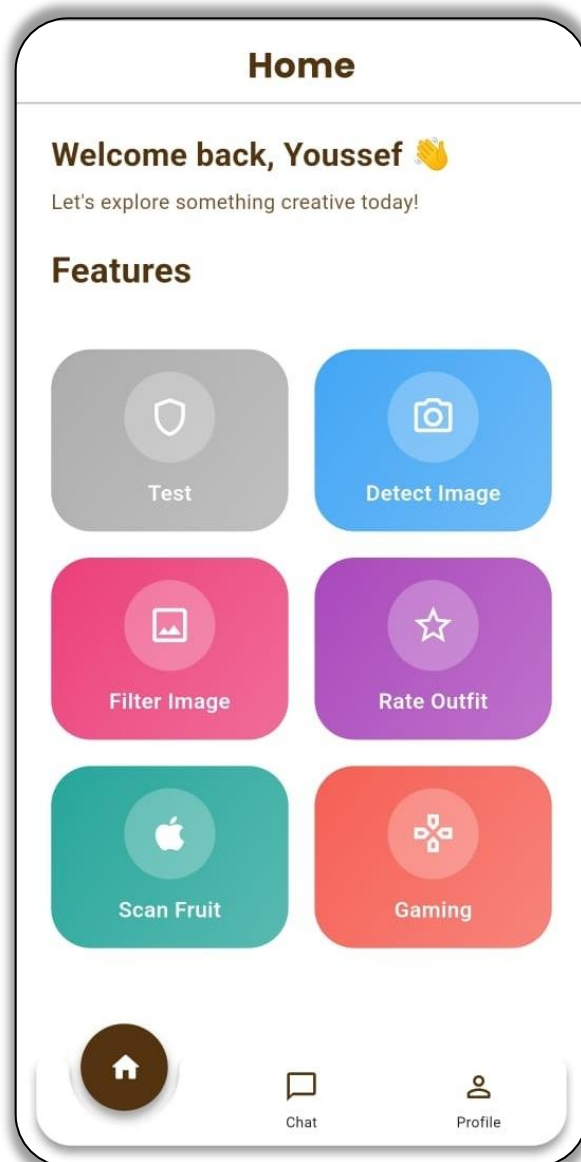


Figure 6.25: Home Screen.

• Color Blindness Test Screens

The color blindness test feature is designed to help users determine whether they have a color vision deficiency and identify its type. Based on the user's responses, the system analyzes the results and indicates whether the user has normal color vision or a specific type of color blindness, such as Protanopia, Deuteranopia, or Tritanopia.

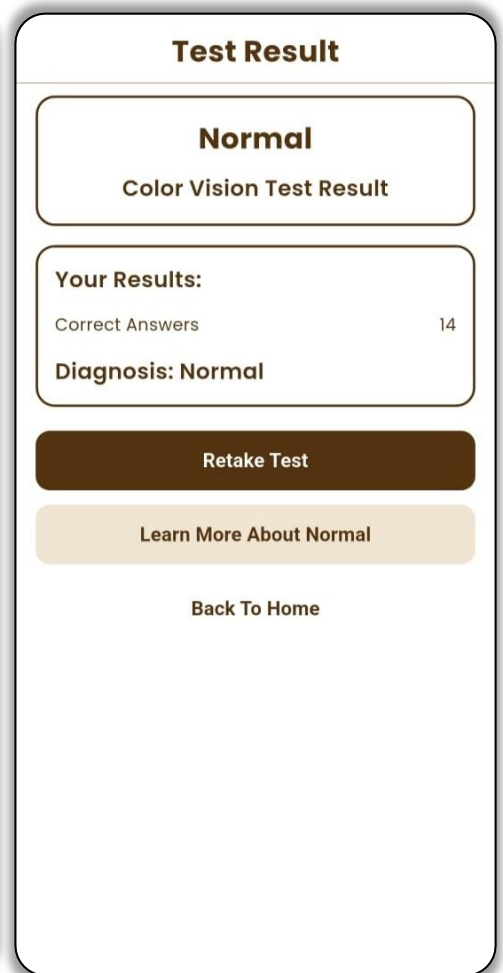
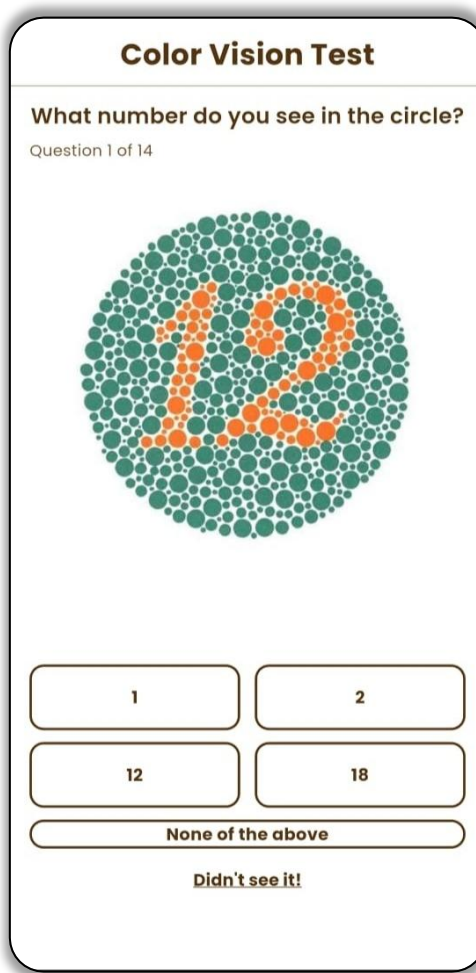
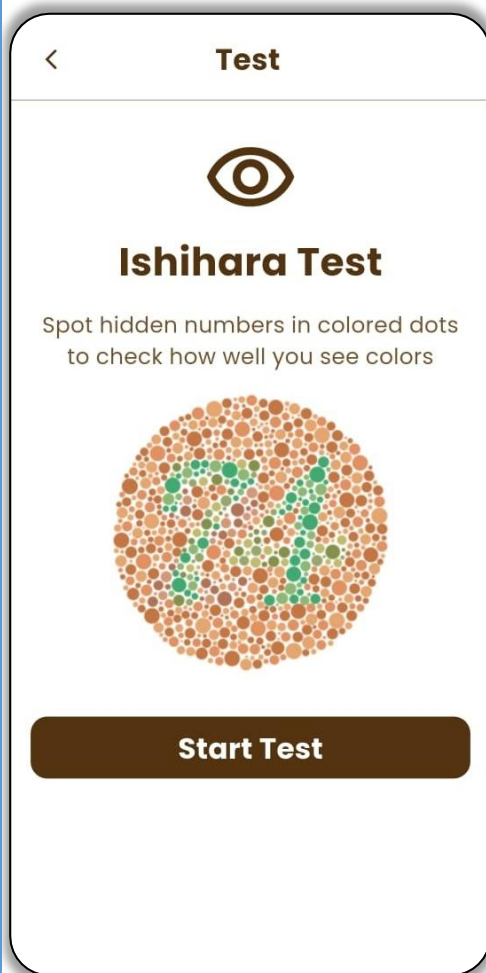


Figure 6.26: Start Test Screen.

Figure 6.27: Test Screen.

Figure 6.28: Test Result Screen.

• Object Color Detection Screens

The object color detection feature enables users to upload an image and automatically identify the objects present within it. After analyzing the image, the system detects each object and determines its corresponding color.



Figure 6.29: Upload Image Screen.

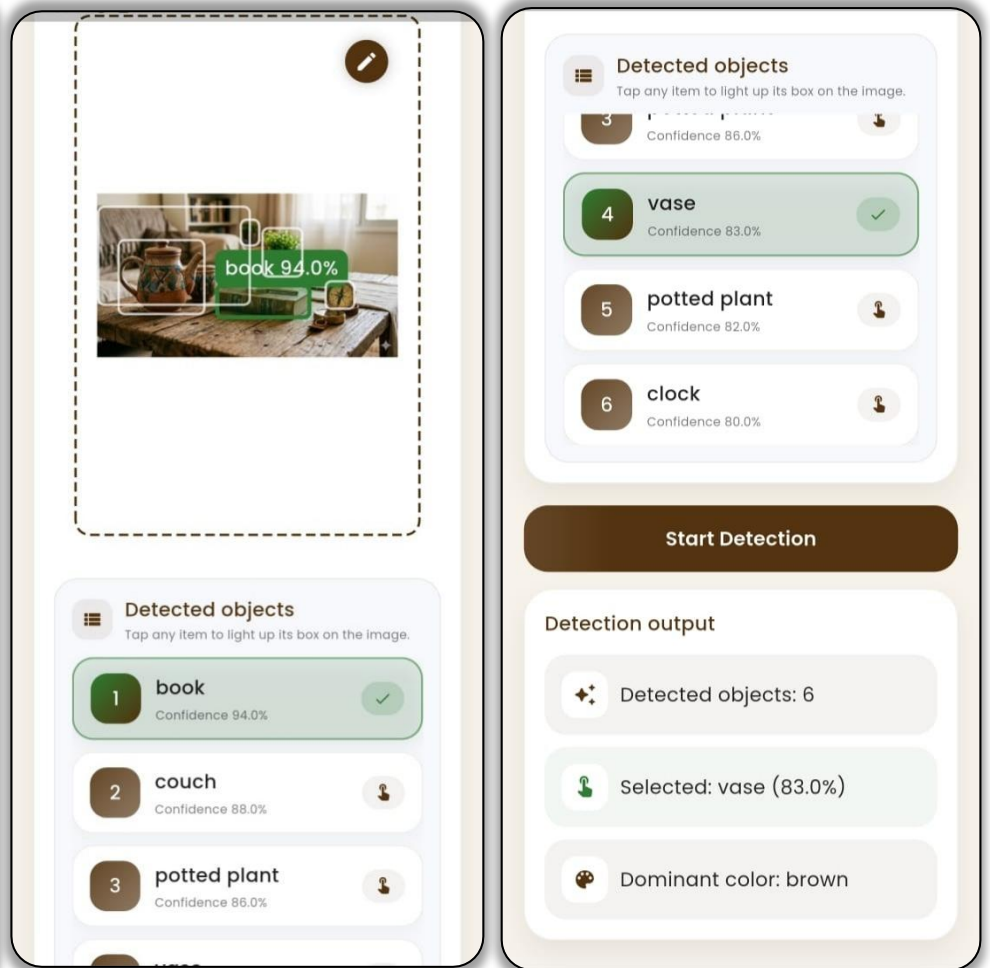


Figure 6.30: Object & Color Detection Result Screens.

• Outfit Analysis and Rating Screens

These screens allow users to upload outfit images and receive compatibility evaluations. The interface displays the analyzed outfit along with the generated rating and feedback provided by the recommendation system.

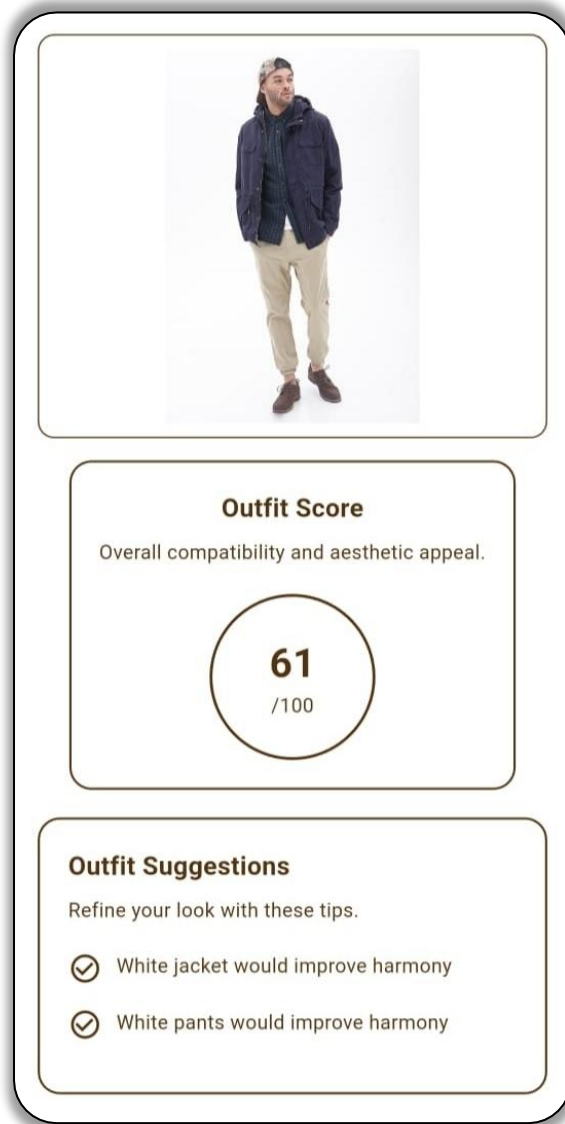


Figure 6.31: Upload Outfit Image and The Color Matching Result Screen.

- **Fruit Scanning Screens**

The fruit scanning feature enables users to capture or upload images of fruits and determine whether they are fresh or rotten based on their color characteristics. The interface analyzes the fruit image and displays the freshness status along with the corresponding result, helping users assess fruit quality quickly and easily.



Figure 6.32: Upload Fruit Image Screen.

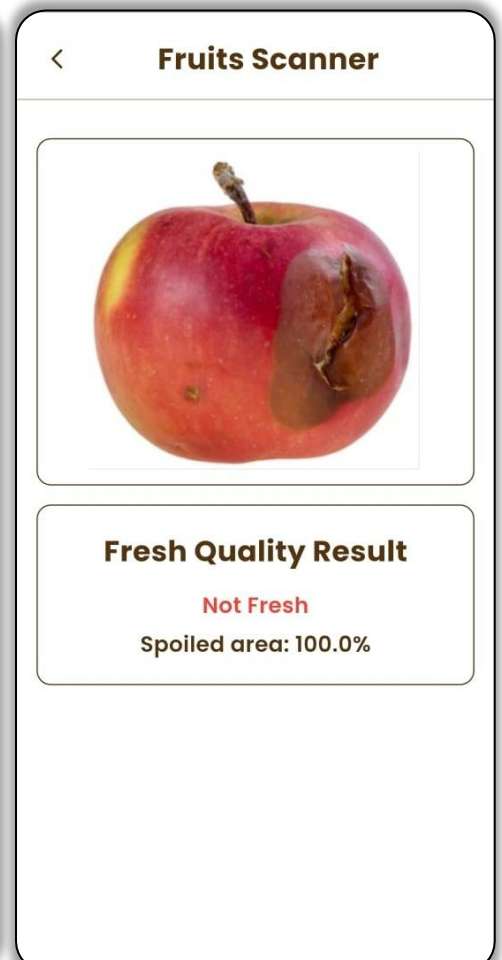
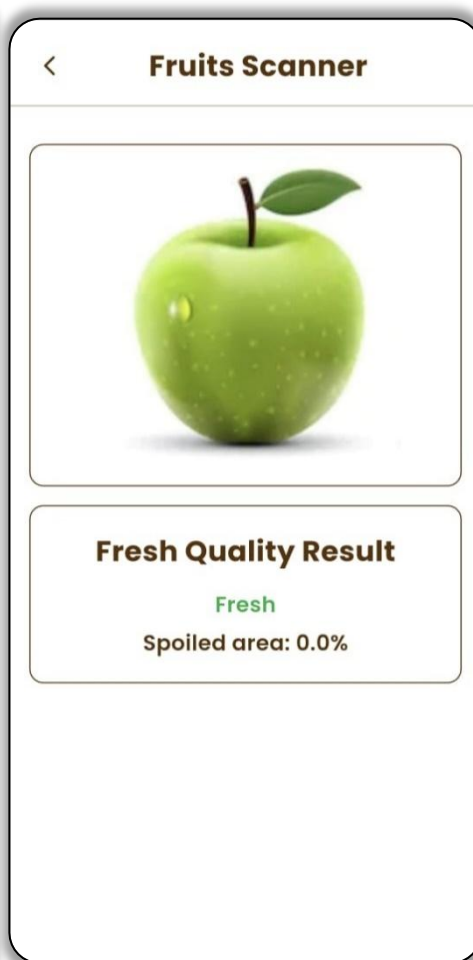


Figure 6.33: Fruit Quality Check Screen.

- **AI Chatbot Screens**

The chatbot interface enables users to communicate with an AI assistant powered by the Gemini API. Users can ask questions related to colors, fashion, outfit recommendations, and application usage through a conversational interface.

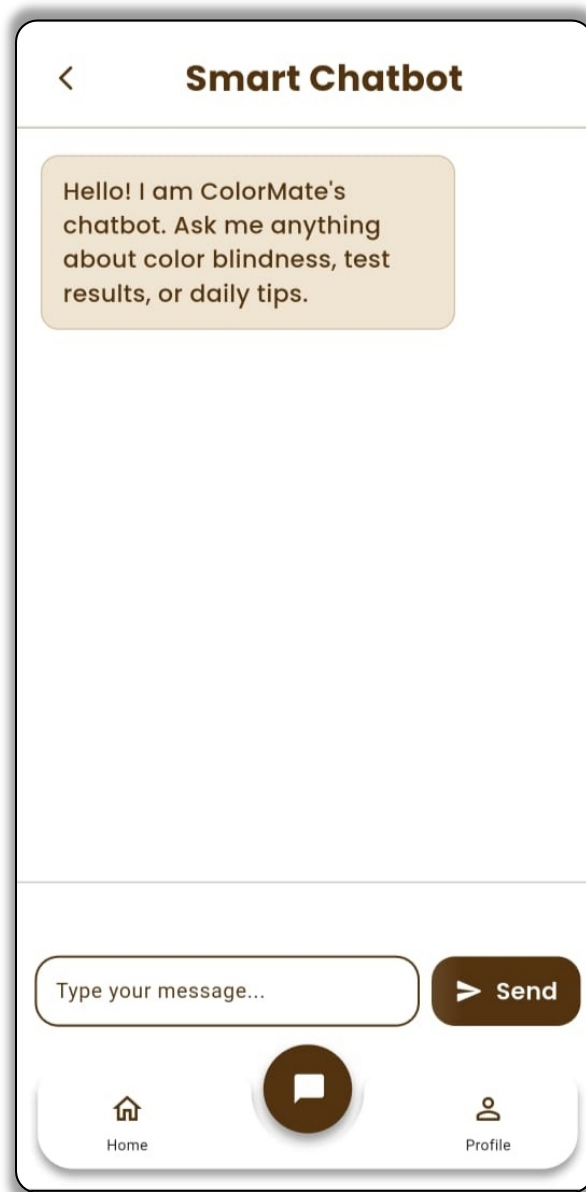


Figure 6.34: Chatbot Chatting Screen.

• User Profile Screens

The profile interface allows users to manage their personal information. It also provides access to saved preferences and account-related features.

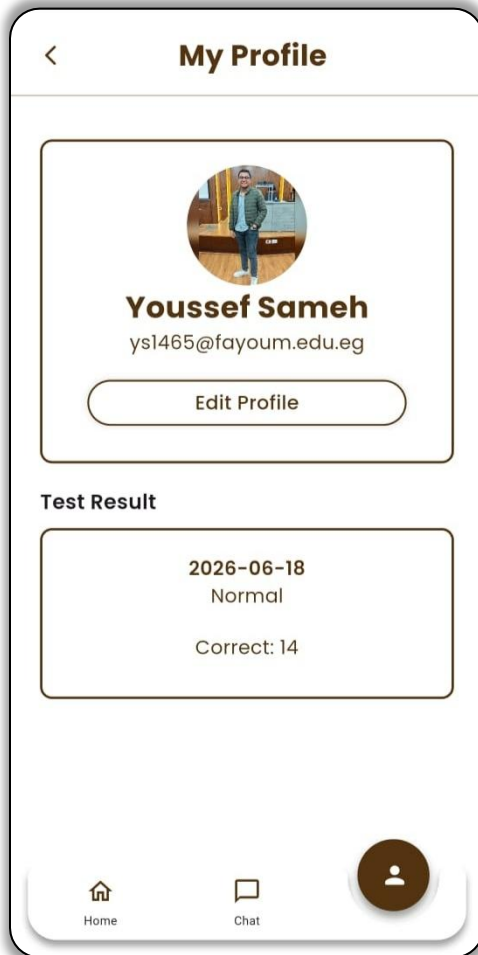


Figure 6.35: User Profile Screen.

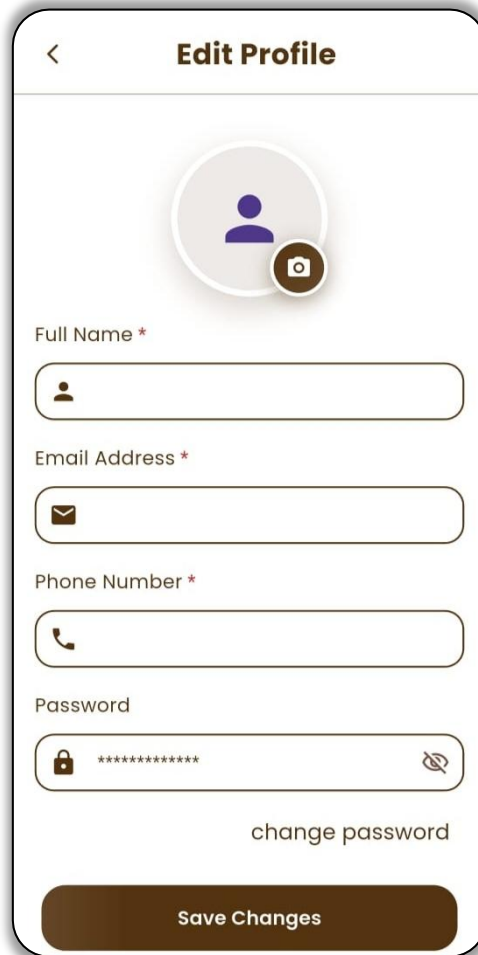


Figure 6.36: Edit Profile Screen.

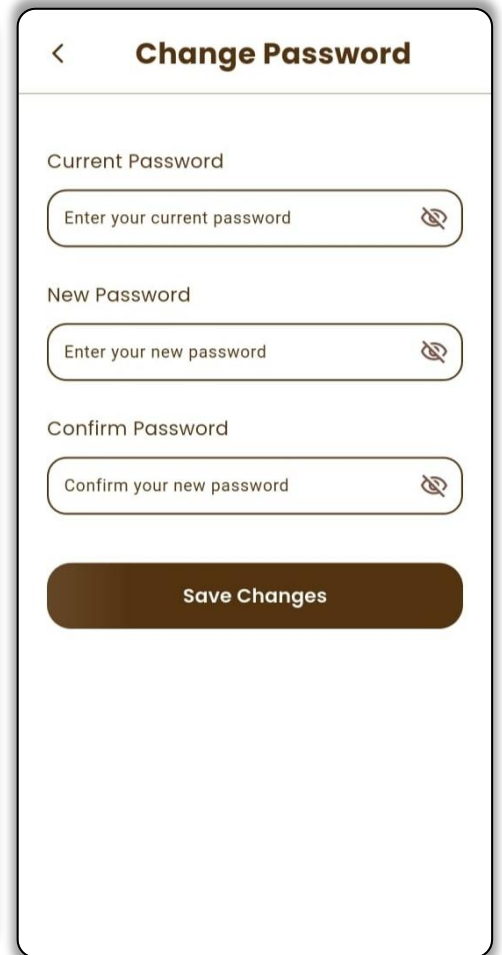


Figure 6.37: Change Password Screen.

- **Color Blindness Simulation and Correction Screens**

These screens provide both color blindness simulation as shown in Figure 6:38 and correction functionalities as shown in Figure 6:36. Users can upload images to simulate different types of color vision deficiencies, including Protanopia, Deuteranopia, and Tritanopia, allowing them to understand how images appear to individuals with color blindness. The interface also applies daltonization techniques to improve color distinguishability for color-deficient users and displays both the original and corrected images to demonstrate the effectiveness of the correction process.

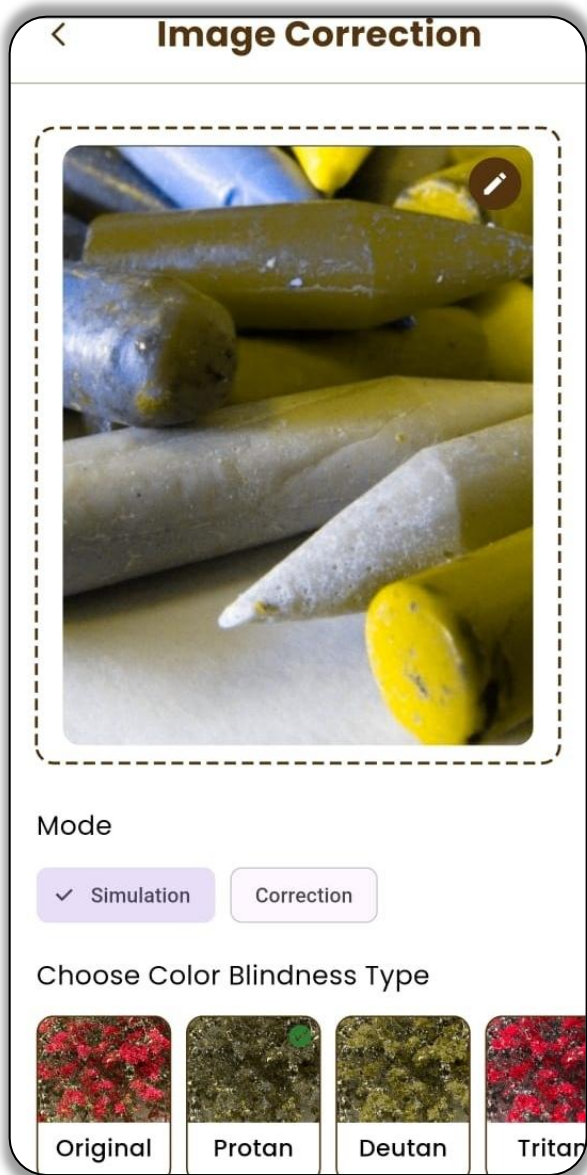


Figure 6.38: Simulated Image in Protan Screen.

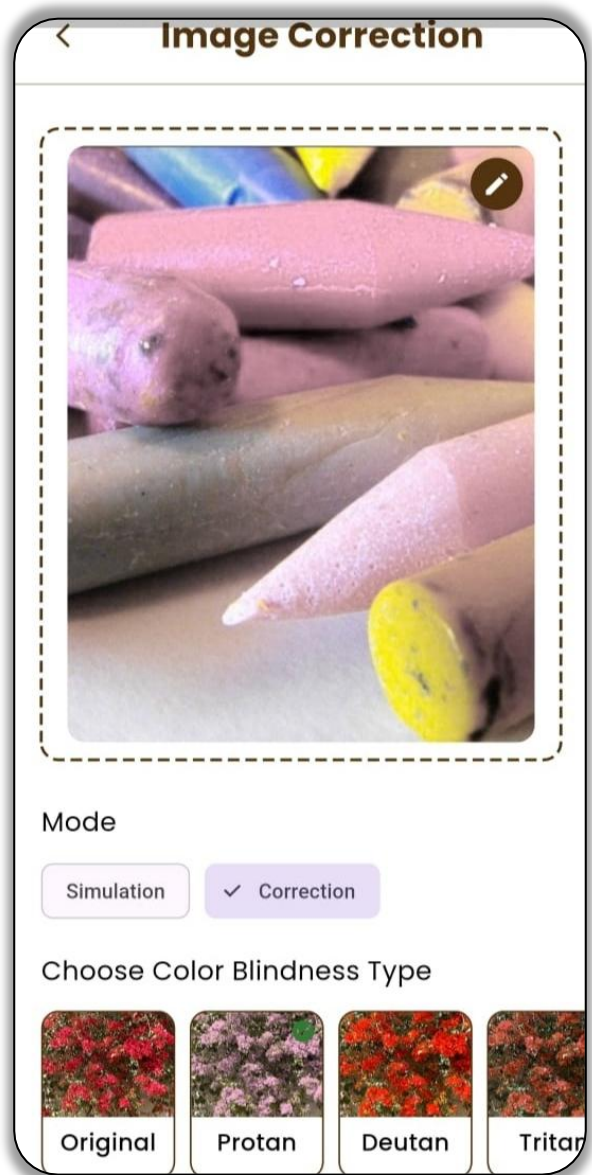


Figure 6.39: Corrected Image from Protan Screen.

• Educational Mini-Games Screens

The educational games module provides interactive activities designed to help children learn and recognize colors. These screens present various color-learning challenges in an engaging and user-friendly format.

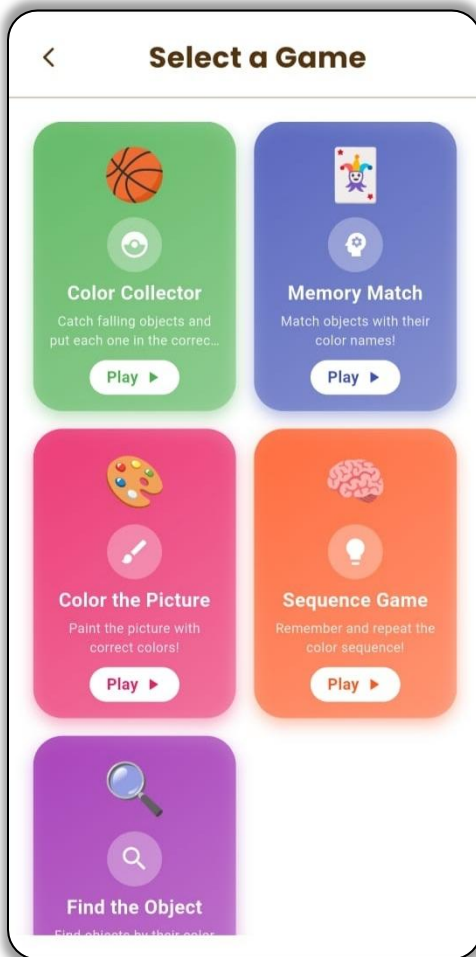


Figure 6.40: Select Game Screen.

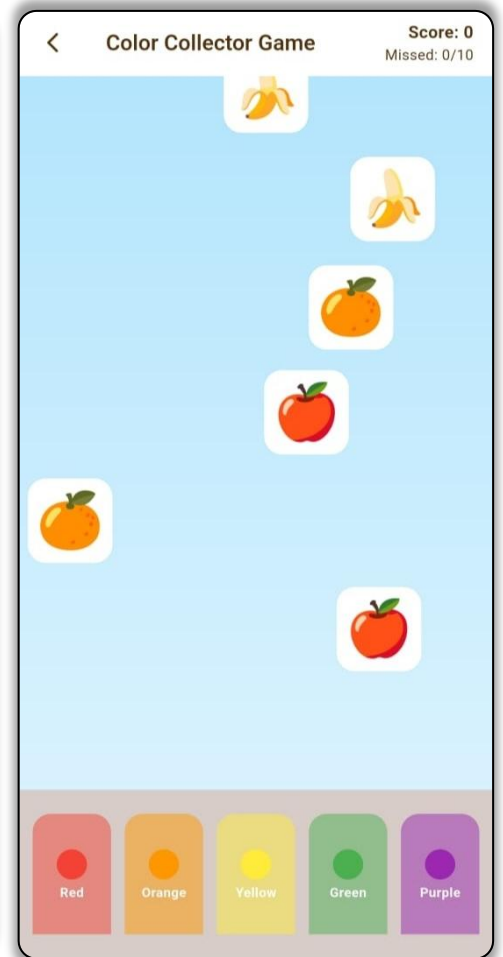
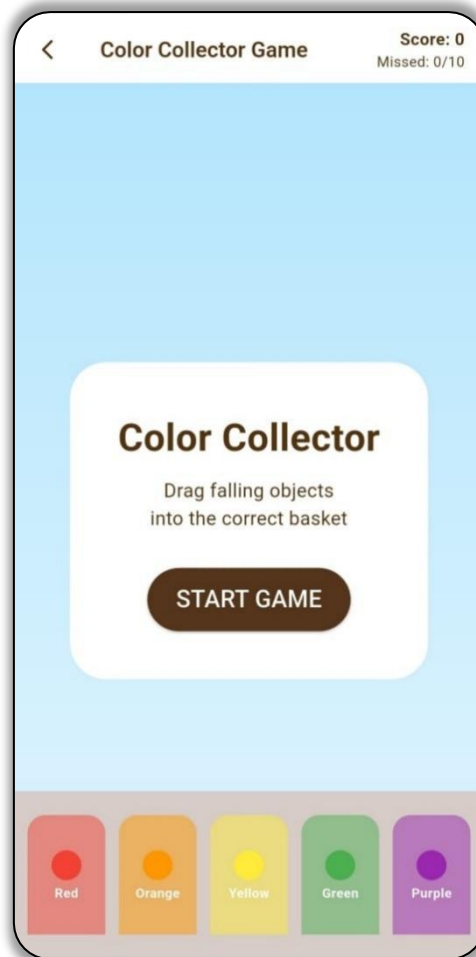


Figure 6.41: Color Collector Game Screens.

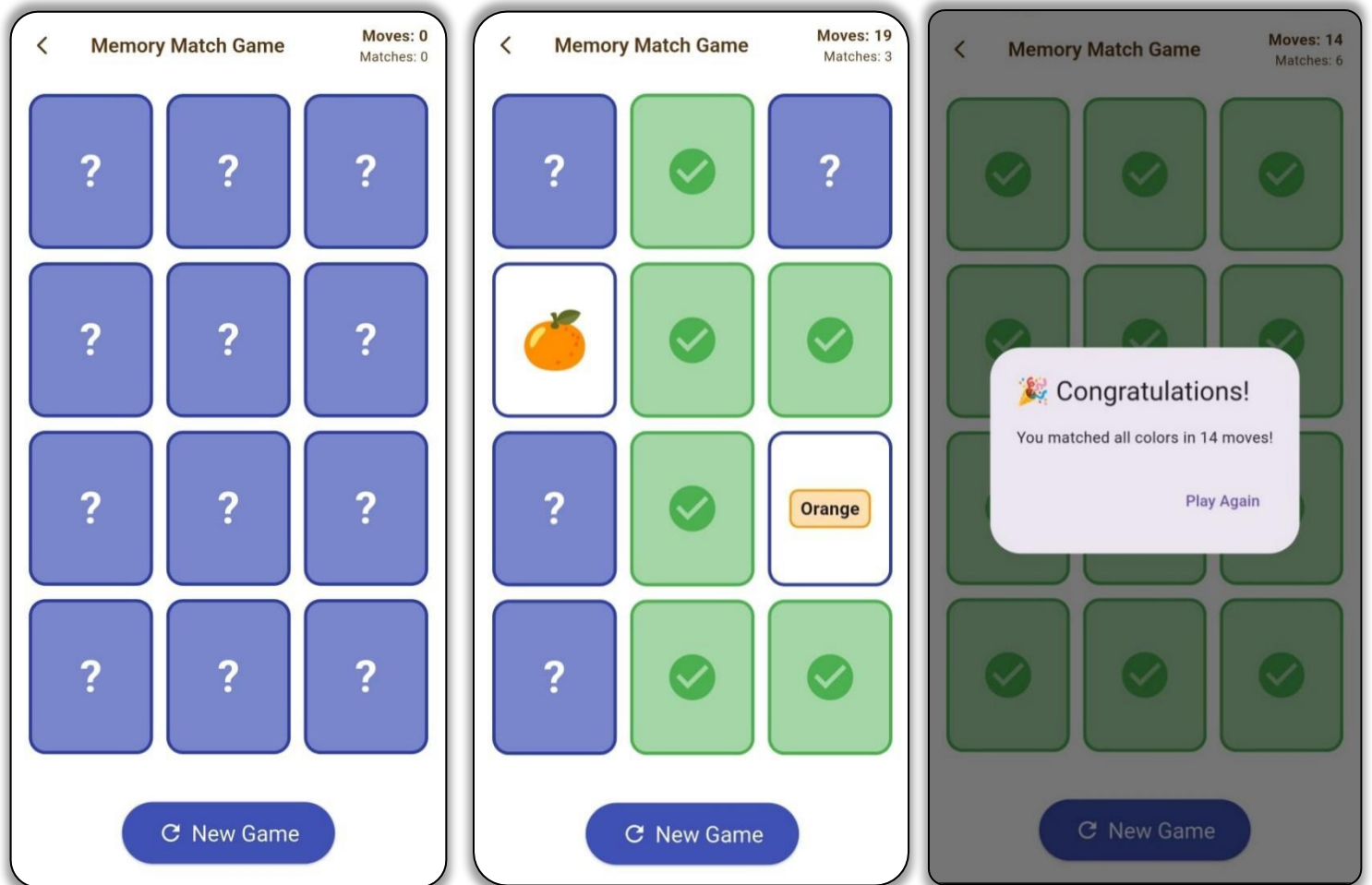


Figure 6.42: Memory Match Game Screens.

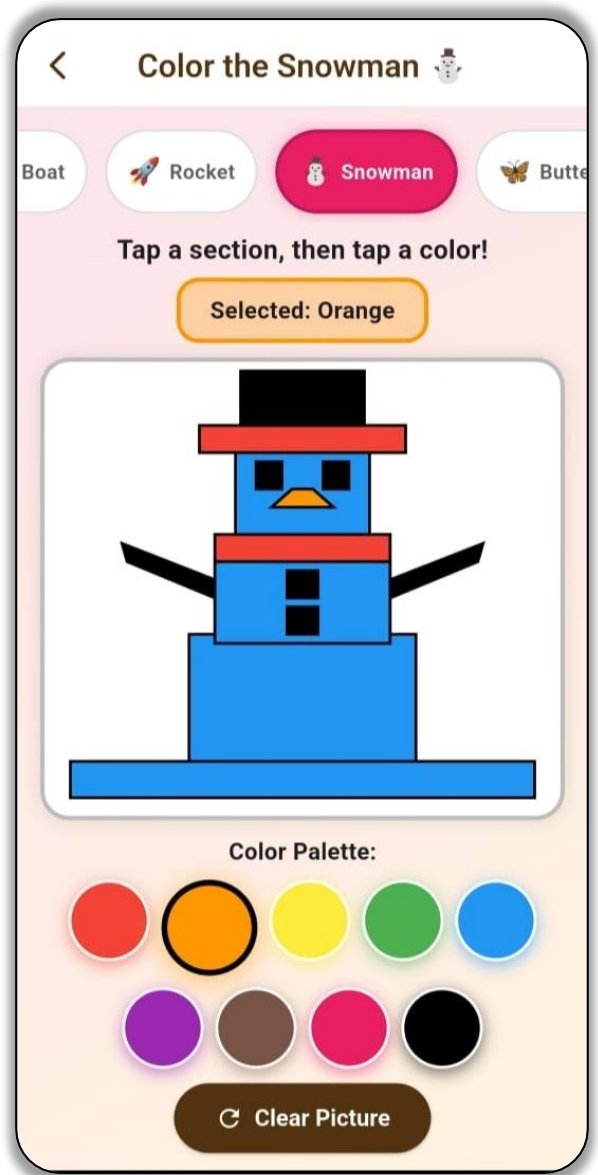
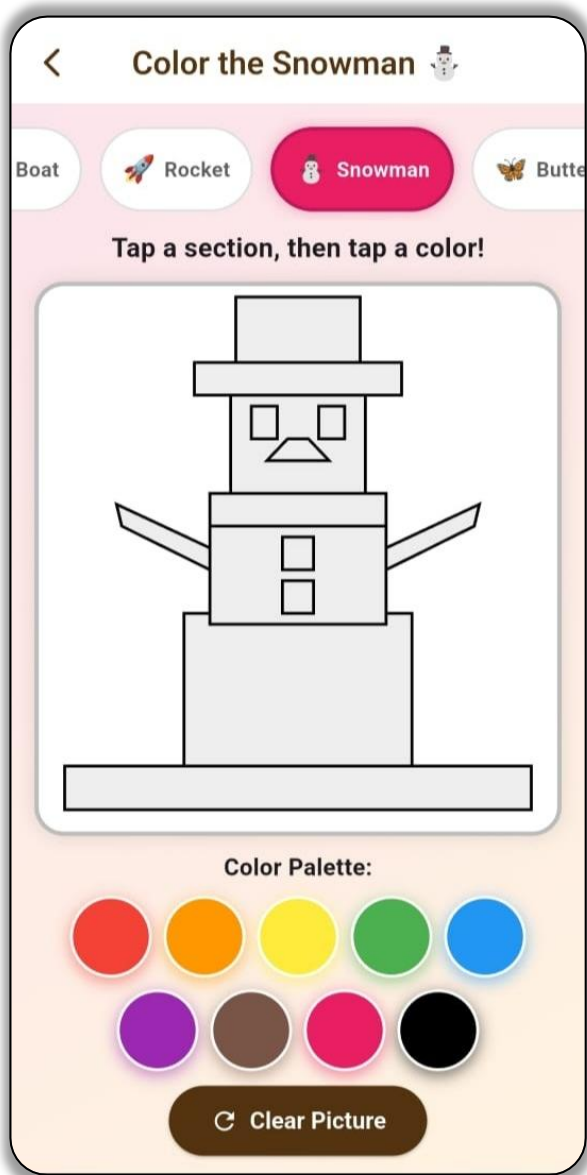


Figure 6.43: Color the Picture Game Screens.

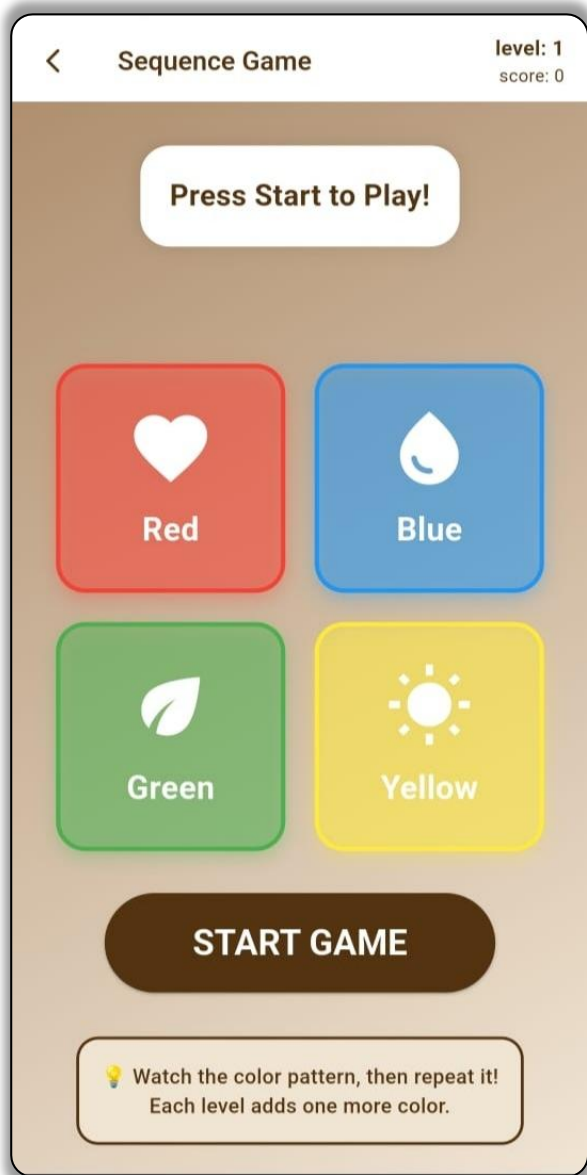


Figure 6.44: Sequence Game Screen.

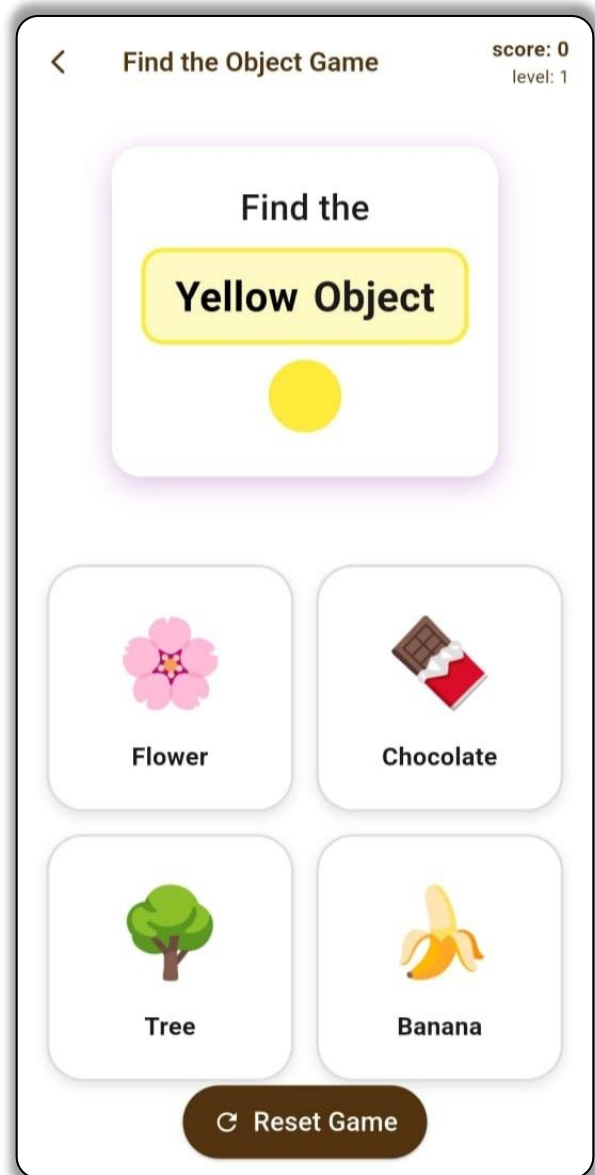


Figure 6.45: Find the Object Game Screen.

6.15 Testing and Debugging

Testing methods used:

- Unit Testing for APIs
 - Manual Testing on Android Emulator & Devices
 - AI Model Validation
 - AI features were tested using:
 - Predefined question sets for the chatbot to verify response accuracy and relevance.
 - Sample images with different objects and colors to test detection and color naming performance.
 - Various outfit images to evaluate color harmony scoring and recommendation quality.
 - Fruit images (fresh and rotten) to validate the accuracy of the freshness detection model.
 - Performance & Load Testing
-

6.16 Challenges During Implementation

- Handling Image Processing on AI models
 - Ensuring Low-Latency Communication Between AI Services and Mobile App
 - Training and Optimizing AI Models for Accuracy
 - Integrating Multiple AI Features Seamlessly
-

6.17 Conclusion

This chapter explored the detailed steps of implementing ColorMate. From backend API development to mobile application integration, and from database design to deployment of AI modules, every component was carefully structured to ensure performance, maintainability, and scalability. The modular implementation, combined with modern technologies such as Flutter, ASP.NET Core, and artificial intelligence, enabled the system to deliver accurate and efficient features including color detection, correction, and analysis. As a result, ColorMate represents a smart and reliable assistive application that enhances the daily experience of individuals with color vision deficiencies.

Conclusion & Future Work

Chapter seven

7.1 Conclusion

ColorMate represents a significant step toward improving the daily experience of individuals with color vision deficiencies by providing an intelligent and personalized assistive solution. By integrating advanced AI technologies — including deep learning-based image classification, perceptual color harmony evaluation, and real-time object detection — the system not only detects color vision issues but actively helps users adapt to their environment.

The centerpiece of the AI pipeline is the Fruit Freshness Classification Model, built on the MobileNetV3Large architecture and trained using a structured two-phase transfer learning strategy. In Phase 1, only the custom classification head was trained while the pretrained base was frozen, achieving a validation accuracy of 93.98%. In Phase 2, all 187 layers were fine-tuned end-to-end at a reduced learning rate, pushing the **final test accuracy to 95.12%** with a **precision of 0.9523, recall of 0.9512, F1-score of 0.9511**, and an **AUC of 0.9781**. When benchmarked against the reference paper model, **ColorMate's** fruit freshness module outperformed it by **+0.98%**, raising the classification **accuracy from 94.14% to 95.12%**. This improvement is attributed to the two-phase fine-tuning approach, structured regularization through L2 penalties, dropout layers, and label smoothing, and the careful application of class weights to handle dataset imbalance.

Alongside the fruit model, the **Perceptual Color Harmony System (PCHS)** provides outfit color evaluation and recommendations, while the **YOLOv8**-based object detection module enables real-time localization of items before color analysis. These three AI modules work together as an integrated pipeline that covers detection, quality assessment, and color compatibility in a single system.

The broader application further provides an Ishihara-based color blindness test, image color correction filters, and an AI-powered chatbot, all wrapped in a cross-platform mobile interface built with **Flutter**, **ASP.NET Core**, and **SQL Server**. Through **ColorMate**, we addressed several limitations found in existing

applications, particularly the lack of solutions that combine detection, correction, education, and real-world assistance in one platform.

7.2 Future Work

While ColorMate already offers a comprehensive set of features, several areas can be further improved and expanded in future development:

1. Multilingual Support

Implementing support for multiple languages to make the application more accessible to users from different regions and backgrounds.

2. Improved AI Model Accuracy

Expanding training datasets and applying advanced deep learning techniques to improve the accuracy of color detection, outfit analysis, and fruit classification under different lighting conditions.

3. Real-Time Object and Color Detection

Enhancing the system to support real-time video-based object and color detection using the device camera, allowing users to receive continuous visual assistance in dynamic environments.

4. Augmented Reality (AR) Integration

Incorporating AR technology to overlay color information and object labels directly onto the camera view, providing a more interactive and intuitive user experience.

5. Community Support and Experience Sharing

Developing a community support feature that allows users with color vision deficiencies to share experiences, practical tips, and daily challenges within the application. This feature can help users learn from each other, exchange advice about handling color-related situations, and create a supportive environment that

promotes awareness, confidence, and social connection among individuals with color blindness.

6. Interactive Color Learning Encyclopedia for Children

Developing a dedicated educational section that contains a comprehensive collection of colors along with their names, visual examples, and real-world objects associated with each color. The section would help children learn and recognize colors through interactive content, illustrations, and educational activities, making ColorMate a more engaging learning platform for young users.

By continuously evolving ColorMate, we aim to provide a smarter, more inclusive, and highly effective assistive solution that improves independence and quality of life for individuals with color vision deficiencies.

References:

[1] Colour Blind Awareness Orgnization

<https://www.colourblindawareness.org/colour-blindness/> (Available online; last accessed January 2026)

[2] App ColorADD

https://play.google.com/store/apps/details?id=com.coloradd.app&pcampaignid=web_share (Available online; last accessed February 2026)

[3] Color Blind Test App

<https://play.google.com/store/apps/details?id=com.thulotechnology.colorblind>
(Available online; last accessed February 2026)

[4] Color Blind Pal

<https://play.google.com/store/apps/details?id=com.colorblindpal.colorblindpal>
(Available online; last accessed February 2026)

[5] COCO 2017 Dataset

<https://www.kaggle.com/datasets/awsaf49/coco-2017-dataset>
(Available online; last accessed February 2026)

[6] Deep Fashion Images

<https://github.com/yumingj/DeepFashion-MultiModal?tab=readme-ov-file>
(Available online; last accessed February 2026)

- [7] **Fruit and Vegetable Disease (Healthy vs Rotten)**
<https://www.kaggle.com/datasets/muhammad0subhan/fruit-and-vegetable-disease-healthy-vs-rotten> (Available online; last accessed February 2026)
- [8] **H.-C. Li, L.-K. Wang, Y.-K. Chang, and K.-Y. Huang, “Establishing colour harmony evaluation and recommendation model for clothing colour matching based on machine learning and deep learning,” *Fashion and Textiles*, vol. 12, art. no. 27, 2025.**
- [9] **M. Adithya Vardhan, Arjun Maddipatla, and M. S. J. Sarojini Premalatha, “Color Harmony Analysis and Outfit Recommendation,” *International Journal of Engineering Research in Computer Science and Engineering (IJERCSE)*, vol. 12, no. 03, pp. 89-??, Mar. 2025, Online ISSN: 2394-2320.**
- [10] **S. Gulati, “Fashion recommendation: outfit compatibility using GNN,” arXiv preprint arXiv:2404.18040, 2024.**
- [11] **G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8: Supported tasks and modes,” *Ultralytics YOLO Docs*, 2023. [Online]. Available: <https://docs.ultralytics.com/models/yolov8/>.**
- [12] **S. Aksoy, P. Demircioglu, and I. Bogrekci, “Evaluating pre-trained CNNs for distinguishing fresh vs rotten fruits and vegetables,” *Journal of Applied Horticulture*, vol. 26, no. 3, pp. 361–366, 2024**
- [13] **H. Zhang, X. Yang, J. Tan, C.-H. Wu, J. Wang, and C.-C. J. Kuo, “Learning color compatibility in fashion outfits,” arXiv preprint arXiv:2007.02388, 2020.**

- [14] **G. M. Machado, M. M. Oliveira, and L. A. F. Fernandes, “A Physiologically-Based Model for Simulation of Color Vision Deficiency,” IEEE Transactions on Visualization and Computer Graphics, vol. 15, no. 6, pp. 1291–1298, Nov.–Dec. 2009, doi: 10.1109/TVCG.2009.113.**
- [15] **J. R. B. Ferreira, M. M. Oliveira, and L. A. F. Fernandes, “Evaluating Color Vision Deficiency Daltonization Methods Using a Behavioral Visual-Search Method,” ACM Symposium on Applied Perception, pp. 63–70, 2014, doi: 10.1145/2628257.2628259.**
- [16] **Flutter:** <https://docs.flutter.dev/> (Available online; last accessed February 2026)
- [17] **Asp.Net Core Framework:**
<https://learn.microsoft.com/en-us/aspnet/core/?view=aspnetcore-10.0>
(Available online; last accessed February 2026)
- [18] **Convolutional Neural Networks:**
<https://developer.nvidia.com/discover/convolutional-neural-network>
(Available online; last accessed February 2026)
- [19] **YOLO (You Only Look Once) – YOLOv8**
<https://docs.ultralytics.com/models/yolov8/>
(Available online; last accessed February 2026)
- [20] **Python**
<https://www.python.org/> (Available online; last accessed February 2026)

- [21] **PyTorch**
<https://pytorch.org/> (Available online; last accessed February 2026)
- [22] **NumPy**
<https://numpy.org/> (Available online; last accessed February 2026)
- [23] **OpenCV**
<https://opencv.org/> (Available online; last accessed February 2026)
- [24] **PIL (Pillow)**
<https://pillow.readthedocs.io/en/stable/> (Available online; last accessed February 2026)
- [25] **Matplotlib**
<https://matplotlib.org/> (Available online; last accessed February 2026)
- [26] **S. Ishihara, *Ishihara's Tests for Color Deficiency: Concise Edition*, Tokyo, Japan: Kanehara & Co., Ltd.**